

UNIVERSITATEA TEHNICĂ „Gheorghe Asachi” din IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DOMENIUL: Calculatoare și Tehnologia Informației
SPECIALIZAREA: Tehnologia Informației

LUCRARE DE DIPLOMĂ

Coordonator științific:
Ș.l.dr.ing. Cristian-Nicolae BUTÎNCU

Absolvent:
Mădălina-Elena BOACĂ

Iași, 2024

UNIVERSITATEA TEHNICĂ „Gheorghe Asachi” din IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DOMENIUL: Calculatoare și Tehnologia Informației
SPECIALIZAREA: Tehnologia Informației

***Platformă distribuită pentru executarea
serviciilor în cadrul unui cluster***

LUCRARE DE DIPLOMĂ

Coordonator științific:
Ș.l.dr.ing. Cristian-Nicolae BUȚINCU

Absolvent:
Mădălina-Elena BOACĂ

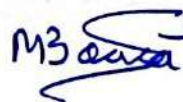
Iași, 2024

**DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII
PROIECTULUI DE DIPLOMĂ**

Subsemnatul Bosca Madalina - Elena,
legitimată cu CI seria NZ nr. 049443, CNP 6010524271714
autorul lucrării Platformă distribuită pentru execuția
serviciilor în cadrul unui cluster

elaborată în vederea susținerii examenului de finalizare a studiilor de licență, programul
de studii TI organizat de către Facultatea de
Automatică și Calculatoare din cadrul Universității Tehnice „Gheorghe Asachi” din
Iași, sesiunea iulie a anului universitar 2023-2024, luând în
considerare conținutul Art. 34 din Codul de etică universitară al Universității Tehnice
„Gheorghe Asachi” din Iași (Manualul Procedurilor, UTI.POM.02 - Funcționarea
Comisiei de etică universitară), declar pe proprie răspundere, că această lucrare este
rezultatul propriei activități intelectuale, nu conține porțiuni plagiate, iar sursele
bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) și a
convențiilor internaționale privind drepturile de autor.

Data
28.06.2024

Semnătura


Cuprins

Introducere	1
1 Fundamentarea teoretică și documentarea bibliografică	3
1.1 Realizări actuale privind execuția distribuită a serviciilor	3
1.2 Referințe la teme/subiecte similare	4
1.2.1 Mecanisme de descoperire a nodurilor	4
1.2.2 Mecanisme de echilibrare a încărcării	6
1.2.3 Mecanisme de toleranță la erori	7
1.2.4 Mecanisme de securitate	7
1.2.5 Mecanisme de jurnalizare în sisteme distribuite	9
1.3 Utilizarea clusterelor de calculatoare pentru executarea serviciilor	10
1.4 Metode de execuție remote a codului	11
1.4.1 Executarea directă pe sistemul gazdă	11
1.4.2 Executarea în containere Docker	11
1.4.3 Încărcarea dinamică în programul gazdă	12
2 Proiectarea aplicației	13
2.1 Prezentare generală	13
2.2 Aplicații client	17
2.3 Servicii agregate	18
2.4 Componenta master	22
2.5 Componenta de execuție de cod	24
2.6 Componenta de jurnalizare	25
2.7 Baze de date	26
3 Implementarea aplicației	29
3.1 Aplicații client	29
3.2 Servicii agregate	30
3.3 Componenta master	32
3.4 Componenta de execuție de cod	34
3.5 Componenta de jurnalizare	35
3.6 Baze de date	36
4 Testarea aplicației și rezultate experimentale	37
4.1 Lansarea aplicației și elemente de configurare	37
4.2 Testarea sistemului	37
4.3 Aspecte legate de încărcarea procesorului, memoriei și timpilor de execuție	40
4.4 Aspecte legate de securitate și fiabilitate	47
Concluzii	49

Bibliografie	51
---------------------	-----------

Anexe	55
--------------	-----------

1	Diagrame de clase	55
2	Secvențe de cod	63
3	Grafice suplimentare	68

Platformă distribuită pentru executarea serviciilor în cadrul unui cluster

Mădălina-Elena BOACĂ

Rezumat

În contextul actual al avansului tehnologic din domeniul *IT* și al locului important pe care acesta îl ocupă în viețile oamenilor, s-au dezvoltat numeroase aplicații, iar numărul acestora continuă să crească. Acestea au evoluat de la programe instalate local la cele stocate în cloud și livrate prin browsere, mai ales în cadrul conceptului de *Software as a Service*. Programatorii aleg adesea să își creeze singuri programe care să le rezolve anumite cerințe, atunci când nu găsesc o soluție care să le satisfacă necesitățile. Există o multitudine de platforme online unde se pot publica creațiile oamenilor din domeniul *IT*, dar marea majoritate nu oferă și suport pentru executarea directă a soluțiilor publicate.

Soluția proiectată își propune să ofere un sprijin în această sferă, prin crearea unei platforme care să ofere atât suportul pentru publicarea propriilor creații, cât și mediul în care acestea să poată fi executate. Dezvoltarea unei astfel de aplicații este o sarcină destul de dificilă și trebuie să ofere suport cel puțin pentru următoarele concepte: mecanisme de scalabilitate, echilibrare a încărcării, toleranță la erori, jurnalizare, descoperire a nodurilor și, cel mai important, securitate mai ales în cadrul execuției remote.

În cadrul platformei implementate, pentru a integra cât mai facil conceptele amintite, se propune o arhitectură orientată pe *microservicii* cu accent pe *scalabilitate*. S-au creat două metode de executare remote a codului: prima este executarea prin încărcarea codului în *JVM* (Java Virtual Machine), iar a doua este executarea în containere *Docker* cu suport pentru limbajele Java și Python. Alegerile de proiectare și de implementare sunt analizate și comparate pe baza unor rezultate experimentale.

Tema prezentată analizează domeniul aplicațiilor *distribuite*, beneficiile utilizării unui *cluster* și a infrastructurii *cloud* pentru a sprijini publicarea și execuția facilă a serviciilor definite de utilizator. Dezvoltarea suportului cloud a revoluționat modul în care companiile gestionează și accesează datele și aplicațiile lor, oferind flexibilitate și scalabilitate fără precedent. Însă aceste caracteristici nu ar trebui să fie limitate la aplicații complexe, ci trebuie să fie accesibile oricui, permițându-le oamenilor să își publice și să ruleze propriile creații care rezolvă probleme specifice.

Introducere

Digitalizarea a câștigat o importanță deosebită în ultimii ani, nu doar pentru specialiști, ci și pentru persoane din alte domenii. Multe dintre activitățile industriale sau cotidiene au devenit mai ușoare datorită tehnologiei, ceea ce impune nevoia de a avea la dispoziție programe care să automatizeze anumite sarcini. De la unirea mai multor fișiere, modificarea documentelor pdf, compararea anumitor texte, generarea automată de grafice până la site-uri ce oferă suport asistat de inteligență artificială, toate acestea i-au luat forma unor *servicii* și ne fac viața mai ușoară.

Numărul de aplicații disponibile crește din ce în ce mai mult, *Windows report* anunță în octombrie 2023 existența a aproximativ 800.000 de aplicații ¹ în magazinul Microsoft store. Chiar dacă acest număr pare impresionant cifrele pentru magazinele de aplicații destinate telefoanelor mobile sunt și mai amețitoare, astfel utilizatorii Android pot alege din peste 3.55 milioane de aplicații, iar cei iOS din peste 1.6 milioane menționează una dintre cele mai importante platforme globale de date și business intelligence ². Totuși în timp lucrurile au început să evolueze în altă direcție o dată cu dezvoltarea tehnologiilor *cloud*. Dacă în mod clasic furnizorii își vindeau software-ul ca produs, începând cu anul 1999 când Salesforce și-a lansat un sistem de gestiune a relațiilor cu clienții, găzduit în cloud și livrat publicului prin browsere web conceptul de software as a service (SaaS) a început să prindă contur și să acapareze treptat o mare parte din piață [1], mai există încă două concepte similare PaaS (platform as a service) și IaaS (infrastructura as a service). Cele trei niveluri ale serviciilor cloud au diferențe fundamentale între ele, dar au un punct comun: faptul că în fond sunt servicii. Importanța acestora este vizibilă și din cifrele financiare publicate de Gartner pentru 2022 platformele de tip SaaS au avut venituri de 167 miliarde de dolari, IaaS de 90 miliarde, iar PaaS de 89 de miliarde ³. De asemenea previziunea pentru 2023 era o creștere cu 20.7% ceea ce arată modul rapid de creștere a utilizării lor și implicit a importanței.

Fie că vorbim de aplicații clasice sau de aplicații găzduite în cloud și accesate prin browsere web, ambele au avantaje și dezavantaje. De asemenea, se înregistrează creșteri ale numărului de utilizatori și ale veniturilor realizate din acestea. Elementul cert este că oamenii utilizează din ce în ce mai mult aplicațiile, fie pentru relaxare, fie pentru a-și ușura anumite sarcini. Există o cerere crescută și este necesar să se creeze mijloace adecvate pentru a pune la dispoziție serviciile necesare.

De-a lungul anilor, oamenii ce posedă cunoștințe de programare și-au scris aplicații care au rezolvat anumite probleme punctuale, dar chiar și atunci când aceste programe sunt publicate, ele nu ajung ușor la publicul larg, mai ales dacă utilizatorii nu au cunoștințe cel puțin medii în domeniu și nici nu sunt dispuși să investească timp pentru a executa local programul (a face configurări sau a instala dependențele necesare). Beneficiile publicării rezultatelor propriei munci sunt diverse și se întind de la satisfacție personală până la beneficii financiare, totodată faptul că altcineva își dă cu părerea despre munca sa îl ajută pe dezvoltator să evolueze.

Tema pe care am abordat-o a plecat de la ideea de serviciu, și mai exact am dezvoltat o platformă care să le permită utilizatorilor ca direct din browsere web să își publice și execute programele. Ideea vine de la o problemă de care m-am lovit și eu. Adesea avem de rezolvat o problemă minoră, iar soluțiile deja disponibile sunt contra cost (eventual și cu abonamente de minim o lună) sau au foarte multe reclame și atunci cei din domeniu aleg să scrie ei însuși un program

¹How Many Apps Does Microsoft Store Have: Year-By-Year Guide, 2023. Disponibil online: <https://windowsreport.com/state-windows-8-apps-windows-store/>

²Number of apps available in leading app stores as of 3rd quarter 2022, 2022. Disponibil online: <https://www.statista.com/statistics/276623/number-of-apps-available-in-leading-app-stores/>

³Gartner Forecasts Worldwide Public Cloud End-User Spending to Reach Nearly \$600 Billion in 2023, 2022. Disponibil online: <https://www.gartner.com/en/newsroom/press-releases/2022-10-31-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-reach-nearly-600-billion-in-2023>

sau caută pe internet o soluție deja existentă. Dacă o construim noi există posibilitatea de a publica soluția noastră pe diferite platforme, dar adesea doar la nivel de cod (dacă există o posibilitate de colaborare și execuție ele sunt sub adiacente și destul de slab dezvoltate), dar adesea presupun pentru execuție copierea codului și executarea lui locală, element care introduce pierderi de timp.

Dezvoltarea pentru programatori a unui spațiu în care să-și publice *micile creații* care rezolvă anumite probleme este o necesitate. Aceste soluții trebuie să fie ușor accesibile oamenilor din domenii variate, care posedă cunoștințe minime de interacțiune pe internet. Cu un cont creat pe această platformă utilizatorii vor putea publica fișiere ce conțin cod, executa/vizualiza cod sursă al altor utilizatori dacă acesta este disponibil.

În mod evident dezvoltarea unei astfel de platforme este o sarcină complexă și are o multitudine de probleme sub adiacente printre care: cerințele de securitate, cerințe de echilibrare a încărcării, gestiune de resurse, toleranță la erori, jurnalizare. Ea devine astfel nu doar dezvoltarea unei simple aplicații ce execută cod, ci crearea unei adevărate platforme distribuite ce oferă servicii publicului.

Tema a fost gândită încă de la început cu ideea de executare în *cluster* a serviciilor platformei, abordarea este justificată prin următoarele necesități intrinseci: pierderea la nivelul unui cluster local ce se execută în același LAN (Local Area Network) a conexiuni la internet să nu ducă la indisponibilizarea totală a platformei, reducerea încărcării și a vulnerabilității pe nodurile din același LAN (se face o separare între nodurile ce execută cod și cele ce se ocupă efectiv de clienți). Astfel chiar dacă un nod sau o grupare devin indisponibile vor fi afectați un număr mic de clienți, atâta timp cât sunt pornite mai multe noduri în rețele sau pe stații diferite. Toată platforma este accesată printr-o interfață web, ideea centrală în cadrul metodologiei software as a service (SaaS). Platforma are o arhitectură bazată pe microservicii în partea de back-end care sunt legate prin următoarele protocoale: HTTP, TCP, UDP și WebSockets.

Lucrarea de față prezintă în primul capitol intitulat „*Fundamentarea teoretică și documentarea bibliografică*” o serie de realizări similare pe această temă prezente deja pe piață, precum și modul în care au fost dezvoltate anumite caracteristici de alte companii. Se insistă de asemenea pe motivele utilizării cluster-elor în cadrul executării serviciilor. O parte semnificativă a acestui capitol este și partea de analiză a metodelor de execuție remote.

Capitolul al doilea numit „*Proiectarea aplicației*” cuprinde descrierea modului în care a fost gândită arhitectura aplicației pornind de la necesitățile documentate în capitolul 1. Inițial se prezintă serviciile dezvoltate, apoi se face o trecere în revistă a dezvoltării lor pornind de la aplicațiile client, nodurile agregare care sunt în fond aplicații Spring Boot și care se ocupă de concepte separate ale aplicației. Urmează apoi o analiză mai detaliată a nodului master și a celui de execuție, deoarece ele sunt cele mai importante componente ale aplicației. La final se prezintă partea de jurnalizare și bazele de date folosite.

„*Implementarea aplicației*” este cel de al treilea capitol al aplicației și prezintă modul concret în care au fost implementate modulele, a căror arhitectură a fost descrisă în capitolul precedent. De remarcat este faptul că în cadrul lui se insistă în mod deosebit pe problemele întâmpinate și pe modul în care au fost soluționate.

Ultimul capitol cu numele „*Testarea aplicației și rezultate experimentale*” prezintă în prima parte modul de pornire a platformei și configurațiile necesare apoi diferite aspecte legate de modul în care a fost testată aplicația cu unele capturi de ecran privind execuția diferitelor coduri sau înregistrarea unor excepții (așteptate din cauza atacurilor de securitate prezente în acele fișiere). O altă parte importantă a acestui capitol sunt analizele cu privire la utilizarea memoriei și procesorului de către microserviciile aplicației, acestea sunt făcute în scopul de a demonstra utilitatea alegerilor de proiectare făcute în capitolul 2. De asemenea se prezintă și rezultate ale performanței platformei precum timpi de răspuns, consumul de memorie și CPU folosite la apariția cererilor. În final se prezintă un rezumat al diferitelor aspecte legate de măsurile de securitate implementate și fiabilitate care au fost detaliate pe parcursul întregii lucrări.

Capitolul 1. Fundamentarea teoretică și documentarea bibliografică

1.1. Realizări actuale privind execuția distribuită a serviciilor

Există multe platforme care oferă suport pentru executarea de cod, unele mai simple (ca de exemplu compilatoarele/interpretoarele online) sau mai complexe. A doua categorie oferă suport pentru executarea programelor complexe sau chiar configurarea unor mașini virtuale pe care este executat codul, acestea nu sunt dedicate publicării micilor creații. Datorită complexității ele sunt descurajante pentru cei mai mulți dintre dezvoltatori care nu își creează aplicații neapărat în scop comercial. Primele variante nu integrează și posibilitatea de partajare nativă a rezultatului munci în timp ce a doua este descurajantă din pricina complexității. Există foarte multe platforme ce execută cod la distanță (en. remote), de exemplu: Programiz, JDoodle, Repl.it, Ideone, OnlineGDB, CompileJava, Paiza.IO, Rextester, W3schools, Tutorialspoint etc. O selecție de platforme ce oferă funcționalități similare celei pe care doresc să o dezvolt sunt comentate în cele ce urmează, iar tabele 1.1 și 1.2 reprezintă o centralizare a principalelor funcționalități.

- *Programiz*⁴ - platformă centrată pe învățare care include posibilitatea de executare de cod al clientului în limbajele: Python, JavaScript, R, C, C++, Java și Kotlin. Suportul oferit este doar pentru cod sursă într-un singur fișier și care nu poate fi publicat pe platformă. De asemenea nu are suport pentru manipularea fișierelor.
- *Jdoodle*⁵ - platformă dezvoltată în sensul de IDE online centrat pe mai multe limbaje (C, C++, Java, Python, JavaScript etc.). Oferă suport pentru fișiere, adăugarea de librării externe ca fișiere jar. De asemenea oferă suport pentru partajarea cu drept de editare a codului. De asemenea oferă o multitudine de tutoriale și extensii.
- *Repl.it*⁶ - site-ul oferă suport pentru mai multe limbaje (Python, Java, JS, PHP, Ruby etc.), un mediu colaborativ în care se pot atât publica programe cât și scrie articole. Oferă suport pentru lucrul cu fișiere, deploy în cloud, scalare automată a programelor și configurarea de mașini virtuale rezervate. De asemenea oferă suport pentru a vizualiza în timp real costurile rezultate din utilizarea ei.
- *Ideone*⁷ - platforma oferă suport pentru execuția de cod în foarte multe limbaje printre care cele mai importante sunt : Bash, Pascal, C, Perl, C#, PHP, C++, Python, C++, Haskell, Ruby, Java, Prolog, Assembler 32bit. Oferă suport pentru preluarea input-ului de la consolă, dar nu oferă suport pentru lucrul cu fișiere. Codurile pot fi publice sau private și pot avea asignate etichete cu privire la statusul lor (sunt funcționale cu succes, au erori, avertismente, apeluri ilegale de sistem, limite de timp depășite). Se pot vedea codurile altor persoane, dar nu sunt oferite metode de căutare.
- *OnlineGDB*⁸ - această platformă oferă suport pentru execuția de cod în mai multe limbaje: C#, C++, Java, Kotlin, Python, JS etc. Oferă posibilitatea de a folosi fișiere, input de la consolă, de a partaja codul cu ajutorul unui link. Platforma mai oferă suport pentru întrebări între programatori și o secțiune de învățare. Se pune de asemenea la dispoziție un workspace în care pot fi salvate proiectele.

⁴<https://www.programiz.com/>

⁵<https://www.jdoodle.com/>

⁶<https://replit.com/>

⁷<https://ideone.com/>

⁸<https://www.onlinegdb.com/>

- *CompileJava*⁹ - spre deosebire de cele anterioare aceasta oferă suport doar pentru limbajul Java. Codul poate fi executat, se pot primi argumente în linie de comandă și descărca sub formă de arhivă jar. Nu se oferă suport pentru lucrul cu fișiere sau colaborare.
- *Paiza.IO*¹⁰ - platforma oferă suport pentru peste 20 de limbaje, parametrii în linie de comandă, colaborare, integrare cu Github, nu oferă suport pentru fișiere. Aceasta se mișcă destul de greu și are foarte multe erori.
- *Rextester*¹¹ - suportul oferit este pentru mai multe limbaje, nu permite lucru cu fișiere sau preluarea argumentelor din linia de comandă. Expune un API pentru execuție la distanță, un top al celor mai votate coduri, un mecanism de validare de text cu regex-uri, colaborare live pe cod. De asemenea oferă acces la un grup Google unde se pot purta discuții între utilizatori.
- *W3schools*¹² - aceasta integrează (pe lângă multitudinea de tutoriale) și posibilitatea de executare de cod în mai multe limbaje, se poate prelua input-ul utilizatorului prin prescrierea variabilelor așteptate într-un text box (ceea ce aduce probleme când apare chiar un simplu if), nu se oferă suport pentru citirea din fișier sau mediu colaborativ. Centrul este mai mult pe testarea micilor bucăți de cod implicate de procesul de învățare.
- *Tutorialspoint*¹³ - în platformă este prezentă execuția în mai multe limbaje, preluarea input-ului de la tastatură, nu oferă posibilitatea de lucru cu fișiere, simulează o funcționalitate colaborativă prin trimiterea unui link (nu se oferă niciun fel de verificare a identității celui ce editează codul altcuiva).
- *Amazon Web Services, Microsoft Azure, Google Cloud Platform, Heroku* sunt platforme foarte complexe, dedicate în special publicării programelor mari, a căror complexitate depășește ideea mea de mediu colaborativ de dezvoltare de programe care rezolvă probleme punctuale.

Din recenziile prezentate mai sus se deduce faptul că majoritatea oferă funcționalitatea de a executa cod în mai multe limbaje cu preluarea inputului utilizatorului prin intermediul unei linii de comandă. Centrul în mare parte este pe execuția de cod nu pe colaborare, cele care expun astfel de funcționalități o fac minimal și deseori nesigur sau necentrat pe ideea de ai ajuta și pe alții. Celelalte au mai mult o funcționalitate pur comercială și sunt dedicate publicării soluțiilor complexe, deci descurajante pentru publicul cărui mă adresez.

1.2. Referințe la teme/subiecte similare

Platformele enumerate mai sus nu precizează aproape nimic legat de implementarea lor. Astfel încât pentru părțile cruciale ale lucrării mele am preluat informații despre modul de implementare a funcționalităților din alte surse pentru a face o analiză (aplicații a căror domenii nu se intersectează ca scop, dar se intersectează la nivelul unor concepte implementate).

1.2.1. Mecanisme de descoperire a nodurilor

Una dintre cele mai importante caracteristici ale cluster-elor de calculatoare în general sau a mai multor programe ce se execută dependent pe o platformă este aceea de cunoaștere a nodurilor pe care le pot folosi în diverse scopuri. Problema este abordată diferit în funcție de scopuri,

⁹<https://www.compilejava.net/>

¹⁰<https://paiza.io/en>

¹¹<https://rextester.com/>

¹²<https://www.w3schools.com/>

¹³<https://www.tutorialspoint.com/index.htm>

Nume platformă/ Funcționalitate	Cluster ServExe	Programiz	Jdoodle	Repl.it	Ideone	OnlineGDB
Suport multilimbaj	✓	✓	✓	✓	✓	✓
Preluare input de la tastatură	✓	✓	✓	✓	✓	✓
Suport pentru fișiere	✓		✓	✓		✓
Dependențe Maven			✓	✓		
Mediu colaborativ de editare			✓	✓		✓
Publicare programe	✓			✓		
Căutare facilă a programelor	✓			✓		

Tabelul 1.1. Comparăție cu platformele analizate (1)

Nume platformă/ Funcționalitate	Cluster ServExe	Compile Java	Paiza.IO	Rextester	Tutorials point
Suport multilimbaj	✓		✓	✓	✓
Preluare input de la tastatură	✓	✓	✓		✓
Suport pentru fișiere	✓				
Dependențe Maven					
Mediu colaborativ de editare			✓	✓	✓
Publicare programe	✓			✓	
Căutare facilă a programelor	✓				

Tabelul 1.2. Comparăție cu platformele analizate (2)

în marea majoritatea a cazurilor nu se recurge la o descoperire "automată", ci se merge pe metoda unor *fișiere de configurare preexistente* (metodă deseori ineficientă și care aduce probleme majore din cauza imposibilității prezenței unei scalări în timp real, de exemplu). Metoda este folosită, în general, la aplicațiile simple orientate pe servicii în care nodurile nu sunt scalate decât prin intermediul unor mecanisme adiacente (de exemplu, cel pus la dispoziție de Docker).

O altă soluție de descoperire a nodurilor (comercială) este *Amazon Elastic Compute Cloud* [2], metoda se bazează pe adrese IP publice (folosește ocazional și asocierile din DNS), în unele cazuri și private. Din păcate nu sunt disponibile prea multe date legate de modul de funcționare.

Soluția celor de la *Kubernetes* se bazează pe un API, astfel fiecare nod efectuează un apel HTTP spre nodul master pentru a se înregistra la el [3].

Descoperirea automată a nodurilor nu este folosită doar în cadrul unui cluster de calculatoare, despre soluții ce privesc această temă scriu și cei de la *Biamp Systems*¹⁴ care prezintă cu argumente pro și contra descoperirea nodurilor cu ajutorul *TCP și UDP* [4]. Astfel și aceștia insistă pe folosirea configurării directe în situația în care nu se poate folosi o descoperire dinamică folosind UDP-uri (din varii motive printre care: folosirea în mai multe subrețele, timpul mai îndelungat de descoperire, protecția firewall și riscurile pierderi pachetelor, întrucât protocolul UDP nu garantează ajungerea la destinație sau un feedback al operației de trimitere).

Din cele prezentate anterior se observă că mecanismele de descoperire folosite variază în funcție de context, iar cele mai multe au la bază configurări manuale. Folosirea TCP/HTTP necesită configurații suplimentare, iar folosirea UDP-ului aduce întârzieri și lipsa unei siguranțe în descoperire (totuși aceasta este făcută pur automat).

1.2.2. *Mecanisme de echilibrare a încărcării*

Prezența mai multor entități de același tip fie că este vorba despre mașinării fizice, persoane, echipamente hardware sau software introduce în acel context o problemă nouă și anume necesitatea de împărțire a sarcinilor pe respectivele entități și este de la sine înțeles că acest fapt nu se poate face aleatoriu ci este necesară o echitabilitate în distribuirea lor. Model în care se poate realiza această acțiune se împarte în două clase mari: *statică* și *dinamică*, așa cum menționează și sursa [5].

Echilibrarea statică nu ține cont de statusul actual al nodurilor ci consideră că toate trebuie să primească aceeași cantitate de cereri. Cel mai folosit algoritm în acest caz este *Round robin* (care are și o *variantă ponderată*) în care se distribuie cererile pe baza unei liste circulare. În varianta ponderată nodurilor li se menționează un grad după care se face distribuția (un nod cu un grad mai mare va primi mai multe cereri). Metoda este ineficientă chiar și în varianta ponderată, deoarece este posibil ca mai ales atunci când cererile sunt diferite unele noduri să aibă de fapt o încărcare a resurselor mai mare, chiar dacă au același număr de cereri.

Echilibrarea dinamică are la bază alegerea nodului spre care este distribuită cererea pe baza statusului fiecărui nod, acest fapt implică necesitatea de a ști în timp real *statusul celorlalte noduri*. Cea mai folosită metodă de clasificare este legată de resursele disponibile (memorie, procesor, număr de conexiuni disponibile), problema care apare este determinarea celui mai puțin încărcat nod dacă se dorește luarea în considerare a mai multor parametri.

Cele două metode prezentate au avantaje și dezavantaje, folosirea unui algoritm Round robin aduce o simplitate în implementare, dar și posibilitatea ca un nod să primească sarcini de o complexitate mai mare care să îl supraîncarce (dacă sarcinile nu au toate aceeași complexitate). Metoda dinamică aduce problema necesității de a cunoaște în timp real starea nodurilor (deci o creștere de trafic TCP/HTTP/UDP sau foarte multe mesaje în cozi, dacă se folosește această metodă de comunicare) dar și pași suplimentari pentru determinarea nodului spre care se trimite cererea (complexitatea poate fi mare atunci când se caută în multe noduri și mai ales dacă se

¹⁴Companie specializată în soluții de sisteme audio pentru medii comerciale, instituționale și industriale.

folosesc mai mulți parametrii).

1.2.3. Mecanisme de toleranță la erori

În mod general toleranța la erori se referă la *capacitatea unui sistem de a continua să funcționeze fără întrerupere* chiar și atunci când una sau mai multe dintre componentele sale se defectează [6]. Aceeași sursă precizează că una dintre principalele caracteristici ale sistemelor tolerante la erori este faptul că folosesc *componente de rezervă* care înlocuiesc componentele defecte, acest mecanism se numește *redundanță*.

Unul dintre cele mai folosite mecanisme de toleranță la erori este executarea mai *multor instanțe ale aplicației* sub un nod de echilibrare a încărcării (en. load balancer), astfel se elimină existența unui singur punct de defecțiune care ar putea face nefuncțională aplicația. În acest mod ele rezistă și mai bine la încărcare mare scăzând defectarea din cauza suprasolicitării. Load balancer-ul asigură și faptul că cererile nu vor mai fi trimise spre un nod care a devenit nefuncțional. O altă soluție, cea de *failover*, este folosită atunci când rețeaua principală devine total nefuncțională, în acest caz un sistem de failover primește automat toată activitatea până la repararea rețelei principale, pentru un timp null de nefuncționare se trimit cereri duplicitate spre ambele sisteme.

Tot din sfera toleranței la erori face parte și *capacitatea de recuperare a datelor*, de exemplu Kubernetes repornește nodurile în caz de eșec [7], dar acest lucru nu este suficient atunci când apare o eroare la sistemul de stocare caz în care trebuie făcută recuperarea datelor, eventual dintr-un spațiu de backup precizează aceeași sursă.

Din cele prezentate anterior se observă că atenția deosebită este în asigurarea funcționării platformei în cazul căderii anumitor noduri, dar și a recuperării datelor. Continuarea funcționării în cazul căderii unui nod este asociată adesea cu disponibilitatea ridicată chiar dacă ele reprezintă concepte diferite [8]. În cazul erorilor de stocare se încearcă recuperarea de pe partiția defectată sau se folosește un spațiu de backup (cu dezavantajul duplicării datelor).

1.2.4. Mecanisme de securitate

Securitatea digitală *nu este doar o opțiune*, ci o necesitate fundamentală, în general, în cadrul atacurilor de securitate se urmărește obținerea de acces neautorizat la date sensibile, compromiterea integrității datelor sau sistemelor, perturbarea serviciilor, cauzarea de pagube financiare sau de reputație. Există diverse atacuri care ținesc platformele web, în figura 1.1 sunt prezente nouă cele mai comune atacuri și procentul de platforme ce au fost atacate din cele analizate.

Dintre acestea anticipez faptul că aplicația mea se expune la: *SQL Injection*, *OS Command Execution*, *Path traversal*, *Denial of Service*, *Arbitrary File Upload* dar și *Cross-Site Scripting (XSS)* în cadrul interfeței client.

Cel mai des întâlnit atac este *SQL Injection* căruia prin folosirea bazelor de date relaționale mă expun și eu. De cele mai multe ori atacurile sunt efectuate folosind mecanisme automate, iar metoda simplei filtrări a parametrilor (respingerea cererilor ce prezintă caractere speciale) nu este suficientă [9]. Există cinci metode fundamentale de protecție împotriva acestor atacuri, menționează sursa [10]. Prima metodă este cea de filtrare a parametrilor de input și respingerea cererilor care conțin caractere speciale, metoda nu poate opri singură atacurile de acest tip, dar poate elimina cele mai multe probleme. O altă metodă este dezactivarea tuturor funcționalităților necesare ale unui utilizator pe baza de date (un exemplu este restricționarea accesului la *procedura xp_cmdshell din Microsoft SQL Server* care permite utilizatorilor să execute comenzi de sistem pe serverul de bază de date [11]), astfel *trebuie să fie creați utilizatori separați* pentru anumite părți ale bazei de date. O altă vulnerabilitate care poate fi exploatată de atacatori este expunerea prea detaliată a erorilor legate de baza de date apărute. De asemenea, în timp sunt descoperite noi vulnerabilități ale bazelor de date din acest motiv ar trebui actualizate de către organizație la cea mai nouă versiune. Jurnalizarea poate juca și ea un rol decisiv în descoperirea vulnerabilităților

prin analiza evenimentelor care au apărut în sistem.

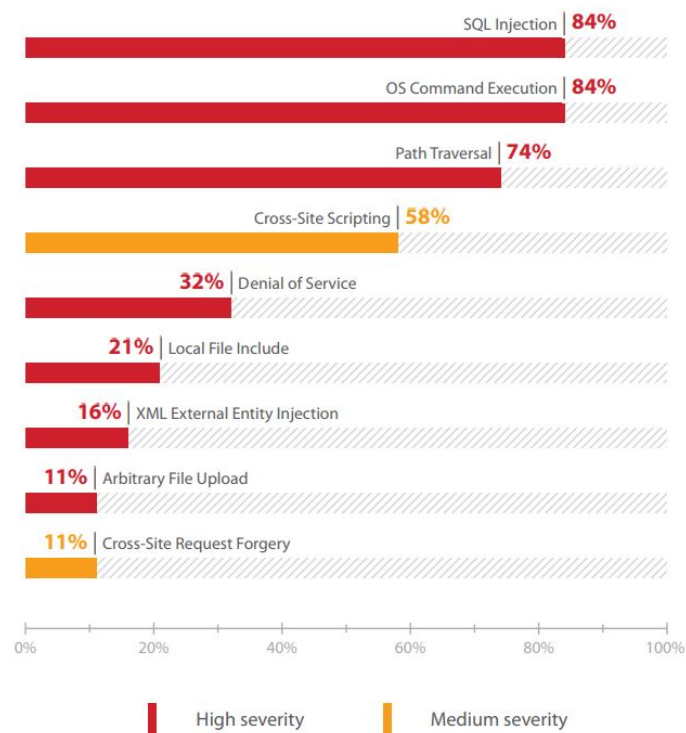


Figura 1.1. Tendințele atacurilor în aplicațiile web (preluare din [12])

Platforma pe care doresc să o dezvolt se expune din cauza execuției de cod remote în mod direct la atacul de tip *OS Command Execution*. Injecția de comenzi este un atac în care scopul este executarea de comenzi arbitrare pe *sistemul de operare gazdă* printr-o aplicație vulnerabilă, privilegiile sunt de obicei aceleași cu cele ale aplicației care expune vulnerabilitatea [13]. Există numeroase metode de apărare, comunitatea *The Open Worldwide Application Security Project* (OWASP) prezintă în articolul disponibil la [14] mai multe metode printre care cele mai relevante sunt evitarea comenzilor directe către sistemul de operare și înlocuirea lor cu funcții de bibliotecă, tratarea șirurilor de caractere ca șiruri efective prin încadrarea între ghilimele (vulnerabilitate PHP), validarea intrărilor, reducerea privilegiilor aplicațiilor. Majoritatea limbajelor expun biblioteci sau metode de apeluri către sistemul gazdă, iar aceste funcționalități trebuie folosite cu o maximă responsabilitate.

Atacul de tip *Path traversal* are ca scop accesarea fișierelor și directoarelor care sunt stocate în afara folderului rădăcină web, metoda folosită este în general accesarea de fișiere cu ajutorul căilor *“dot-dot-slash (../)”* sau varietăți ale acestora după cum menționează articolul din sursa [15]. Scopul este accesarea fișierelor și directoarelor care conțin date care ar putea fi folosite pentru a aduce câștiguri (fișiere de configurare, documente, cod sursă al unor aplicații etc.). Acesta se pretează atât pentru servere web, dar și în cazul aplicațiilor ce efectuează execuției de cod, ca în cazul meu, unde utilizatorul ar trebui să își poată accesa doar propriile fișiere. Metodele de protecție sunt validarea căii utilizatorului înainte de a satisface cererea și folosirea căii ca subordonat al directorului de bază al aplicației [16].

Denial of Service (DoS) este un atac destul de întâlnit și se axează pe afectarea disponibilității unui serviciu (site, aplicație, server). Metodele sunt diverse și se axează în mare pe trimiterea unui număr mare de cereri, manipularea pachetelor de rețea, vulnerabilități logice sau de gestionare a resurselor, *adesea un atac DoS este însoțit de injectarea de cod*, deoarece ținta poate deveni mai vulnerabilă în acest context [17]. Metodele de protecție depind de modul în care este atacată ținta, și trebuie plecat în primul rând de la aflarea punctelor slabe. *Există atacuri la nivelul aplicației*

sau la nivelul *protocoalelor de comunicație*. La nivel de rețea cererea HTTP este livrată lent, iar până la livrarea completă resursele sunt blocate, la un moment dat se va atinge capacitatea maximă a pool-ului de resurse ceea ce va duce la indisponibilizarea aplicației conform [18]. Metodele de prevenție la nivelul aplicației se rezumă la validarea parametrilor (de exemplu: limitarea numărului de obiecte ce se vor instanția, a numărului de elemente cerute în cadrul unei agregări de resurse, limitarea dimensiunii parolelor, dacă acestea sunt prea lungi unele metode de criptare pot dura foarte mult). Importantă este, de asemenea, prevenirea unui *single point of failure* - SPOF. O altă metodă eficientă este limitarea timpului de sesiune pe baza inactivității sau a unui time-out, pentru a reduce numărul de conexiuni simultane (mai ales în cadrul protocoalelor ce mențin conexiuni active) sau a dimensiunii fișierelor încărcate (mai ales dacă se fac verificări ale conținutului). În general se implementează și trebuie realizate metode de blocare a tuturor cererilor care vin de la o adresă IP sau un utilizator dacă acestea sunt repetate sau au conținut suspect, de asemenea se pot folosi captcha-uri pentru limitarea atacurilor făcute cu tool-uri automate. Metodele de protecție sunt similare și în cazul *Distributed denial of service (DDoS)*, dar trebuie avut în vedere faptul că cererile multiple vin de la sisteme diferite (organizate ca rețele botnet), deci este mai greu de identificat și blocat cererile pe baza identificatorilor atâta timp cât ele vin de la entități diferite în număr mai mic pe fiecare, conform [19]. Mai multe detalii sau metode pot fi consultate în sursa [18].

Arbitrary File Upload este un tip de atac centrat pe încărcarea de fișiere cu potențial periculos pe un server care apoi să fie folosite pentru a efectua operații pe server cu scopul furtului de date, a indisponibilizării țintei, a creării de atacuri cross-site scripting (XSS) sau a efectuării de pe țintă a atacurilor spre alte sisteme conectate la server. Se folosesc diferite tipuri de fișiere dar mai ales *PHP* (o vulnerabilitate în această direcție a avut și *WordPress* când s-au încărcat astfel de fișiere în locul imaginilor conform [20]). Metodele de protecție strict pentru acest atac sunt: verificarea extensiilor și a fișierelor pe baza primilor octeți care au ca informații tipul, pentru că unele sisteme de operare (de exemplu, Linux) nu determină tipul pe baza extensiei. O prezentare a modului de determinare a tipului după conținut este disponibilă în [21].

Atacul *Cross-Site Scripting (XSS)* țintește interfața web a aplicației cu scopul de a injecta cod JavaScript în paginile web, acest cod poate fi folosit pentru a fura datele utilizatorilor. În mod implicit framework-ul *Angular* (pe care îl voi folosi și eu) se ocupă de protecție împotriva acestui atac prin mecanisme de sanitizare atunci când se folosesc metode ale framework-ului (ex. interpolare), dar dacă se folosesc proprietăți HTML clasice (ex. *innerHTML*) vulnerabilitatea este prezentă. Totodată există și API-uri externe care pot fi folosite în prevenție, mai multe detalii pot fi consultate la referința [22]. Metoda folosită, în general, este cea de sanitizare, iar framework-urile dezvoltate oferă, în general, suport de securitate implicit în această direcție.

1.2.5. Mecanisme de jurnalizare în sisteme distribuite

Trecerea de la aplicații monolitice la servicii distribuite, care a apărut și s-a dezvoltat în ultimul timp a adus în discuție noi probleme ce trebuie depășite, iar una dintre acestea este jurnalizarea în sisteme distribuite. Datorită importanței ei, jurnalizarea nu poate fi trecută cu vederea. Rolul vital în ciclul de viață al unei aplicații include printre altele: capacitatea de a detecta erori, evaluarea performanțelor și detectare eventualelor atacuri de securitate. Mecanismul de jurnalizare într-un mediu distribuit constă din două faze: *transmisie și stocare* menționează sursa [23]. Fiecare dintre cele două etape are propriile sale probleme, în faza de transmisie câteva probleme potențiale sunt confidențialitatea, integritatea, autenticitatea și livrarea fiabilă, iar în cea de stocare integritatea.

Una dintre metodele de jurnalizare este transmiterea fișierelor de jurnal locale spre un *nod central* ce se ocupă de acest lucru și apoi replicarea pe alte servere de *backup*, menționează [24]. O îmbunătățire care poate fi adusă acestei metode este transmiterea spre *noduri de jurnalizare locale* și apoi acestea să transmită spre un *manager centralizat pentru agregare* [25].

Din cele menționate se observă faptul că se folosesc jurnale locale care la anumite intervale de timp sunt trimise spre un nod de jurnalizare central. Metoda este eficientă și recomandată pentru a reduce numărul de mesaje trimise. Folosirea unor noduri de management local și a unui sistem de backup sunt de asemenea elemente de dorit, totuși, în timp, dimensiunea jurnalelor poate crește foarte mult, fapt care aduce o îngreunare în analiza log-urilor. Mecanismele de jurnalizare ar trebui să prezinte și metode facile de clasificare/transformare/filtrare a informațiilor pentru a fi mai ușor utilizate atunci când este necesar.

1.3. Utilizarea clusterelor de calculatoare pentru executarea serviciilor

Din punct de vedere generic un cluster reprezintă: „Un grup de lucruri similare care sunt apropiate unele de altele”¹⁵. În domeniul industriei IT un cluster reprezintă o colecție de computere interconectate care funcționează împreună ca o singură entitate. Acestea comunică, în general, în rețea și au scopul de a executa diferite servicii, există multe avantaje ale folosirii lor. Din [26], [27], [28] am dedus următoarele beneficii semnificative:

- *Performanță* - Necesitățile de resurse ale programelor au crescut o dată cu dezvoltarea tehnologiilor, fapt care a dus la incapacitatea unui singur calculator de a executa multe dintre programele actuale care necesită multe resurse, acest lucru a dus la dezvoltarea aplicațiilor distribuite. În contextul acestui tip de aplicații cluster-ele sunt cel mai adesea folosite și oferă o performanță ridicată deoarece sarcinile se distribuie pe mai multe unități și implicit scade gradul de încărcare la care ar fi supusă o singură entitate. Practic, chiar dacă o singură unitate de calcul nu are capacitatea de a executa o anumită aplicație aceasta poate fi împărțită în entități mai mici și executate pe noduri diferite. Performanța unui cluster vine tocmai din faptul că se pot executa sarcini mult mai complexe și care necesită mult mai multe resurse, adică scade gradul de suprasolicitare a unui nod. Pe lângă aceste beneficii de performanță există și beneficii financiare, întrucât este mai eficientă economic cumpărarea mai multor calculatoare/serveere cu resurse moderate decât procurarea unui care să prezinte un număr semnificativ de resurse.
- *Scalabilitate* - Executarea serviciilor în cluster aduce posibilitatea de scalare ușoară pe orizontală, prin adăugarea sau eliminarea unor noduri în funcție de necesitățile platformei găzduite în el, fără a fi necesară înlocuirea întregului sistem.
- *Redundanță și rezistență la erori* - Cluster-ele sunt concepute pentru a fi rezistente la eșecuri, eșuarea unui nod nu duce la eșuarea platformei. Alte noduri pot prelua automat sarcinile celor eșuate mecanism implementat adesea prin redundanță. În acest context se minimizează timpul de inactivitate.
- *Distribuție eficientă a sarcinilor* - Cluster-ele permit prin mecanisme de echilibrare a încărcării posibilitatea de a distribui în mod echitabil cereri spre noduri. Acest lucru duce la o utilizare mai eficientă a resurselor hardware și la evitarea suprasolicitării unui nod.
- *Flexibilitate și adaptabilitate* - Cluster-ele pot fi proiectate pentru a satisface specific cerințele anumitor aplicații, fiecare nod al cluster-ului poate fi optimizat/ales pentru un anumit tip de sarcini, ceea ce permite o adaptabilitate mai mare în funcție de nevoile platformei.

Din cele prezentate anterior se observă că atunci când se vorbește despre platforme mari care execută sarcini complexe este mult mai ușor ca ele să fie alcătuite din aplicații mai mici distribuite. În acest context utilizarea unui cluster pentru a executa platforma este o metodă optimă, care aduce multe beneficii și soluționează foarte multe probleme.

¹⁵Cambridge Dictionary Cluster. Disponibil online: <https://dictionary.cambridge.org/dictionary/english/cluster>

1.4. Metode de execuție remote a codului

Odată cu dezvoltarea domeniului aplicațiilor distribuite a apărut necesitatea de a muta codul din mediul local într-un mediu de producție sau izolat pentru a restrânge dependența de mașină. Atunci când programul este executat pe o altă mașină procesul poartă denumirea de "*execuție remote*". Metodele sunt diverse și depind de scopul în care sunt folosite. Câteva exemple sunt: executarea directă pe sistemul gazdă, executarea în containere Docker și executarea dinamică a codului prin integrarea într-o aplicație gazdă. Procesul presupune două etape: transmiterea codului ce trebuie executat și executarea efectivă.

1.4.1. Executarea directă pe sistemul gazdă

Una dintre cele mai simple metode de a executa programe pe alte sisteme este transmiterea programului spre cealaltă unitate de procesare și executarea directă pe mașină. Se pot crea programe specializate care să descarce și execute codul pe sistemul destinație (folosind subprocese de exemplu) sau se pot folosi protocoale special dezvoltate pentru a executa comenzi remote pe alte calculatoare. Există două protocoale de bază în acest sens: *SSH (Secure Shell)* și *Remote Desktop Protocol (RDP)*.

SSH - Secure Shell este un protocol de transmitere în siguranță a comenzilor către un computer printr-o rețea nesecurizată, se folosesc mecanisme de criptografie pentru autentificare și criptare a conexiunii dintre dispozitive, conform [29]. Metoda este foarte utilizată în controlul serverelor datorită securității dată în special de folosirea protocoalelor de criptare cu cheie asimetrică și a autentificării utilizatorului pentru a executa comenzi. Metoda în care se poate executa codul presupune două etape: transmisia codului spre server și apoi lansarea comenzii de execuție. Transmiterea codului se poate face prin executarea unei comenzi care îl descarcă de pe un server web, prin utilizarea *FTP* sau și mai sigur prin folosirea directă a transferului prin *SSH*, întrucât *FTP* nu criptează fișierele ce se transmit, conform [30]. Executarea se poate face prin executarea directă a comenzilor shell specifice sistemului de operare. *SSH* oferă suport pentru *Linux*, *MacOS* cât și pentru *Windows*.

RDP - Remote Desktop Protocol este un protocol ce permite unui client să se conecteze la un computer la distanță. Acesta a fost dezvoltat de Microsoft și îi permite clientului să folosească o interfață grafică pentru a interacționa cu el. Ca *SSH*, și *RDP* este un protocol criptat cu ajutorul cifrului de flux *RC4*, de asemenea oferă mecanisme de compresie pentru reducerea traficului de rețea, mai multe detalii despre protocol pot fi consultate în [31]. Metoda de execuție este mult mai ușoară datorită interfeței grafice, se elimină în acest fel necesitatea de a cunoaște comenzi shell.

Din cele prezentate anterior se observă că există posibilitatea de a executa cod la distanță direct pe mașina gazdă, fie printr-o aplicație intermediară ce acceptă cereri de execuție, fie prin protocoale dezvoltate specific în acest sens. Problema ridicată este securitatea perimitei acestui lucru. Trebuie definit foarte clar accesul pe care clienții îl au pe server. O altă problemă este necesitatea prezenței unor cunoștințe avansate mai ales dacă se folosește *SSH*.

1.4.2. Executarea în containere Docker

O metodă adesea folosită pentru executarea de cod remote presupune containerizarea lui. Există mai multe astfel de platforme printre care cele mai importante sunt: Docker, Kubernetes, Podman, Rkt, LXC(Linux Containers), OpenShift, CoreOS, VMware Tanzu etc. Fiecare dintre acestea are avantaje și dezavantaje. Cea mai folosită fiind de departe Docker, conform [32] aceasta are o cotă de piață de 82.84%. Principalele avantaje ale utilizării sunt:

- *Izolarea* - containerele permit executarea izolată a codului fără a interacționa cu sistemul gazdă sau cu alte aplicații, astfel în container sunt integrate toate dependențele necesare. Elementul amintit anterior asigură o siguranță maximă în cadrul executării de cod remote, se pot stabili și limite (RAM, CPU, GPU, cereri în rețea) în care containerul să funcționeze.

Astfel se elimină posibilitatea ca un cod malițios să afecteze sistemul gazdă.

- *Portabilitatea* - fișierul Docker specifică toate dependențele și setările necesare astfel pe baza lui se poate crea o imagine și apoi un container indiferent de caracteristicile sistemului gazdă. O platformă care execută codul cu această metodă va putea oferi un suport universal, indiferent de mediul suport.
- *Rapiditate* - Pornirea și oprirea containerelor Docker au un overhead minimal, element care aduce o performanță ridicată.
- *Eficiență* - Executarea în container nu aduce o creștere foarte mare a resurselor utilizate (comparativ cu beneficiul izolării totale) și nu există o problemă în executarea mai multor containere pe aceeași mașină. În plus, se oferă și mecanisme de scalabilitate care pot fi folosite.

Metoda are pași asemănători ca execuția directă pe sistemul gazdă, codul trebuie să ajungă pe server, de unde în locul execuției clasice se urmează o serie de pași: creare Dockerfile (dacă nu este furnizat de client), construire imaginii urmată de crearea și rularea containerului. De asemenea, se pot specifica diferiți parametri de execuție a containerului în funcție de regulile organizației.

Există foarte multe platforme care folosesc containere pentru execuție, context în care s-a dezvoltat noțiunea de *CaaS (Container-as-a-Service)*, cele mai relevante astfel de exemple preluate din [33] sunt: Amazon Elastic Container Service (Amazon ECS), AWS Lambda, Azure Container Instances, Google Cloud Run.

În concluzie, folosirea containerelor este de foarte mare actualitate în momentul de față pentru executarea serviciilor în cloud, toate marile companii din domeniul tehnologiilor de acest tip oferă suport. Dintre toate platformele de containerizare *Docker* este cea mai importantă.

1.4.3. Încărcarea dinamică în programul gazdă

Majoritatea limbajelor de programare permit executarea dinamică de cod direct în cadrul unui program prin evaluarea șirurilor de caractere. În Python există metoda `eval()` sau `exec()`, iar JavaScript oferă și ea o metodă `eval()` în acest scop. De asemenea, suportul este oferit și de limbaje precum Ruby, PHP, Java (ScriptEngine), C# etc.

Limbajul Java are la bază folosirea încărcării claselor în *JVM (Java Virtual Machine)*. Acest suport poate fi folosit și pentru a executa cod remote, din fișiere sau direct din conținutul clasei. Practic, spre deosebire de majoritatea limbajelor, în Java clasele sunt încărcate dinamic, adică fișierele `.class` sunt aduse pe rând în memorie folosind încărcători de clasă (eng. *class loaders*). Există mai multe încărcătoare de clasă: unul folosit în procesul inițial de pornire și încărcare a componentelor principale (Bootstrap), unul pentru clase de extensie, cel pentru clase de sistem și unul care poate fi folosit de utilizator prin personalizare conform [34]. Această metodă este folosită pentru încărcarea codului în *Apache Tomcat*. Acesta folosește mai multe încărcătoare de clasă personalizate care implementează *java.lang.ClassLoader*, denumite WebappX [35].

Ambele metode de executare remote direct în cadrul programului gazdă necesită metode pentru ajungerea codului în aplicație de unde el va fi apoi executat, fie prin evaluarea unor șiruri de caractere, fie prin folosirea unor încărcătoare de clasă. Trebuie reținut faptul că se impune redirectarea fluxurilor de intrare/ieșire. De asemenea, metoda introduce riscuri mari de securitate pentru care trebuie oferite metode potrivite de protecție.

Capitolul 2. Proiectarea aplicației

2.1. Prezentare generală

Proiectarea aplicației a început de la definirea funcționalităților pe care aceasta trebuie să le prezinte. Există mai multe funcționalități pentru utilizatori, unele legate de gestionarea contului (crearea și actualizarea diferitelor informații), altele legate de autentificarea în cadrul aplicației (logare și delogare) și funcționalități legate de executarea și publicarea de cod pe platformă. Administratorul are, de asemenea, funcționalitățile clientului (excepție crearea de cont) și, în plus, vizualizarea log-urilor și gestionarea formularelor de contact transmise. Conform logicii, contul de administrator va fi creat intern, fără a exista o interfață grafică pentru acest proces. În figura 2.1 este redată diagrama de *use case*.

Pentru dezvoltarea platformei, am propus o abordare bazată pe microservicii, care a avut inclus încă din etapa de proiectare suportul pentru o scalabilitate ridicată. Astfel, nodurile sunt organizate pentru a putea fi executate pe stații diferite sau în *LAN-uri* diferite. Un alt motiv pentru care am ales folosirea microserviciilor în locul unui monolit este posibilitatea mutării nodurilor ce execută codul remote pe stații diferite decât cele care gestionează efectiv utilizatorii. Mai exact coruperea unui nod de execuție (care este mai vulnerabil datorită codului remote executat) nu ar trebui să poată afecta decât microserviciile de execuție de pe acea stație, unde nu ajung informații relevante legate de utilizator. Aceste noduri pot fi, de asemenea, integrate în containere *Docker*, element care aduce, totuși, un overhead. O altă parte importantă, pe lângă părțile ce se ocupă de utilizator și executare de cod, este cea a componentei de jurnalizare.

În figurile 2.2 și 2.3 este prezentată arhitectura generală a modulelor aplicației pentru care s-a realizat o descriere sumară a scopului lor și interacțiunile dintre ele, apoi o descriere detaliată a proiectării fiecăruia în parte. A fost realizată o diagramă separată pentru partea de jurnalizare din cauza complexității crescute a diagramei principale care ar fi rezultat dacă ar fi fost inclus și acest element.

Cele două aplicații front-end (pentru clienți și administratori) au fost realizate cu ajutorul framework-ului Angular și vor comunica prin *HTTP* cu gateway-ul și nodul de audit și prin *WebSocket-uri* cu nodul master. Nodurile *address microservice*, *feedback microservice*, *code microservice*, *idm microservice* și *gateway* sunt *microservicii* ce gestionează utilizatorii, codurile, sesiunile, autentificarea, echilibrarea încărcării pe nodurile master și sunt create cu ajutorul framework-ului Spring Boot. Alegerea Spring Boot pentru crearea lor este determinată de simplitatea creării acestui tip de aplicații în *Java*, dar și de suportul extins pe majoritatea sistemelor de operare (spre deosebire de *.NET*). Cele mai importante și complexe noduri sunt nodul master și cel de execuție. Nodul master are ca scop gestionarea conexiunilor persistente menținute pentru clienți în timpul executării codului, efectuarea mecanismelor de echilibrare a încărcării pe nodurile de execuție, dar și transmiterea codului de executat spre acestea. Astfel, acesta folosește patru mecanisme de comunicare diferite: *WebSocket-uri*, multicast-uri *UDP*, conexiuni *TCP*, dar și *HTTP*, având o legătură cu gateway-ul pentru obținerea codului publicat pe platformă și a face verificări de securitate privind utilizatorul. Nodul de execuție are scopul de a executa efectiv codul primit, iar în cadrul acestuia, *descoperirea automată* (eng. auto discovery) va fi realizată cu ajutorul *UDP-urilor*. Așa cum s-a menționat și anterior, scopul principal al acestei abordări este de a putea ține separat nodurile de execuție, dar și de a minimiza codul ce va fi scalat (modul va folosi doar *TCP* și *UDP*, deci se va elimina necesitatea unui server web ce va apărea la nodul master din cauza folosirii *WebSocket-urilor*).

În figurile 2.4 și 2.5 sunt prezentate câteva diagrame de secvență (cu principalele funcționalități) care facilitează buna înțelegere a modului în care am gândit comunicarea între module, fapt care a influențat deciziile ulterioare.

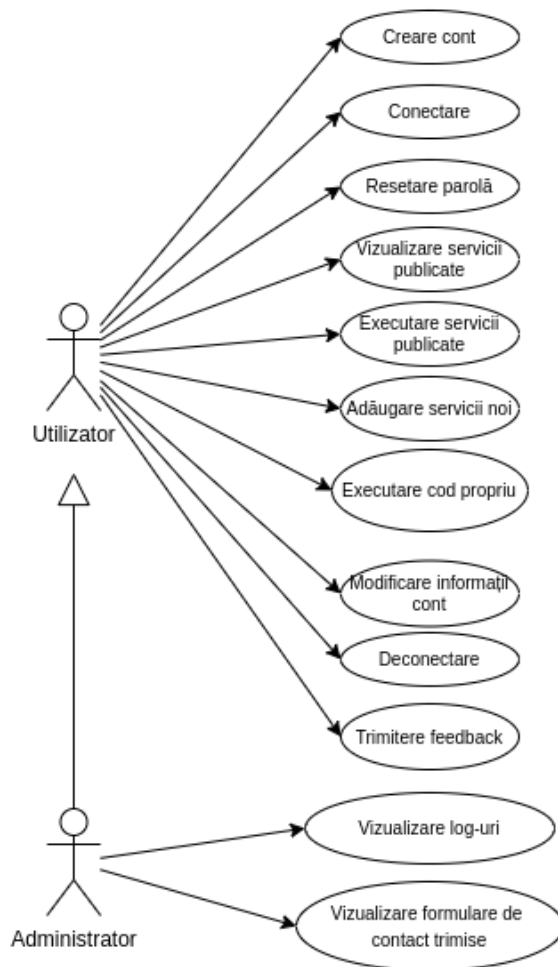


Figura 2.1. Diagrama cazurilor de utilizare

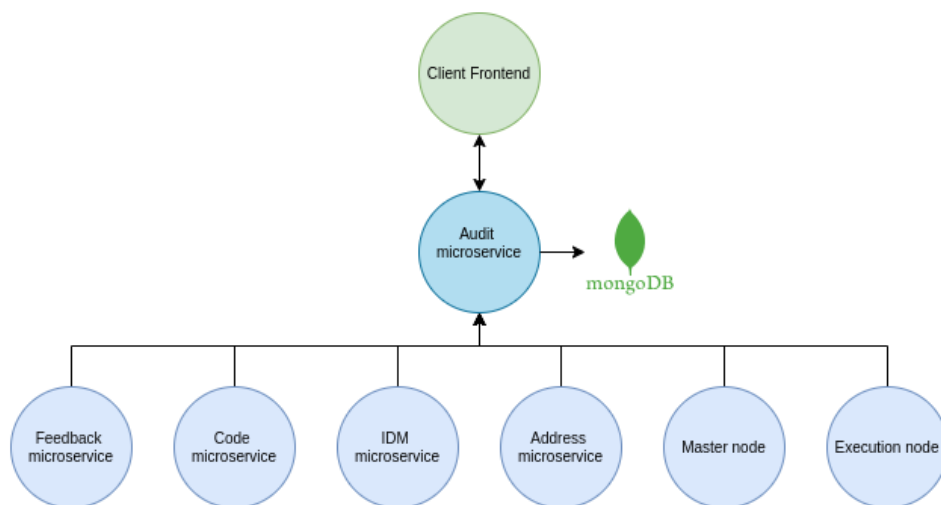


Figura 2.2. Modulele principale ale aplicației - jurnalizare

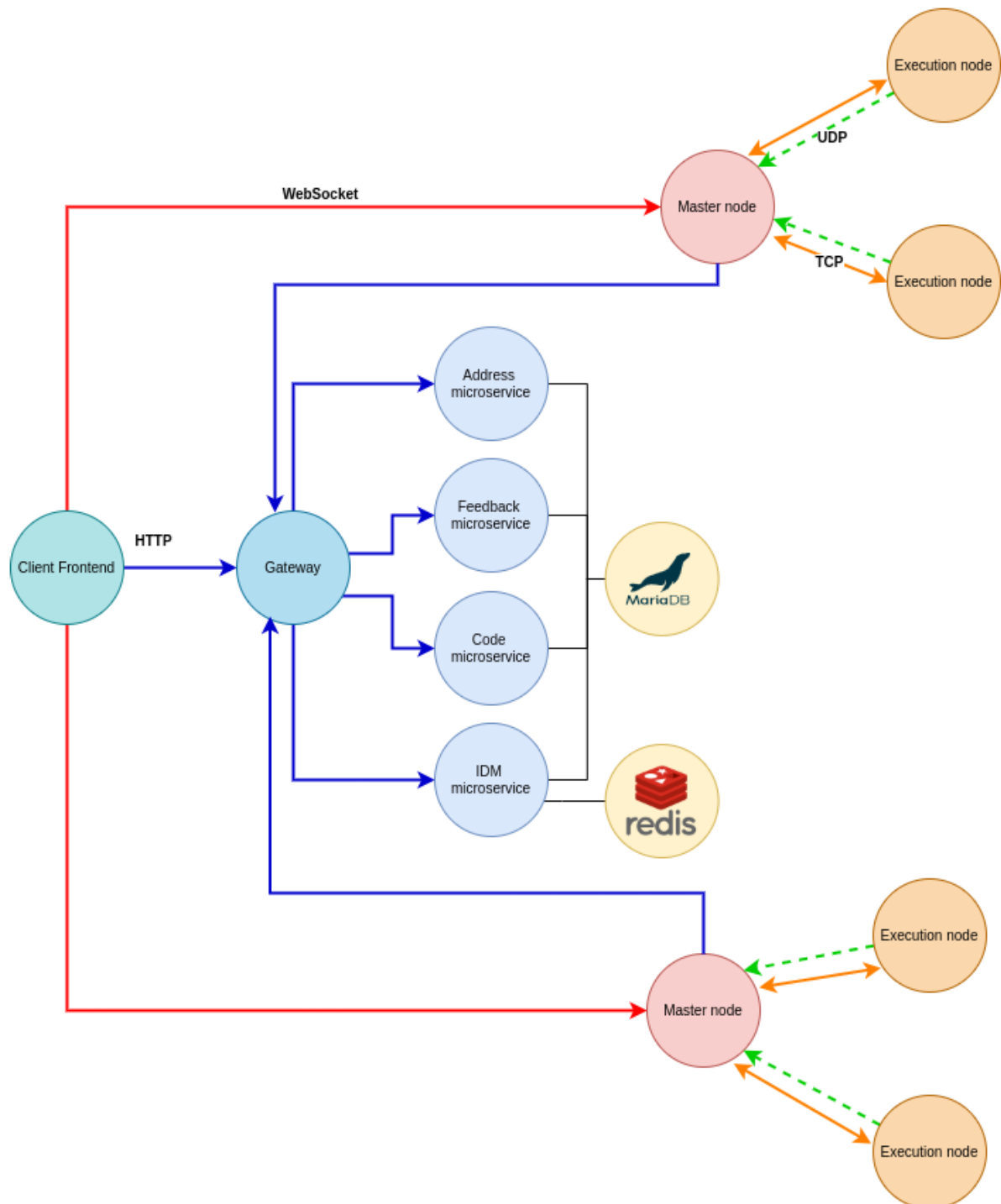


Figura 2.3. Modulele principale ale aplicației

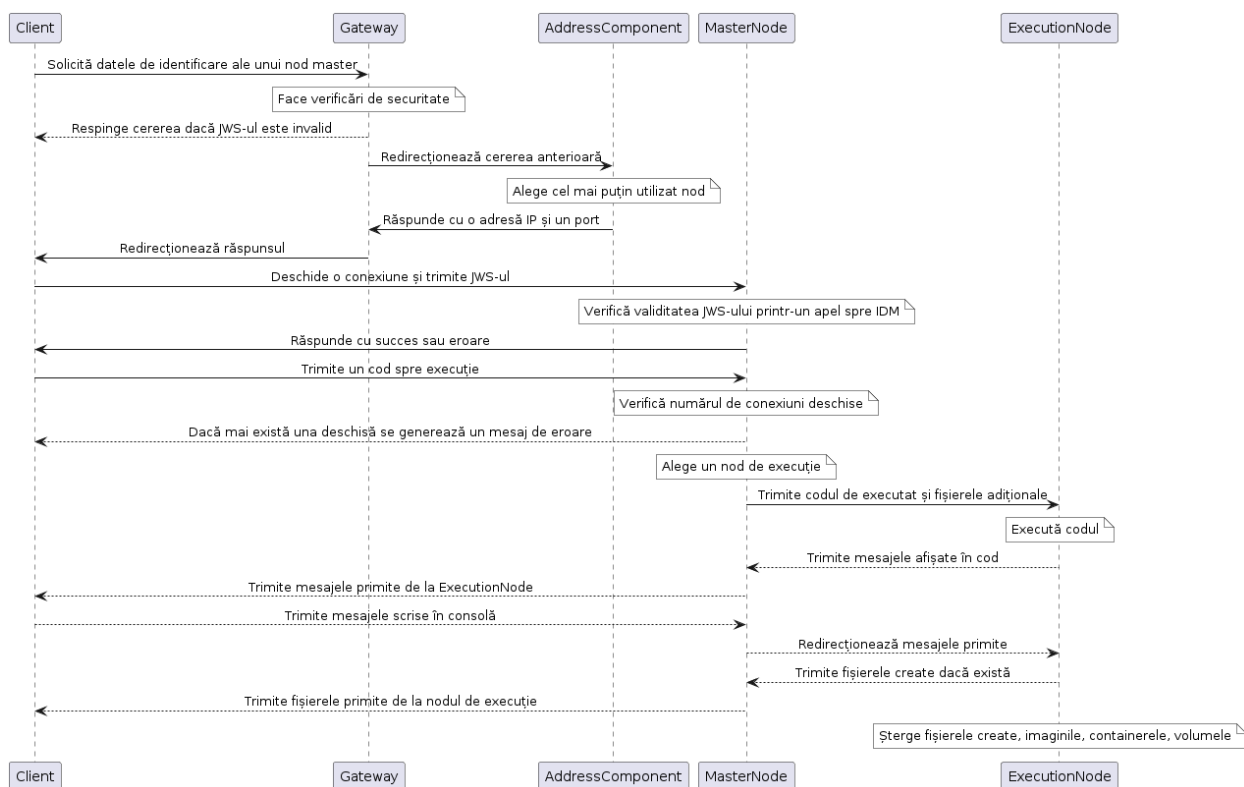


Figura 2.4. Comunicarea între noduri pentru executarea unui cod nou

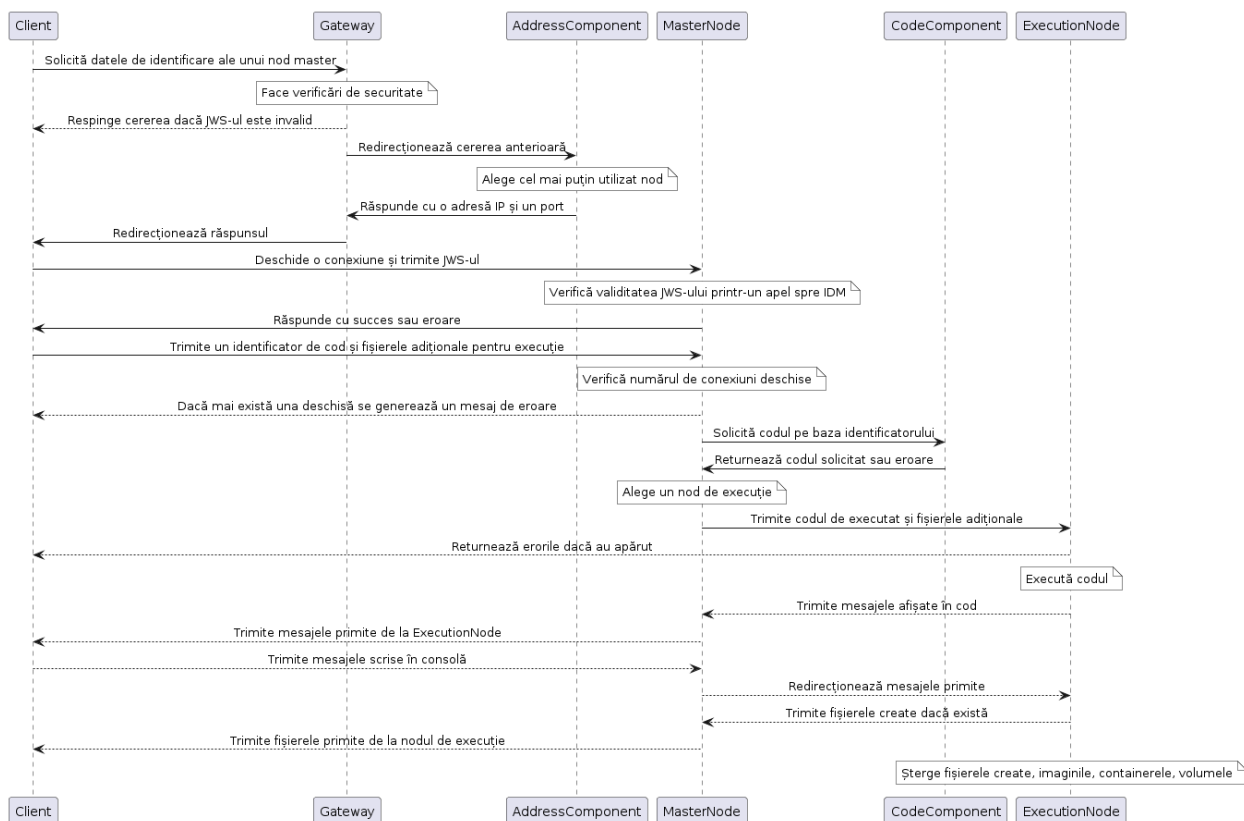


Figura 2.5. Comunicarea între noduri pentru executarea unui cod deja prezent

Figura 2.6 cuprinde o secvență din diagrama de activități, cea mai relevantă. Nu a fost inclusă toată diagrama aici din cauza dimensiunii mari, dar aceasta poate fi consultată în A.9 din anexa 3.

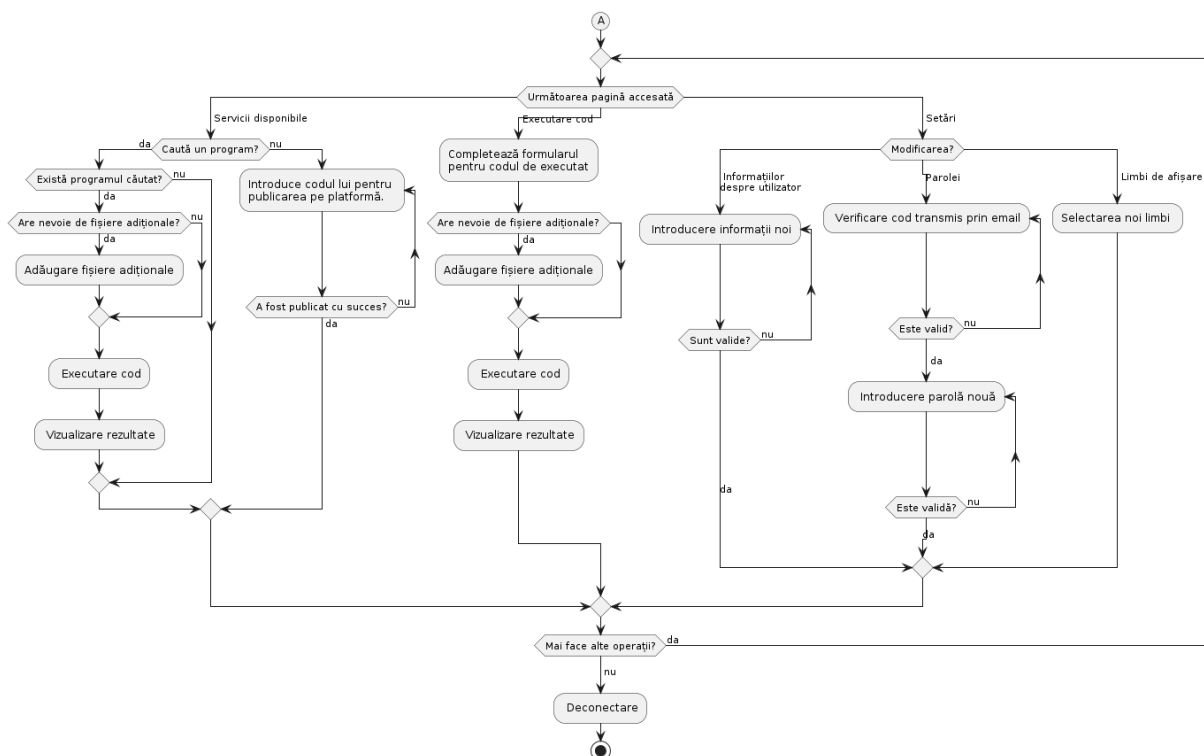


Figura 2.6. Secvență din diagrama de activități

2.2. Aplicații client

Scopul aplicației este de a oferi celor care o vor utiliza posibilitatea de a executa la distanță serviciile publicate de alți utilizatori sau codurile lor proprii în cel mai simplu mod posibil. Necesitatea amintită este motivul pentru care am considerat că cea mai potrivită aplicație pentru acest tip de cerință este una *web*. Astfel, oricine este interesat își poate publica propriile creații sau executa cod publicat de alți utilizatori direct din *browser*, fără a instala aplicații suplimentare. Practic, pagina web devine o poartă spre o multitudine de aplicații care pot fi executate direct din browser. Nu voi implementa o *aplicație desktop*, deoarece aceasta ar presupune pași suplimentari pentru un utilizator care are un scop minor și ar avea efecte descurajante legate de instalarea ei în prealabil.

Pentru dezvoltarea mai rapidă și mai eficientă a acestei părți a platformei, voi folosi framework-urile *Angular* și *Bootstrap*. Există multe cadre de lucru frontend, fiecare cu avantaje și dezavantaje. Cele mai folosite la momentul proiectării acestei aplicații au fost *React* și *Angular*. Am ales *Angular* datorită modului mult mai nativ de organizare a aplicației, dar și datorită folosirii limbajului *TypeScript*, care introduce tipuri de date, reducând astfel erorile apărute din neatenție la scrierea codului. *Bootstrap* este cel mai puternic framework pentru stilizarea paginilor web, iar cu ajutorul lui voi scrie un cod mai compact, reducând semnificativ codul CSS scris. Site map-ul aplicației pentru client este redat în figura 2.7.

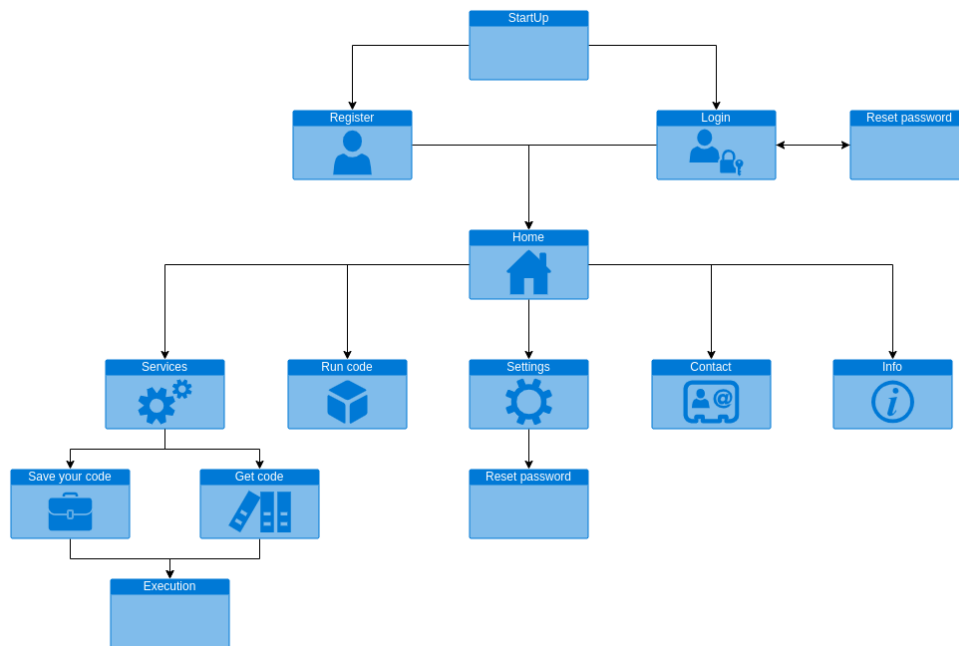


Figura 2.7. Site map-ul aplicației pentru client

2.3. Servicii agregate

Aceste microservicii au fost dezvoltate ca *API-uri web* realizate în *Java* cu ajutorul framework-ului *Spring Boot* și sunt destinate următoarelor elemente de logică: componenta de management al identității (IDM - Identity management), componenta care gestionează codul prezent pe platformă, componenta pentru feedback-ul utilizatorilor și componenta ce va gestiona echilibrarea încărcării pe nodurile master. Toate acestea sunt agregate printr-un gateway realizat cu *Spring Cloud Gateway* și includ câte un *controller* care definește rutele specifice fiecăruia, către care face redirectionări. În figurile 2.8, 2.9, 2.10, 2.11 și 2.13 sunt prezentate diagramele de clase ale serviciilor (din cauza complexității, s-au păstrat doar clasele relevante, adică, s-au exclus DTO-urile, clasele de excepții și clasele destinate jurnalizării). Figurile 2.12 și 2.14 conțin clasele pentru excepții din cadrul IDM, respectiv clasele folosite pentru audit. În anexa 3 sunt prezentate diagramele în forma lor completă.

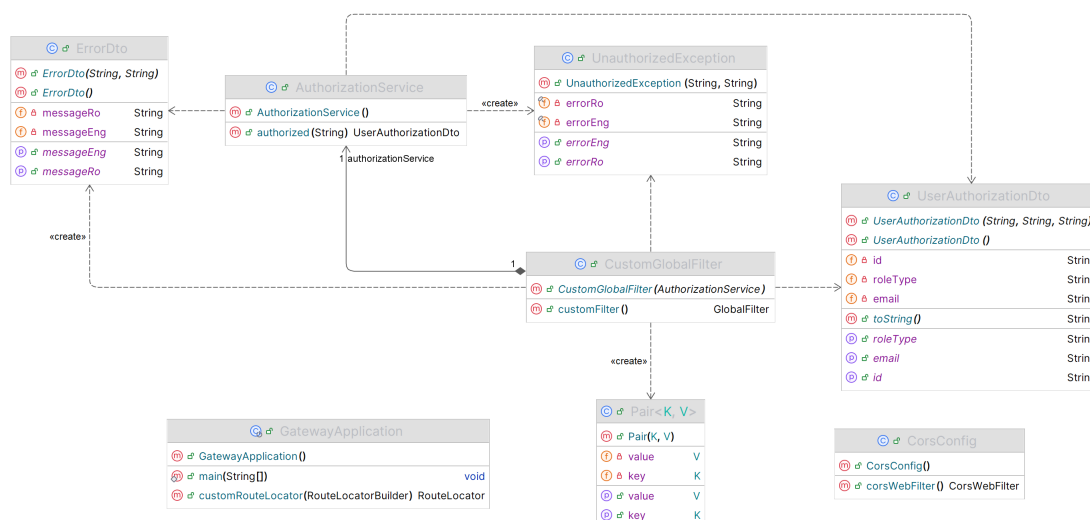


Figura 2.8. Diagrama de clase pentru gateway



Figura 2.9. Diagrama de clase pentru address microservice



Figura 2.10. Diagrama de clase pentru IDM

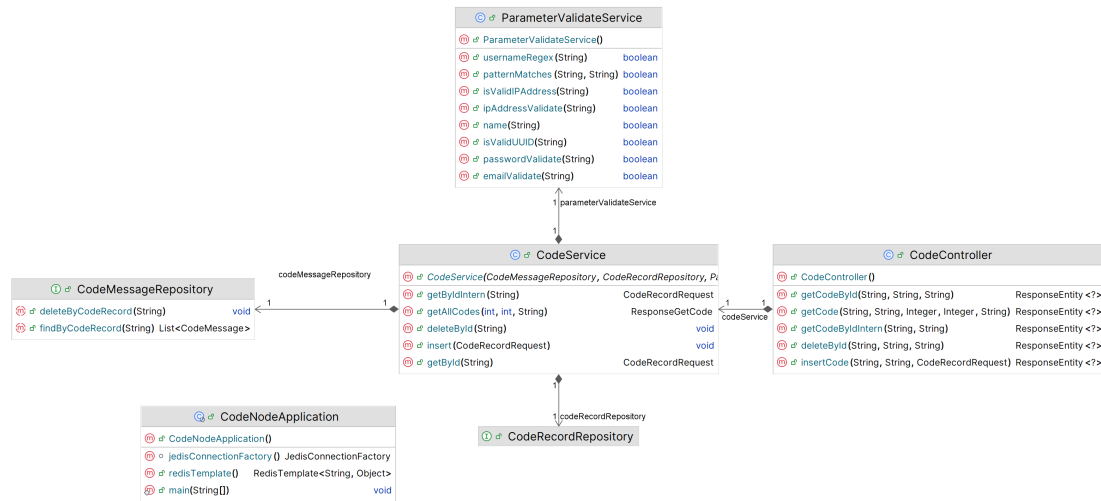


Figura 2.11. Diagrama de clase pentru code microservice

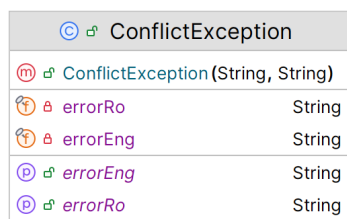
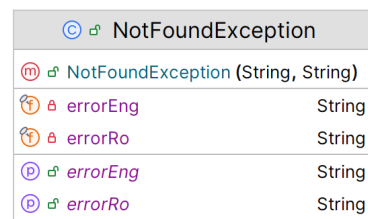
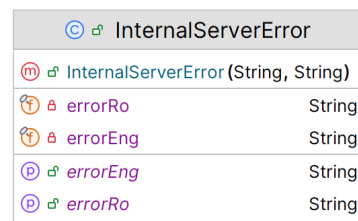
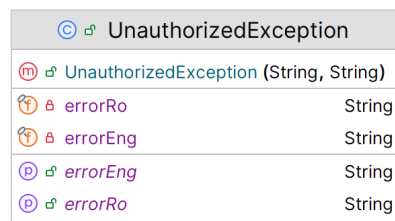


Figura 2.12. Diagrama de clase pentru excepții

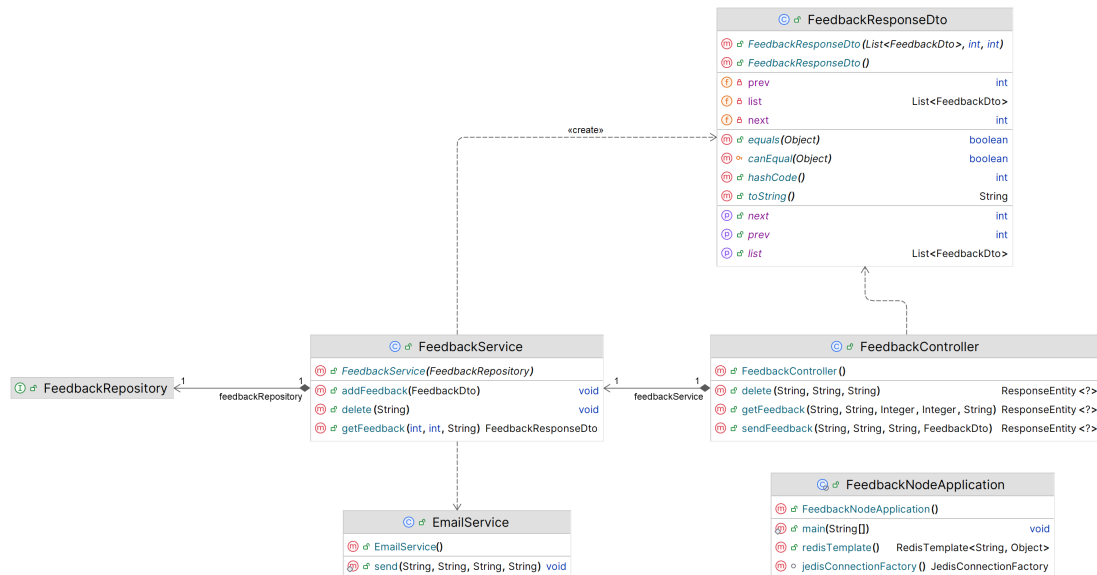


Figura 2.13. Diagrama de clase pentru feedback microservice

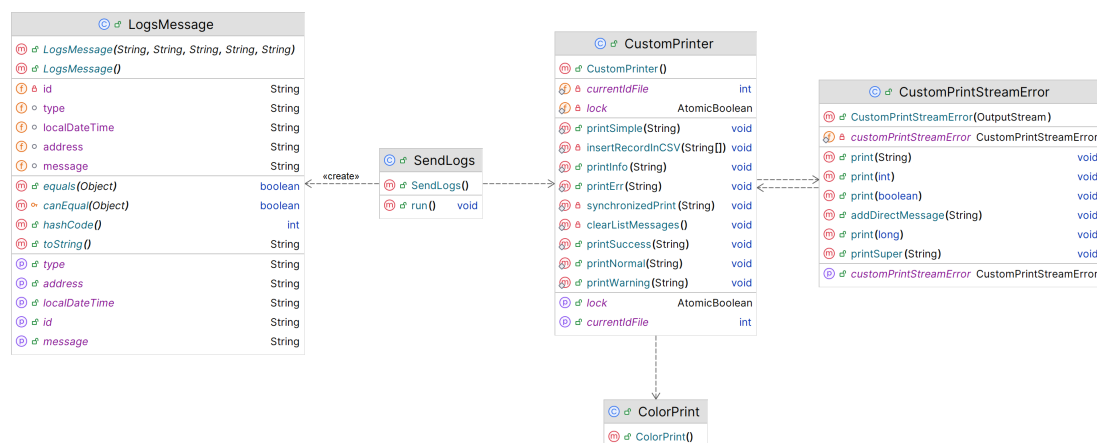


Figura 2.14. Diagrama de clase pentru audit

Un element important al acestor componente va fi securitatea, întrucât vor fi făcute atât cereri din partea utilizatorilor umani, cât și de la nodurile interne ale platformei. În cadrul primului tip, se vor folosi token-uri *JWS*, iar în cadrul celei de-al doilea se vor transmite credențiale, așa cum se face, de exemplu, la integrarea în aplicații a suportului Gmail pentru transmiterea de email-uri. La nivelul gateway-ului se vor defini rutele protejate și cele publice sau care sunt apelate intern. Astfel, pentru validarea *JWS*-urilor clienților se va face în gateway un apel spre IDM, iar credențialele extrase vor fi transmise prin header corespunzătoare celorlalte noduri. În acest mod se restrânge *cuplarea dintre microservicii*.

Partea de feedback va permite trimiterea unor mesaje care va avea ca și consecință salvarea mesajului în baza de date, pentru care va exista și o interfață de unde să poată fi citite și șterse, dar și trimiterea prin email a respectivului formular către un administrator. Metodele de ștergere și vizualizare vor fi disponibile în controller-ul dedicat microserviciului și vor fi integrate în aplicația destinată administratorului.

La nivel de IDM se va permite crearea contului, logarea, suport pentru logica de validare a codurilor trimise prin email (folosită la crearea contului), schimbarea parolei, validarea *JWS*-ului

folosit intern de nodul master, modificarea contului și a parolei, limitarea numărului de conexiuni persistente etc.

Gestiunea codului va presupune posibilitatea de publicare, ștergere, extragere de coduri disponibile cu excluderea fișierelor ce conțin codul efectiv dacă cel care l-a creat nu dorește acest lucru, ștergerea de către utilizatorul care l-a creat sau administrator, dar și un endpoint intern pentru a obține codul necesar execuției.

Componenta ce se va ocupa de gestionarea adreselor IP va avea intern metode de primire de mesaje cu adresa IP și portul nodului master disponibil, de ștergere a acestora, dar și o metodă de get care îi va returna utilizatorului o adresă IP pe care să o folosească, în cadrul căreia vor fi returnate informațiile nodului spre care nu au fost trimise recent multe cereri.

Toate aceste noduri au aceeași arhitectură, există pachete separate pentru controllere (clase ce descriu endpoint-urile aplicației), DTO-uri (Data transfer object-uri) folosite la serializarea/de-serializarea mesajelor, dar și la comunicarea între niveluri, excepții (vor avea mesaje în mai multe limbi, deoarece la nivel de front-end a fost realizată internaționalizarea), notificări (transmitere de email-uri), jurnalizare (similară celei de la nodul master), repository (ce conțin clase ce folosesc ORM-uri pentru bazele de date utilizate), service (conțin clase ce au implementarea logicii aplicației).

La nivel de securitate vor fi create hash-uri pentru parole, se vor impune validări pentru câmpurile de intrare cu privire la dimensiune și format. De asemenea, nu se va permite unui utilizator să aibă două sau mai multe conexiuni de execuție deschise în același timp.

2.4. Componenta master

Componenta *master* este componenta principală a aplicației și cea mai complicată. Aceasta este componenta care face legătura între clienți și nodurile ce execută efectiv codul, se ocupă de partea de securizare și de cea de echilibrare a încărcării pe nodurile de execuție. Diagrama de clase, unde pentru simplificare au fost eliminate DTO-urile, clasele de excepții și cele legate de jurnalizare, este următoarea:



Figura 2.15. Diagrama de clase pentru nodul master

Serviciul cuprinde următoarele pachete:

- *httpClient*, conține clasele *AutoDiscoveryHttp* și *HttpClientAuthorization* alături de DTO-uri și clase de excepție. Este un pachet care conține funcțiile de comunicare cu gateway-ul pentru validare de token-uri și extragerea codului care a fost publicat anterior pe platformă. Scopul acestei logici este de a separa logica de autentificare/gestiune a codurilor încărcate pe platformă de cea de execuție efectivă.
- *notification* este un pachet ce conține clasa *EmailSend* cu funcții ce realizează notificarea administratorului în caz de probleme apărute pe platformă prin email. Situația în care nu există noduri disponibile sau unele noduri devin nefuncționale trebuie adusă la cunoștință, deoarece poate duce la nefuncționarea platformei sau poate fi un semn că există pericole de securitate sau probleme fizice (un atac cibernetic asupra nodurilor, defectarea unor componente etc). Pentru nodul master, o parte de notificare este realizată în componenta de echilibrare a încărcării din nodul ce gestionează adresele IP.
- *others și sendLogs* sunt două pachete care conțin clasele *ColorPrint*, *CustomPrinter*, *CustomPrintStreamError*, *SendLogs* alături de DTO-uri, ce se vor ocupa de partea de jurnalizare, care se va face atât în consolă cât și în fișiere ce vor fi agregate într-un nod central.
- *tcpManagement* este un pachet care gestionează conexiunile TCP între nodul master și cele de execuție, acesta cuprinde următoarele clase *TCPManagement*, *SocketWithExecutionNode*, precum și diferite DTO-uri. Dată fiind problema fluxurilor de date, se impune o conexiune pentru fiecare cod executat. Astfel, la fiecare cerere de execuție se creează o conexiune cu un nod de execuție. În cazul în care nici un nod nu este disponibil, se va trimite o notificare unui administrator, iar clientul va fi și el notificat pentru a încerca mai târziu. Motivul pentru care am ales o conexiune persistentă este că nu se poate defini timpul în care răspunsul este gata sau în care în execuție se solicită date din exterior, metoda de extragere (eng. pulling) fiind inefficientă mai ales dacă sunt multe operații de intrare/ieșire. În cazul în care un nod devine indisponibil, clientul va fi notificat și se va trimite cererea spre un altul, iar cel indisponibil va fi exclus din lista nodurilor disponibile.
- *udpReceiver* este un pachet pentru descoperirea nodurilor de execuție. Se vor folosi pachete UDP pentru eliminarea necesității de a cunoaște un punct central. Astfel, nodurile vor fi descoperite pur automat. El conține clasa *HeartbeatReceiver* și *NodeInfo*, un DTO după care se realizează serializarea/deserializarea mesajelor. Acest pachet va avea sarcina de a prelucra informațiile din pachetele UDP primite, practic va ști ce noduri sunt disponibile. De asemenea, va avea o metodă ce poate fi apelată din exterior în cazul în care, la o conexiune TCP, apare o eroare care marchează că acel nod este indisponibil. Se va stabili un timp în care, dacă nu se mai primesc pachete UDP, se trage concluzia că acel nod este indisponibil și va fi eliminat. Am ales această metodă, deoarece folosirea unui API la care să se conecteze aceste noduri mai mici ar aduce câteva complicații. În primul rând, ele sunt "ascunse" într-un LAN și pot avea în față oricâte noduri master care ar trebui cunoscute pentru a face cereri către ele, descoperirea în acest caz nu ar fi pur automată și ar fi foarte dificil de implementat. La nivel local, unde pachetele nu vor fi diseminate în rețele externe nu ar trebui să existe lacune de securitate.
- *webSocketManagement* conține doar clasa *CommunicationTCPEndpoint*, acest pachet se ocupă de gestiunea conexiunilor directe cu clienții, care a fost implementată folosind Web-Socket-uri din motivul evident al necesității menținerii conexiunii persistente cu ei pentru a redirecționa fluxurile de intrare/ieșire. Dacă platforma ar fi permis doar executarea de cod cu un singur rezultat la final, fără operații de intrare, atunci s-ar fi putut folosi un simplu API HTTP cu pulling, sau în cazul expirării conexiunii să se trimită un email că rezultatul este disponibil, iar el să fie salvat între timp undeva pe server. Au fost folosite mecanisme care să

dicteze ordinea, mai precis, cozi de mesaje sincronizate pentru fire de execuție în interiorul programului. WebSocket-urile presupun o stabilire a conexiunii, moment în care se va verifica dacă token-ul trimis de utilizator este valid sau nu, caz în care se va accepta sau rejecta cererea cu mesaj de succes sau eroare de autorizare. Un principal dezavantaj al metodei este că se va menține o conexiune persistentă, care va consuma mult mai multe resurse.

Pentru că se vor utiliza WebSocket-uri, va fi necesară utilizarea unui server web pentru a face deploy aplicației, pentru care se vor configura dimensiunile buffer-elor. De asemenea și pentru conexiunea TCP va trebui făcută fragmentarea mesajelor pentru a nu depăși capacitatea de transmisie dată de limitările de la nivelul protocolului.

2.5. Componenta de execuție de cod

Nodul de execuție va avea mai multe elemente integrate care sunt: componenta de executare de cod, componenta de gestiune a conexiunilor cu nodul master, componenta de transmitere de pachete UDP, componenta de jurnalizare, componenta de securitate, componenta care se ocupă de agregarea jurnalelor de execuție.

Diagramele 2.16 și 2.17 reprezintă diagrame de clasă din cadrul modului, prima este generică, iar a doua include clasele legate de executarea de cod în JVM. Ca și în cazul celor anterioare, au fost excluse clasele ne semnificative, dar în anexa 3 se găsesc diagramele în forma completă.

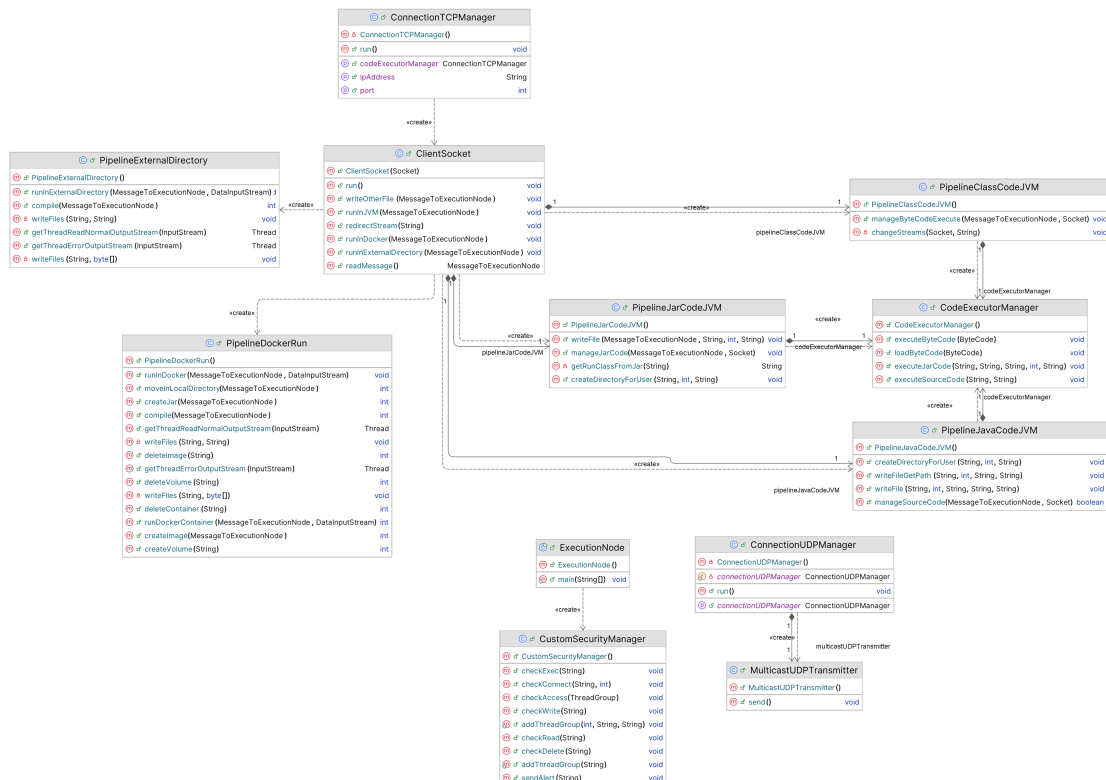


Figura 2.16. Diagrama de clase pentru nodul de execuție

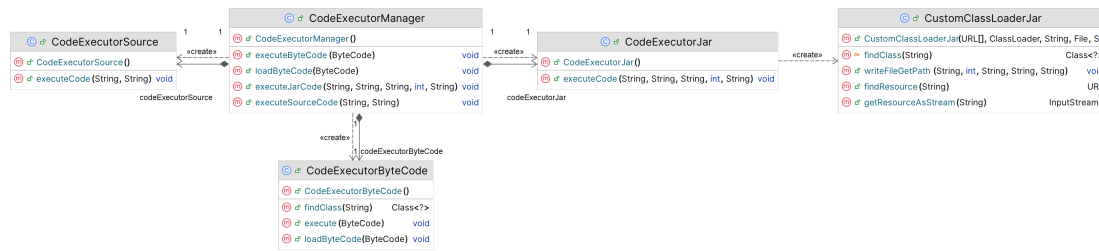


Figura 2.17. Diagrama de clase pentru pachetul de executare de cod în JVM

- Pachetul de executare de cod* este cea mai importantă parte a nodului, dar și a aplicației. Acesta va executa codul clientului în două variante: încărcare directă în mașina virtuală și executarea în containere Docker. Pachetul cuprinde următoarele clase: CodeExecutorManager, PipelineDockerRun, PipelineExternalDirectory, FilesSend, Compile, PipelineClassCodeJVM, PipelineJarCodeJVM, PipelineJavaCodeJVM, CodeExecutorByteCode, CodeExecutorJar, CodeExecutorSource și CustomClassLoaderJar. Prima metodă a fost aleasă pentru a studia modul în care mașina virtuală Java poate executa cod remote într-un mod rapid, dar și problemele multiple pe care această metodă le aduce. În această parte se vor găsi metode ce folosesc încărcătoare de clasă (eng. Class Loaders) care vor încărca programul direct în mașina virtuală. Execuția de cod în Docker va presupune câteva operații pentru crearea fișierului Dockerfile, a imaginii, a unui volum și a containerului, precum și pentru redirectarea fluxurilor de date.
- Pachetul de comunicare TCP* va avea rolul de a primi și transmite mesaje către nodul master și de a executa pași pentru codul de procesare. Acesta cuprinde clasele: ClientSocket, ConnectionTCPManager pentru gestionarea fluxurilor de date și DTO-uri. De asemenea, va gestiona cererile și interacțiunea dintre datele clienților. Va crea câte un fir de execuție separat pentru fiecare client și un director de bază pentru acesta.
- Pachetul de transmitere de datagramme* se va ocupa de multicast-uri UDP care se vor transmite la un anumit interval de timp și pe baza cărora nodul master va ști ce noduri sunt disponibile precum și gradul lor de încărcare. În cadrul lui se găsesc următoarele clase: ConnectionUDPManager, MulticastUDPTransmitter și DTO-uri.
- Pachetul de jurnalizare* cuprinde pachetele other și sendLogs și va avea același rol ca în cadrul nodului master, cu deosebirea că se vor înregistra și date despre autorul operației pentru a identifica ce utilizatori încearcă operații ilegale pe server.
- Pachetul de securitate* se va ocupa de securizarea codului remote executat. Astfel, în cadrul execuției direct în JVM se va avea în vedere o listă de apeluri de sistem permise pentru fiecare utilizator în funcție de tipul de execuție, dar și fișierele care vor fi folosite. În cadrul său există clasa CustomSecurityManager, ce extinde SecurityManager. Din cauza eliminării lui din versiunile recente Java, datorată unui overhead mare de resurse, lipsei implementării pentru toate bibliotecile, dar și utilizări reduse, metoda de execuție în JVM v-a fi eliminată într-o eventuală aplicație disponibilă publicului larg.

2.6. Componenta de jurnalizare

Aceasta este un nod care are ca scop agregarea tuturor mesajelor de jurnalizare de la nivelul celorlalte noduri. El va expune metode de creare și afișare a log-urilor pe cele trei tipuri de noduri (nod-urile agregate, nodul master și nodurile de execuție). De asemenea, se vor oferi mecanisme de filtrare după dată, adresa IP a nodului pentru care se caută în jurnale, a tipului, dar și a identificatorului de utilizator. Aici vor ajunge, la anumite intervale de timp, mai multe mesaje deodată

pentru o eficientizare a transmisiei. Acestea vor fi salvate într-o bază de date nerelațională, deoarece acestea sunt dedicate volumelor mari de date. În figura următoare este prezentată diagrama de clase din care s-au exclus pentru simplificare DTO-urile și clasele de excepții.

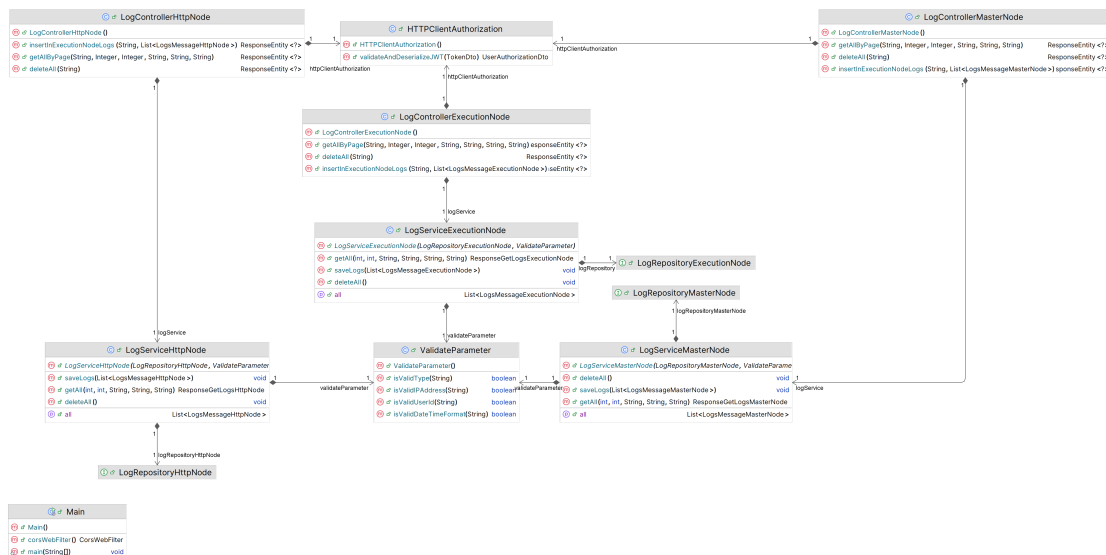


Figura 2.18. Diagrama de clase pentru nodul de execuție

2.7. Baze de date

Aplicația pe care am dezvoltat-o necesită și stocarea persistentă a datelor, motiv pentru care am ales trei tipuri de baze de date care vor fi încapsulate în containere Docker și anume: *MariaDB*, *MongoDB* și *Redis*. Am ales MariaDB, deoarece este o bază open-source, bazată pe MySQL care folosește puține resurse și poate fi integrată în Docker. De asemenea, există deja implementat un *JPA (Java Persistence API)* în Java prin care interacțiunea cu baza de date să fie făcută mai nativ. Pentru stocarea log-urilor se va folosi MongoDB, întrucât este eficientă datorită structurii flexibile a documentelor JSON, performanței și scalabilității sale, precum și capacității de indexare eficiente. *Redis-ul* este folosit pentru un *blacklist* în care vor fi ținute token-urile *JWS* invalide sau folosite la logout.

După cum se poate observa în 2.19, *baza de date relațională* nu are o structură extrem de complexă, deoarece nu se stochează foarte multe informații. Vor exista doar relații *one to many* între tabele iar unele nu au nici o legătură cu celelalte. Tabela *User* este cea în care se vor stoca informații despre utilizator, necesare autentificării (nume utilizator/email și parolă) sau diferite informații personale. Rolul este folosit pentru a identifica dacă un utilizator este simplu utilizator (client) al aplicației sau administrator (caz în care are facilități suplimentare). *Code record* definește structura unei aplicații publicate, și cuprinde, așa cum se observă, id-ul utilizatorului ce l-a publicat (pentru ștergerea în cascadă a codului când dispare utilizatorul), dar și diferite informații de căutare sau utile la determinarea modului de execuție. Tabela *Code_message* salvează efectiv codul publicat. Dimensiunile au fost alese în concordanță cu limitele dimensiunii fișierelor pe care aplicația este dispusă să le proceseze, iar fiecare aplicație (code_record) poate avea unul sau mai multe fișiere de cod, motiv pentru care s-a introdus câmpul code_record ce reprezintă id-ul aplicației în această tabelă.

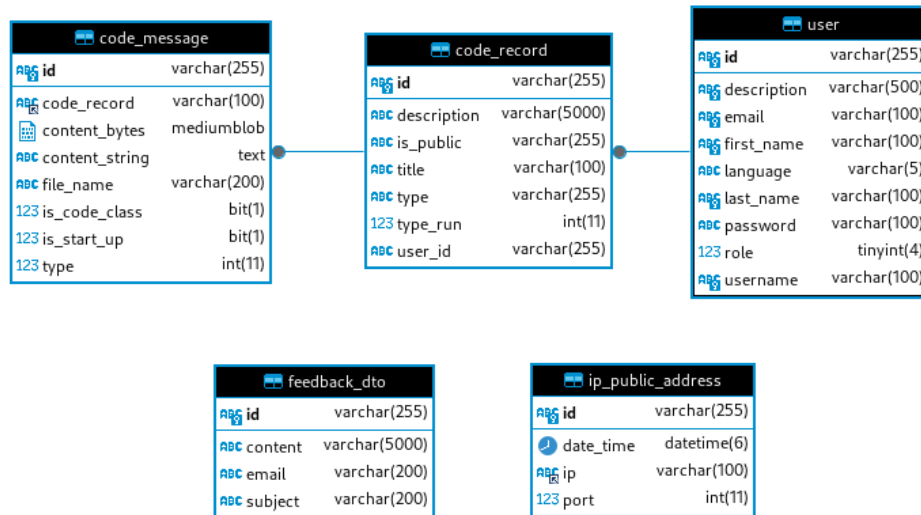


Figura 2.19. Diagrama entitate-relație

Tabela de *feedback* va înregistra mesajele transmise de clienți pentru ca un eventual admin să poată accesa și șterge feedback-ul primit. Se va salva adresa de email și nu un ID, deoarece oricine va putea să transmită feedback fie că are cont sau nu. Tabela *Ip_public_address* are rolul de a salva și actualiza informațiile despre adresele IP și porturile nodurilor master disponibile, folosite la mecanismul de echilibrare a încărcării.

Pentru memorarea log-urilor se va folosi o *bază de date nerelațională*, în care structura colecțiilor cuprinde: data, adresa IP, tipul, mesajul și identificatorul utilizatorului. S-a ales acest tip de bază de date, deoarece este cea mai potrivită pentru volume mari de date. Se vor crea câte o colecție pentru fiecare tip de nod pentru a facilita căutarea. De asemenea, se va crea un index pe câmpul utilizator din log-uri pentru o căutare mai rapidă, în detrimentul creșterii resurselor utilizate.

Capitolul 3. Implementarea aplicației

În capitolul 2 am scris despre modul în care a fost gândită proiectarea aplicației, evidențiind tehnologiile folosite pentru microservicii și interacțiunile dintre ele. În cele ce urmează se va detalia implementarea modulelor descrise sumar în capitolul precedent. Acesta are la bază descrierea următoarelor elemente: interfețele client, componentele agregate, componenta master, componenta de executare de cod, componenta de jurnalizare, bazele de date, precum și detalii de securitate.

3.1. Aplicații client

Așa cum am menționat în capitolul dedicat proiectării, cele două interfețe ale aplicației au fost realizate în *TypeScript* cu ajutorul framework-urilor *Angular* și *Bootstrap*. În fiecare dintre acestea a fost creat un singur modul Angular, cu mai multe servicii și componente, fiecare dedicat unui singur concept logic.

Funcționalitatea sistemului la nivel de front-end este destul de simplă și nativă. Utilizatorul are la început o pagină de start de unde poate selecta crearea unui cont sau logarea. Pentru o securitate mai bună, înainte de crearea efectivă a contului, se va aștepta o confirmare cu un cod trimis prin email. După acești pași, utilizatorul poate intra pe pagina home, unde poate vedea aplicațiile publicate de alții, poate posta propriile sale aplicații, poate executa propriul său cod, poate modifica limba sau informațiile contului său, poate completa un formular de contact sau poate vedea informații despre cum funcționează platforma. La acest nivel este folosit un guard pentru securizarea rutelor.

În cadrul dezvoltării aplicației GUI au fost întâmpinat două aspecte problematice. Primul a fost dat de apelurile asincrone, motiv pentru care am înlocuit fișierul cu mesajele afișate pe interfață (folosit la internaționalizare) cu un JSON stocat în serviciul care gestionează textul afișat (este setată o limbă implicită pe care utilizatorul o poate schimba, caz în care se setează un cookie corespunzător). Metoda stocării tuturor mesajelor afișate într-un array, cu un obiect pentru fiecare limbă, face aplicația deschisă la extinderi lingvistice. Al doilea aspect a fost dat de necesitatea existenței mai multor noduri master, motiv pentru care, înainte de conectarea efectivă prin WebSocket, se va face o cerere către un microserviciu care gestionează echilibrarea încărcării pe nodurile master și care va returna adresa IP și portul.

Partea de executare cuprinde trei sub-pagini: una în care se încarcă fișierele ce conțin codul sau apar detalii ale codului deja publicat, una de încărcare de fișiere și una cu terminalul și fișierele rezultate. Aceasta din urmă este cea mai semnificativă, motiv pentru care am inclus câteva capturi de ecran în cele ce urmează.

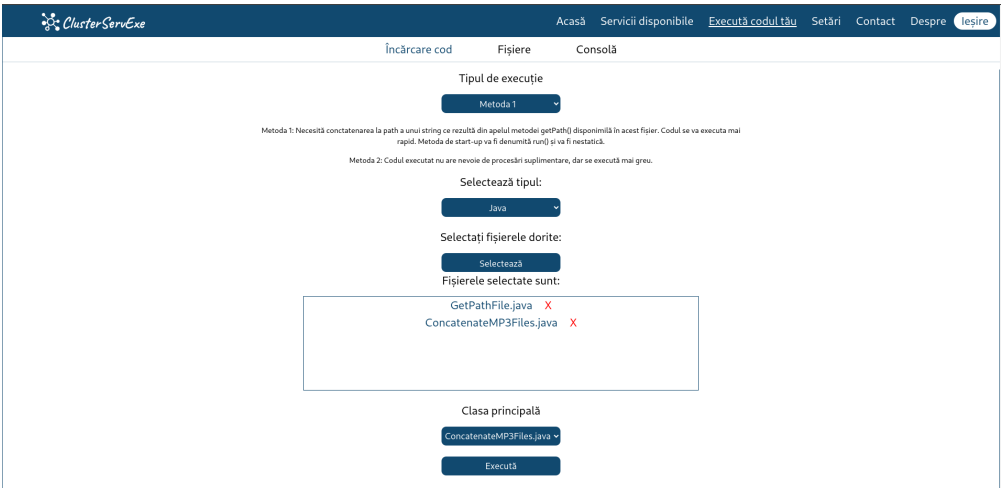


Figura 3.1. Selectarea fișierelor ce conțin cod

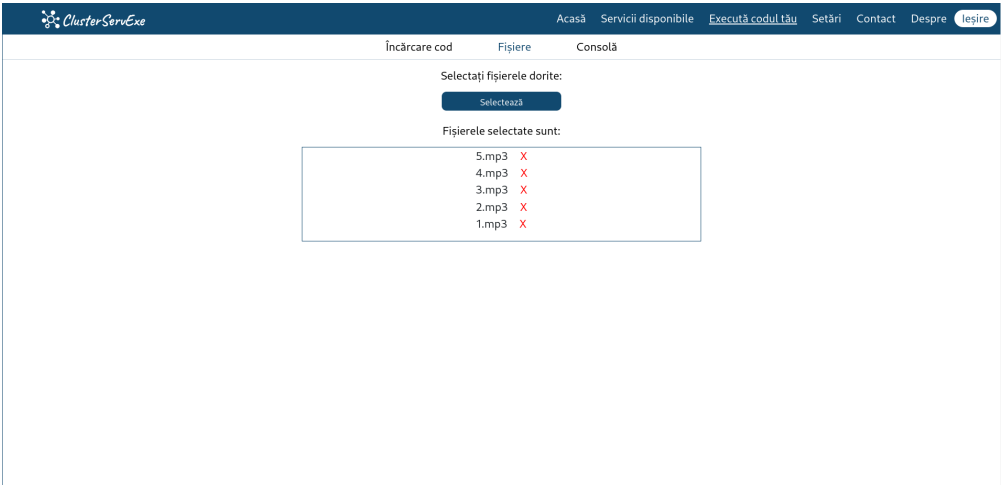


Figura 3.2. Selectarea fișierelor auxiliare

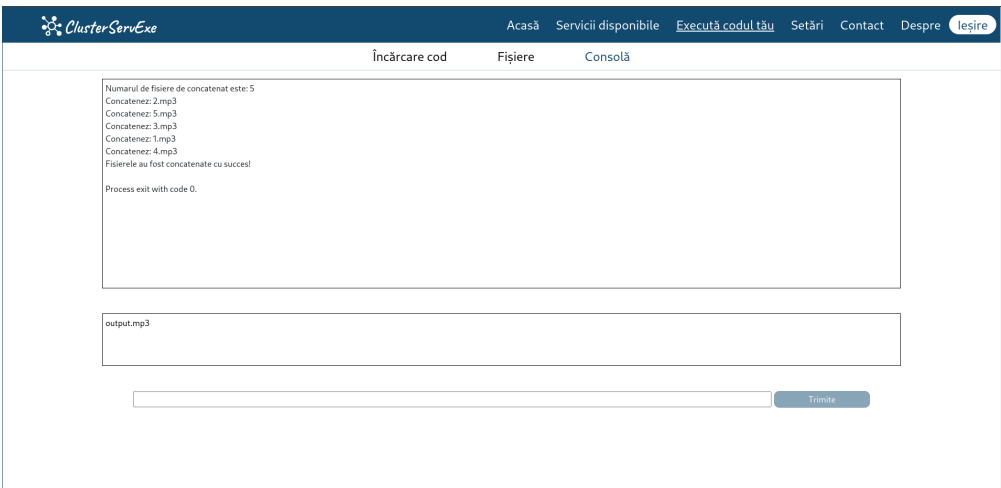


Figura 3.3. Rezultatul execuției

3.2. Servicii agregate

În cadrul platformei dezvoltate a apărut necesitatea de a stoca anumite date, de a oferi anumite servicii care nu necesită o conexiune persistentă și de a crea un punct central de acces, dar

care să facă față unui număr mare de cereri. Din acest motiv, am dezvoltat o grupare de servicii ce primesc cereri HTTP în Java cu ajutorul framework-ului Spring Boot, așa cum am menționat în capitolul anterior. Pentru o mai bună organizare a codului în cele patru servicii, acesta a fost împărțit în pachete: *controller*, *dto*, *exception*, *other*, *repository*, *sendLogs*, *service*, care vor fi detaliate în cele ce urmează.

Pentru fiecare microserviciu a fost dezvoltat câte un controller care se ocupă de un concept logic specific aceluși serviciu: *CodeController* pentru codul publicat pe platformă, *FeedbackController* pentru mesajele trimise de utilizator, *IDMController* pentru gestionarea identității utilizatorilor (creare cont, login, logout, schimbare parolă, actualizare utilizator, validare de JWT-uri pentru nodurile interne etc.) și un ultim controller, *IPAddressManagementController*, care reprezintă punctul central de intrare în partea de bază a aplicației, cea de executare de cod. Aici se adaugă noi adrese IP pentru nodurile master, se șterg adrese IP pentru nodurile master și se extrage o adresă nouă pentru un utilizator (în serviciul aferent se realizează echilibrarea încărcării pe baza unei cozi circulare). Pentru a simplifica dezvoltarea interfeței GUI și pentru a expune mai facil API-ul, endpoint-urile au fost adnotate pentru generarea automată a documentației *OpenAPI*. Astfel, fiecare metodă prezintă în acest document codurile de răspuns posibile, descrierea rutei, tipul de răspuns și parametrii.

În toate serviciile am creat clase de excepții personalizate pentru situațiile care pot apărea în logica de service și care trebuie prezentate clientului. Acestea conțin două mesaje, unul în limba română și unul în engleză, deoarece partea de front-end este internaționalizată și mesajele de eroare trebuie să respecte acest principiu.

Pentru partea de jurnalizare am creat un pachet separat, comun tuturor serviciilor, în care sunt expuse metodele de jurnalizare și care are două funcționalități: afișarea colorată în consolă a mesajelor de eroare și scrierea lor, inițial, în memoria programului, apoi în fișier, pentru a nu face operații de intrare-ieșire la fiecare nou mesaj de salvat. Pentru afișarea excepțiilor în formă de stack trace, am creat o clasă particulară ce extinde *PrintStream* și apelează metodele de jurnalizare proprii. Scrierea mesajelor atât în fișier, cât și în consolă a fost sincronizată prin marcarea blocurilor cu *synchronized* pe obiectele necesare.

Pentru interacțiunea cu baza de date am folosit *Spring Data JPA* (Java Persistence API). Astfel, pentru fiecare tabelă din baza de date a fost creat un DTO ce respectă structura tabelii, pe baza căruia s-au făcut asocierile. Unele metode necesare nu sunt implementate direct de framework, motiv pentru care am adnotat metodele cu instrucțiunile SQL necesare. Conexiunea cu baza de date a fost gestionată de framework-ul Spring Boot pe baza datelor de conectare descrise în fișierul *application.properties*, unde se folosește un utilizator care are doar drepturi *CRUD* (create, read, update and delete) pe tabelele strict necesare.

Pentru trimiterea log-urilor către microserviciul de agregare, am creat o clasă care extinde *Runnable* și este implementată în toate serviciile. Această operație se efectuează în paralel, unde se așteaptă finalizarea operației de scriere într-un fișier (sincronizată folosind un flag de tip *AtomicBoolean*). Dacă fișierul nu este destinat intervalului curent (există un fișier de log pentru fiecare interval de zece minute), conținutul său este citit și trimis către microserviciul corespunzător. Am luat această decizie pentru a minimiza overhead-ul cauzat de stabilirea conexiunii, față de situația în care s-ar transmite fiecare mesaj în momentul apariției acestuia.

Partea de business a aplicației este gestionată în pachetul *service* al fiecărui microserviciu și include mai multe servicii separate în funcție de logica pe care o îndeplinesc. Serviciul *CodeService* din componenta *CodeComponent* se ocupă de management-ul codului publicat pe platformă, inclusiv inserarea, afișarea paginată, extragerea codului după identificator și ștergere. Acesta este apelat de controller-ul corespunzător, iar, la rândul său, utilizează metode din interfețele de comunicare cu baza de date. Serviciul *FeedbackService* din microserviciul dedicat feedback-ului gestionează trimiterea mesajelor utilizatorilor, fiecare feedback fiind trimis prin email și salvat în baza de date. Administratorii pot vizualiza și șterge mesajele. În microserviciul de gestionare a uti-

lizatorilor, clasa *IDMService* definește metode pentru adăugarea utilizatorului, confirmarea creării contului, autentificare, trimiterea codului de resetare a parolei, schimbarea parolei și actualizarea datelor contului. Această clasă folosește un *serviciu de validare a parametrilor* pentru a securiza formatul datelor. De asemenea, utilizează biblioteca *PasswordEncoder* pentru a aplica hash-uri pe parole, asigurând astfel că acestea nu sunt accesibile direct, chiar și în caz de acces ilegal la baza de date. În plus, sunt implementate și servicii pentru generarea *token-urilor JWS*, folosite pentru autorizarea utilizatorilor în anumite operațiuni din aplicație. Ultimul serviciu prezent, care gestionează adresele IP ale nodurilor master, include funcționalități pentru adăugarea și ștergerea nodurilor, extragerea adreselor IP folosind un algoritm de echilibrare a încărcării și ștergerea adreselor atunci când un nod se dezabonează în mod normal prin trimiterea unui mesaj specific. Aceste servicii sunt esențiale pentru funcționarea corectă și securizată a aplicației, asigurând gestionarea eficientă a datelor și a interacțiunilor utilizatorilor și sistemului.

Agregarea cererilor este gestionată printr-un gateway realizat cu *Spring Cloud Gateway*. Acest gateway redirecționează cererile către cele patru servicii care rulează pe aceeași mașină (cu host-ul setat la *localhost*). Astfel, validarea token-urilor JWS este efectuată la nivelul gateway-ului pentru a reduce cuplarea între servicii, iar celelalte microservicii nu pot fi accesate direct din exterior, ci doar prin intermediul gateway-ului.

3.3. Componenta master

După cum am menționat și în capitolul anterior, acest nod este cel mai complicat din cadrul aplicației, deoarece folosește foarte multe protocoale de comunicație și conexiuni persistente, ceea ce implică foarte multe fire de execuție, deci probleme de sincronizare. La nivel înalt, pachetele ce conțin elemente de logică separate au fost definite în capitolul 2, urmând ca în cel prezent să scriu despre modul de implementare a acestora, precum și despre problemele întâmpinate.

Pachetul httpClient conține două clase ce gestionează efectuarea cererilor de autentificare și descoperire automată a nodurilor master spre microserviciul de gestiune a clienților, precum și DTO-urile necesare comunicării.

AutoDiscoveryHttp este clasa ce se ocupă de abonarea și dezabonarea nodului master la cereri din partea clienților. Aceasta implementează interfața *Runnable*, deoarece metoda de dezabonare nu este apelată atunci când nodul se oprește anormal, deci se vor transmite pachete continuu spre componenta de echilibrare a încărcării, care va elimina nodurile de la care nu a mai primit mesaje. Metoda *run* suprascrisă din interfață va efectua tocmai aceste apeluri repetate la perioade constante de timp. Una din *problemele întâmpinate* a fost determinarea adresei IP în mod automat, deoarece aceste noduri pot exista pe mai multe calculatoare, iar configurarea din cod ar fi ineficientă în acest caz (schimbarea rețelei ar necesita modificarea codului). Din cauza lipsei unui număr mare de servere ce să se expună public în internet, testele s-au făcut pe *mașini virtuale EC2*. Determinarea adresei IP se va face prin determinarea interfeței de rețea și a adresei cu ajutorul scanării din Java a interfețelor de rețea disponibile. Metoda de înregistrare presupune un apel HTTP spre endpoint-ul de înregistrare, care este stocat în cloud, într-un container *EC2 (AWS)* și care dispune de o adresă *Elastic IP addresses*, publică. Chiar dacă este un nod intern, accesul lui la nodul http trebuie *autorizat*, motiv pentru care se folosește un header de autorizare (în fapt o parolă lungă și aleatoare cunoscută de cele două entități), metodă similară parolelor de aplicații folosite de Gmail. Metoda de dezabonare are o logică asemănătoare, diferența fiind schimbarea endpoint-ului. În 1 din secțiunea anexe este prezentată o metodă care configurează și returnează un obiect *HttpURLConnection* pentru o cerere HTTP de tip delete către adresa IP publică a containerului *EC2* din cloud și se observă trimiterea header-ului de autorizare intern. Clasa *HTTPClientAuthorization* efectuează apeluri HTTP pentru a verifica dacă token-ul transmis este valid (la deschiderea unei conexiuni persistente) și pentru a semnaliza nodului HTTP terminarea conexiunii persistente. Se efectuează acești doi pași împreună, deoarece pentru un utilizator poate fi doar o conexiune deschisă la un moment dat. Atunci când se utilizează o conexiune persistentă, prezența mai mul-

tor conexiuni pentru un utilizator ar favoriza un atac DOS prin consumarea unui număr mare de resurse de către o singură entitate.

Pachetul notification conține o clasă cu o metodă de transmitere de email-uri utilizată la informarea administratorului despre căderea unui nod de execuție. *Pachetele other și sendLogs* sunt similare tuturor nodurilor aplicației și conține metode de afișare formatată în terminal a mesajelor din cod, pentru salvarea lor în fișiere, precum și pentru transmiterea lor spre nodul de agregare de log-uri. Și în apelul de transmisie de log-uri se folosește ca mecanism de autentificare o parolă cunoscută de ambele entități. Mesajele sunt reținute un timp în memoria programului și apoi un timp în fișier pentru a minimiza overhead-urile introduse de cererile de scriere în fișier sau de apelurile HTTP.

Pachetul *tcp_management* a avut o implementare complexă, deoarece este o legătură între conexiunea cu nodurile de execuție și conexiunea cu aplicația web, motiv pentru care există multe cozi de mesaje sincronizate pentru fire, precum și alte metode de sincronizare. Clasa *TCPManagement* gestionează conexiunile. În ea există metoda care analizează nodurile disponibile și stabilește o conexiune cu ele, dar realizează și echilibrarea încărcării pe nodurile de execuție, prin citirea mesajelor de execuție venite din front-end și transmiterea lor spre diferite noduri de execuție. Clasa mai prezintă și metode de transmitere a mesajelor de la client la nodul de execuție (de exemplu scrierile lui în terminal), precum și de eliminare a nodului din lista celor disponibile în caz de eroare. Fiecare conexiune cu un nod de execuție folosește clasa *SocketWithExecutionNode* care conține socket-ul, stream-urile, cozi de mesaje sincronizate, informații despre nodul partener etc. Aceasta extinde clasa *Thread* și se ocupă de transmiterea de task-uri atunci când nodul de execuție a terminat task-ul precedent, realizând și fragmentarea mesajului în pachete pentru a nu depăși dimensiunea buffer-ului conexiunii TCP. Există un fir deschis în permanență ce citește mesaje de pe front-end pentru a le transmite nodului de execuție pe stream-ul lui de intrare, precum și o parte de cod ce primește mesaje de la nodul de execuție și le transmite spre front-end, inclusiv fișiere. De asemenea, în pachet sunt incluse și obiectele care descriu forma mesajelor ce comunică între cele două entități.

Pachetul *udp_receiver* este cel care se ocupă de gestionarea listei de noduri de execuție disponibile. Astfel, acesta citește în permanență pachetele UDP transmise și le adaugă în lista nodurilor disponibile (la fiecare primire se modifică timpul sosirii mesajului). Pe un alt thread sunt eliminate nodurile care nu au mai trimis mesaje de o perioadă îndelungată. De asemenea, este expusă o metodă care poate fi apelată de celelalte noduri atunci când apare o eroare de conexiune pe un nod, mai exact dacă nodul de execuție devine nefuncțional, eliminarea lui se face mai rapid decât dacă am aștepta un timp până la eliminarea determinată de lipsa pachetelor.

Ultimul pachet al nodului este *web_socket_management*, cel care gestionează conexiunea persistentă dintre nodul master și clienții web. Astfel, clasa *CommunicationTCPEndpoint* este definită ca un endpoint pentru WebSocket-uri cu ajutorul adnotării *ServerEndpoint* din pachetul *javax.websocket.server*, din care se creează câte o instanță pentru fiecare client (de aici și necesitatea de a scala acest nod). Pentru utilizarea clasei se definesc metodele din interfața *ServerEndpoint*. Astfel, metoda *onOpen* primește conform specificațiilor un obiect de sesiune (care poate fi folosit pentru a transmite mesaje spre celălalt capăt) și token-ul specificat în calea din adnotarea clasei, care este verificat printr-un apel spre componenta de IDM pentru a autoriza cererea. Server-ul va răspunde cu eroare sau succes în funcție de autorizarea sau nu a accesului. La deschiderea conexiunii, se va crea, de asemenea, un thread ce va citi dintr-o coadă mesajele din back-end și le va trimite spre front-end. Metoda *onMessage* este apelată automat atunci când se primește un mesaj, caz în care fie se trimite codul de executat spre un nod de execuție direct, fie mai întâi se face o cerere spre microserviciul ce gestionează codurile pentru acesta, iar apoi este trimis spre execuție. Metoda *onClose* apelează automat metoda de informare a nodului HTTP despre oprirea conexiunii, pentru a nu bloca utilizatorul atunci când acesta vrea să înceapă o nouă conexiune.

Clasa *ConnectionManagerNodeInitializer* adnotată cu *WebListener* este apelată automat

de framework-ul *WebSocket API* pentru a inițializa componenta de descoperire a nodurilor master (adică de transmisie a informațiilor despre el), componenta de primire de informații despre nodurile de execuție și componenta de trimitere a log-urilor la intervale prestabilite.

La nivel de server, în fișierul *web.xml* redat în anexa A.10, este stabilită dimensiunea în bytes a mesajelor primite și transmise la maxim 64 MB, o limită similară este pusă și la fișierele încărcate în partea de front-end.

La nivelul nodului master se realizează, așa cum s-a amintit anterior, echilibrarea încărcării pe nodurile de execuție în două moduri. Prima metodă este cea statică, care echilibrează încărcarea pe baza unei cozi circulare prin algoritmul Round-robin. Cea de-a doua metodă ține cont de cantitatea de memorie RAM disponibilă, precum și de numărul de nuclee CPU disponibile. Astfel, lista de noduri este sortată după aceste criterii și se atribuie scoruri în funcție de clasament. Nodul care are pentru suma scorurilor de disponibilitate CPU și de memorie cea mai mică valoare este ales ca nod destinație al cererii de execuție. Codul pentru aceste metode este redat în listing-ul 8 din secțiunea anexe.

3.4. Componenta de execuție de cod

Componenta care constituie baza scopului aplicației și anume execuția de cod, a fost dezvoltată tot în Java și are următoarele pachete: *code_executor*, *conexion_tcp*, *conexion_udp*, *other*, *security*, *sendLogs*.

Cel mai important pachet este *code_executor*, care conține codul ce descrie executarea programelor primite în trei metode: în *JVM*, în *Docker* și direct pe sistemul gazdă ca un subproces (din cauza problemelor grave de securitate, această metodă este exclusă de la execuția către client). Pentru *JVM*, există trei posibilități de execuție: din fișiere *.java*, *.class* și *.jar*. Executarea codului din fișiere sursă *.java* presupune compilarea codului și apoi încărcarea lui în mașina virtuală. Astfel, în clasa *PipelineJavaCodeJVM* sunt descriși pașii necesari, respectiv crearea directorului pentru utilizator (fiecare utilizator are acces doar la directorul său), scrierea fișierului în acel director, compilarea și executarea codului dacă aceasta este clasa de start-up (conform logicii din nodul master, mai întâi se trimit fișierele adiționale, apoi fișierele de cod, iar cel de intrare în aplicație este ultimul). În acest pipeline se tratează și se trimit și excepțiile. Executarea codului presupune încărcarea acestuia în mașina virtuală cu ajutorul unui *URLClassLoader*. Metoda care încarcă și execută punctul de start este redată în listing-ul 2 din anexă. Pentru fișierele adiționale, încărcarea este făcută automat. Este important de remarcat că metoda de start din clasă trebuie să fie non-statică și să se numească *run*.

Executarea în *JVM* pornind de la fișiere *.class* primite de la client începe cu crearea unui class loader personalizat. Clasa *CodeExecutorByteCode* extinde *ClassLoader* și este responsabilă pentru definirea claselor și verificarea pentru evitarea redefinirii acestora. Crearea unei clase se face pe baza unui vector de octeți. În listing-ul 3 și listing-ul 4 din anexe sunt prezentate codurile referitoare la definirea unei clase și executarea codului.

Executarea din arhive JAR folosește, de asemenea, *URLClassLoader*, unde clasa de start este extrasă și metoda *run* este executată (numele acesteia este extras din fișierul Manifest). Funcția este redată în listing-ul 5 din secțiunea anexe. Toate aceste metode menționate anterior redirectionează fluxurile de date, creează directoare pentru utilizatori și trimit fișierele rezultate din executarea codului ca răspuns.

Toate aceste metode presupun redirectarea *fluxurilor de date*. Din acest motiv, obiectele *System.in*, *System.out* și *System.err* au fost înlocuite cu unele personalizate care scriu în socket-uri și citesc din ele, în baza unei asocieri între un utilizator și un *thread group*, așa cum este menționat și în interfața utilizatorului (front-end). Utilizatorul nu poate crea *thread group*-uri; încercarea acestuia va genera o excepție de securitate.

Execuția ca proces extern a fost o etapă înainte de adoptarea execuției în containere Docker. Cu toate acestea, am constatat limitele semnificative ale metodelor de securizare a acestui proces,

ceea ce a determinat renunțarea la posibilitatea de a distribui această funcționalitate către client. Pipeline-ul constă în compilarea, în cazul fișierelor .java, și executarea acestora, unde un proces este creat și fluxurile de intrare/ieșire sunt redirectionate către socket-ul de unde provine cererea. La finalizarea execuției, dacă există fișiere noi, acestea sunt arhivate în format zip și trimise către client.

Executarea în containere *Docker* aduce cea mai bună securitate, dar și un *overhead* major. În clasa *PipelineDockerRun* este prezentă succesiunea de pași pentru execuția în acest mod. Totul începe cu transformarea fișierelor .class (primate de la utilizator sau compilate în prealabil) într-un jar. Urmează crearea unui *volum* pentru fiecare utilizator care va fi șters la final. Pașii următori sunt crearea unei *imagini* în care se specifică jar-ul creat anterior și executarea *containerului* (cu o limitare de memorie la 200MB și interzicerea operațiunilor în rețea). La final, fișierele create în containerul Docker sunt mutate local în folderul utilizatorului, iar containerul, imaginea și volumul Docker creat sunt șterse. Existența volumului este justificată de lipsa accesului pe fișiere după oprirea containerului (și implicit a programului). La fel ca în cazul anterior și aici sunt redirectate fluxurile de intrare/ieșire spre nodul master prin socket-uri. Totuși, aici apar foarte multe probleme legate de faptul că nu există un *API standardizat*. *Docker Java* este un astfel de exemplu, chiar dacă este open source, singura referință de documentație este pe Github, în cadrul codului, care în mod real nu furnizează aproape nimic legat de modul de utilizare ¹⁶. Folosirea *Docker Engine API* este foarte greoaie, trebuie formate request-uri lungi care să includă inclusiv date despre sistemul de operare și arhitectura lui [36]. Utilizarea directă a comenzilor *CLI* este cea mai rapidă, dar trebuie avut în vedere faptul că acest element poate introduce riscuri de securitate. În acest context, se poate executa cod în limbajele *Java* și *Python*.

Pachetul *conexion_tcp* conține clasele ce realizează comunicarea prin TCP cu nodul master. Clasa *ConnectionTCPManager* conține *ServerSocket*-ul la care se vor conecta clienții. Datorită utilizării mecanismelor de echilibrare a încărcării, nu se specifică un port anume de execuție, ci este ales automat. Fiecare client este tratat pe un thread separat cu ajutorul clasei *ClientSocket*, care implementează interfața *Runnable*. În această clasă, se așteaptă mesajele de execuție și se apelează metode din pachetul *code_executor* pentru a executa codul. De asemenea, aici sunt definite și redirectările de stream-uri, precum și verificarea tipurilor de fișiere încărcate. Una dintre problemele întâmpinate a fost modul în care se va deduce pe ce stream trebuie trimis un mesaj, motiv pentru care am impus o limitare de asociere între *utilizator* și un *thread group*. Astfel, task-urile clienților sunt executate într-un thread group separat. Din setările de securitate, se interzice crearea de thread group-uri.

După cum am menționat la nodul *master*, nodurile de execuție sunt descoperite cu ajutorul UDP-urilor (pachetul *conexion_udp* conține clasele pentru transmiterea de *mesaje multicast UDP*). Se transmit mesaje UDP la un interval predefinit, iar informațiile transmise sunt adresa IP și portul server socket-ului.

Clasa *CustomSecurityManager*, care extinde *SecurityManager*, este folosită pentru a bloca crearea de thread group-uri, pentru a limita comenzile executate pe sistemul gazdă, pentru a limita citirea/scrierea/ștergerea de fișiere (inclusiv pe baza thread group-ului pentru a limita accesul fiecărui utilizator la datele lui), dar și pentru a limita adresele IP spre care se pot face cereri. Pentru verificarea tipului fișierelor auxiliare încărcate, se folosește biblioteca *Apache Tika*.

3.5. Componenta de jurnalizare

Nodul de jurnalizare are o structură similară cu nodurile ce deservește cereri HTTP ale clienților și cuprinde următoarele componente: controllere (cu metode de inserare și vizualizare paginată), DTO-uri pentru transferul de date, clase pentru excepții, clase *ODM (Object-Document Mapping)* ce realizează maparea cu baza de date MongoDB, serviciul pentru autorizare (care face

¹⁶<https://github.com/docker-java/docker-java/tree/main/docs>

apeluri HTTP către componenta IDM pentru a verifica token-urile JWS) și servicii pentru salvare și extragere paginată a datelor din baza de date. Pentru legătura cu baza de date se folosește Mongo-Repository, iar credențialele de acces sunt stocate în fișierul `application.properties`. Conexiunea este stabilită și gestionată în mod automat de Spring Boot.

3.6. Baze de date

După cum am menționat și în *capitolul 2*, structura bazei de date *relaționale* nu este foarte complicată, astfel nici comenzile de creare a tabelor și constrângerilor nu au un grad mare de complexitate. Elementul de remarcat este prezența unui utilizator, ce are doar drept de *CRUD* (Create, Read, Update și Delete) pe tablele dedicate lui, așa cum se poate observa în listing-ul 6. De asemenea, un utilizator cu drepturi limitate a fost creat și pentru baza de date *nerelațională*.

Capitolul 4. Testarea aplicației și rezultate experimentale

4.1. Lansarea aplicației și elemente de configurare

Aplicația dezvoltată este executată în cloud, în containere EC2, iar pentru punerea ei în funcțiune trebuie pornite instanțele. O mare parte din nodurile aplicației sunt integrate în containere Docker, iar lansarea lor în execuție se poate face facil prin intermediul fișierului *Docker Compose*. Mai întâi trebuie pornite serviciile agregate și cel de jurnalizare, apoi nodurile master și cele de execuție. Acestea din urmă pot fi pornite sau oprite dinamic în timpul execuției platformei fără probleme.

De menționat că serviciile de IDM, gestionare a codurilor, gestionare a adreselor IP și feedback au nevoie de containere Docker, întrucât în cadrul lor host-urile sunt raportate la acestea. Nodul de executare a codului are și el nevoie ca pe sistemul pe care se execută să fie instalat Docker pentru a putea executa cod în containere dacă este necesar. Fișierul *docker-compose.yaml* conține containerele necesare, precum și dependențele de pornire dintre ele, pentru a evita erorile care pot apărea, de exemplu, dacă un serviciu ar porni înaintea containerului ce conține baza de date și conectarea ar eșua. Nodul master necesită prezența unui server web pentru deploy, iar în acest caz s-a folosit *Smart Tomcat*.

4.2. Testarea sistemului

Testarea sistemului acoperă diverse aspecte, în special testarea API-urilor, testarea metodelor de execuție a codului și testarea sistemului ca întreg. În ceea ce privește testarea serviciilor implementate în *Spring Boot*, am utilizat adnotări pentru generarea automată a documentației *OpenAPI*. Acest lucru a permis utilizarea facilă a *Swagger UI* pentru teste, oferind o metodă mai eficientă și rapidă în comparație cu alternativele precum *Postman*. Pe lângă acest aspect au fost mult mai clare toate situațiile și codurile de răspuns care pot apărea, motiv pentru care s-au putut testa mai ușor toate cazurile posibile. În listing-ul 7 este redat fragmentul din documentul *OpenAPI* destinat creării contului, care a facilitat testarea. Se observă codurile așteptate, descrierea lor, precum și formatul răspunsurilor.

La nivelul bazei de date, dar și la nivel de back-end, au fost impuse constrângeri prin regex-uri pentru o securitate mai bună. De exemplu, pentru crearea unui cont se validează următorii parametri: numele de utilizator, email-ul, parola și numele. Încălcările acestor constrângeri generează codul 422 Unprocessable Entity, care, conform [54], este folosit atunci când sunt înregistrate erori semantice. De asemenea, se tratează și încălcarea constrângerilor de unicitate, situație în care este returnat codul 409 Conflict. O astfel de situație este ilustrată în 4.1.

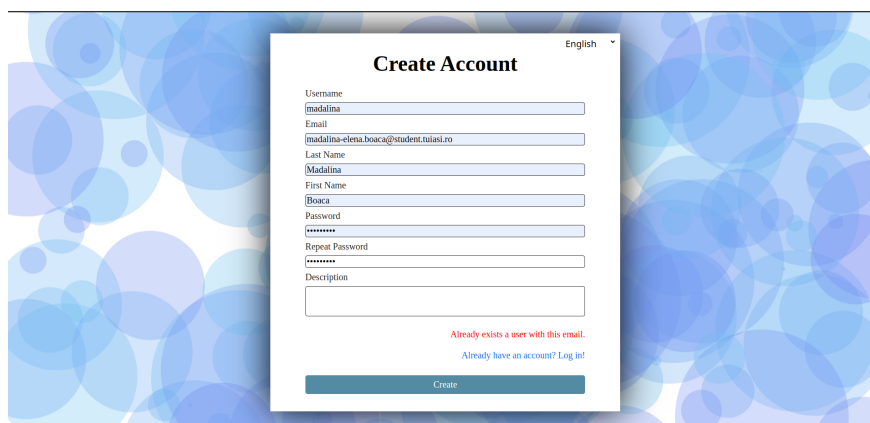


Figura 4.1. Încercarea de creare a unui cont cu o adresă de email deja existentă

Primul serviciu dezvoltat a fost cel de executare de cod, cu funcționalitatea de executare în JVM, moment în care într-un CLI inclus în aplicație se introducea calea fișierului. O dată cu dezvoltarea celorlalte componente am trecut la testarea sistemului ca întreg, iar metodele hibride au fost eliminate. Testarea execuției codului a avut la bază o succesiune de aplicații care cuprindeau operațiile permise sau nepermise pe platformă. Astfel, s-au creat aplicații care fac următoarele operații permise atât în JVM cât și în containere Docker: afișarea de mesaje pe ecran, preluarea inputului din consolă (redat în figura 4.2), scrierea de fișiere și directoare (redat în 4.3), calcul paralel, citirea de fișiere, folosirea mai multor fișiere de cod, pauze în execuție. Dintre operațiile nepermise au fost testate: atacuri de tip path traversal (în JVM, rezultatul execuției este redat în 4.4 în cadrul căruia se încearcă o accesare a presupusului fișier de parole: `"../../../../etc/passwd"`), executarea unor comenzi către sistemul gazdă în JVM, a cărui rezultat este în figura 4.5 și unde se încearcă listarea proceselor sistem, conectarea în rețea externă (în JVM și Docker, un exemplu pentru container este redat în 4.6, iar limitarea memoriei folosite în Docker în figura 4.7). Este important de remarcat că în JVM nu poate fi limitată memoria la nivel de utilizator (un serviciu de execuție deservește mai mulți utilizatori), iar în Docker unele mecanisme de securitate nu sunt necesare datorită mediului izolat.

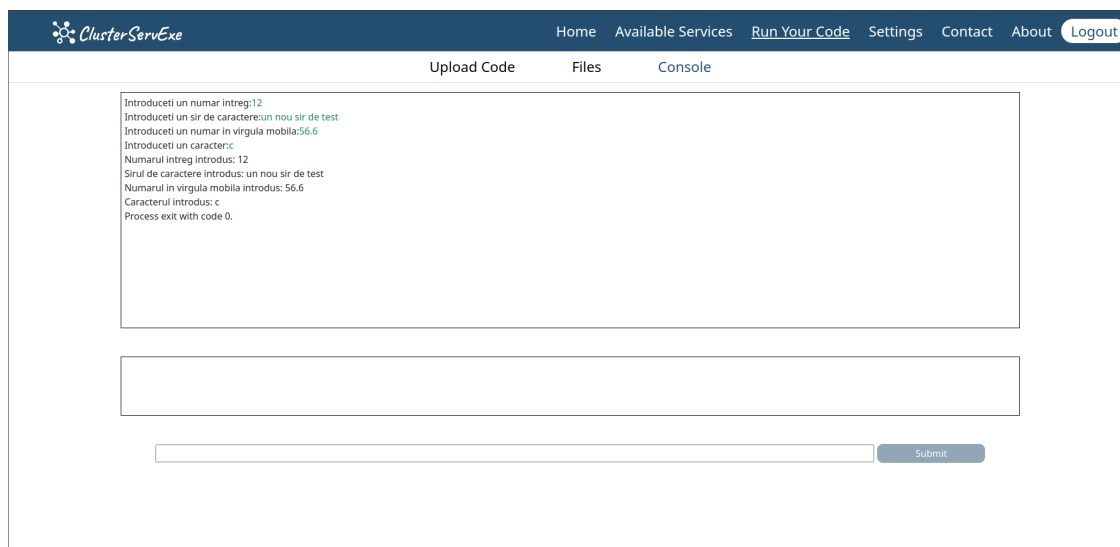


Figura 4.2. Preluarea inputului utilizatorului cu succes, cod executat în JVM

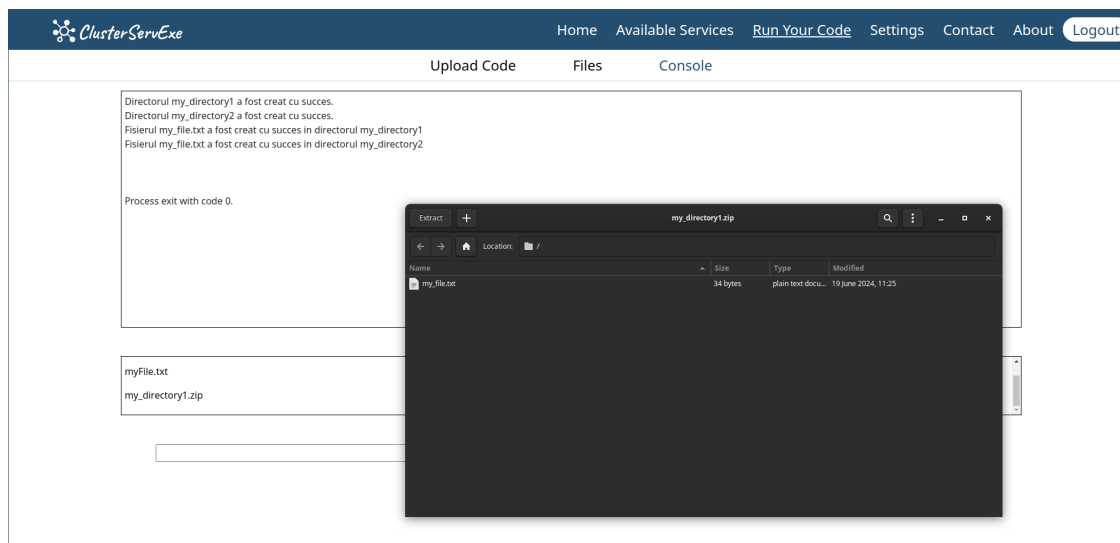


Figura 4.3. Crearea de fișiere și directoare, cod executat în Docker

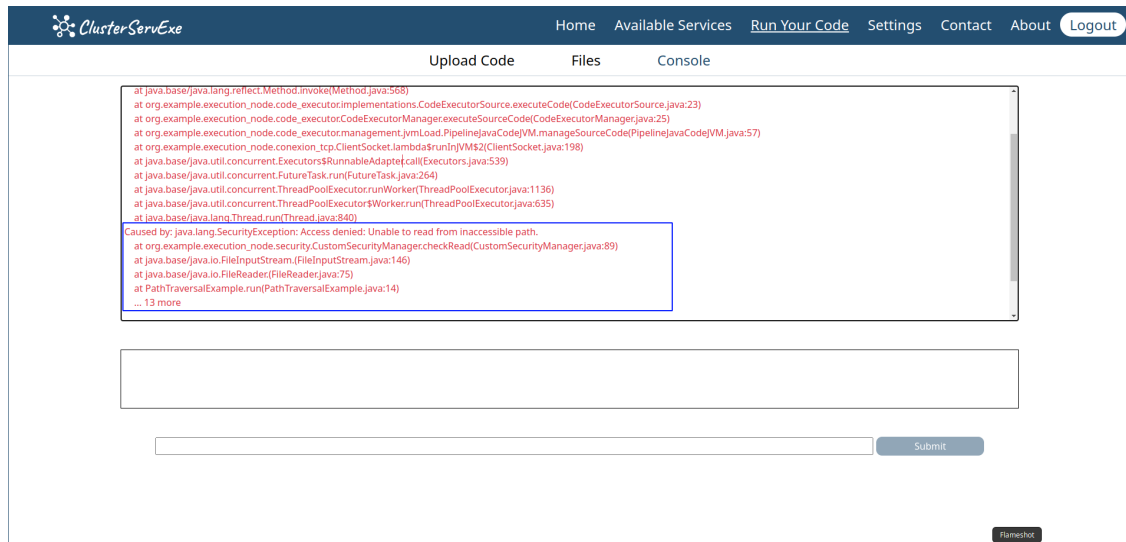


Figura 4.4. Încercare atac path traversal în JVM

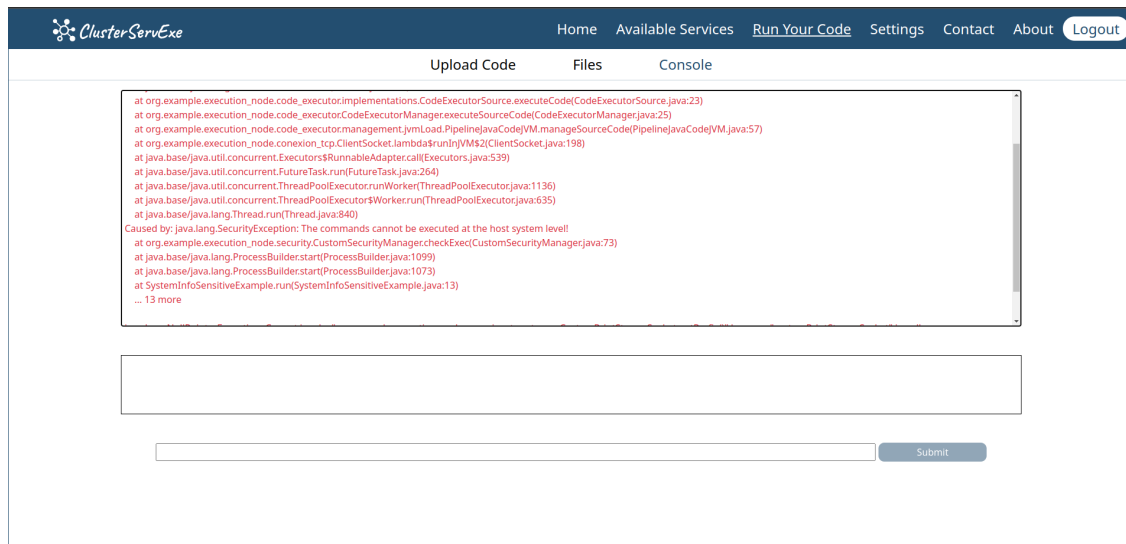


Figura 4.5. Încercarea executării de comenzi către sistemul gazdă

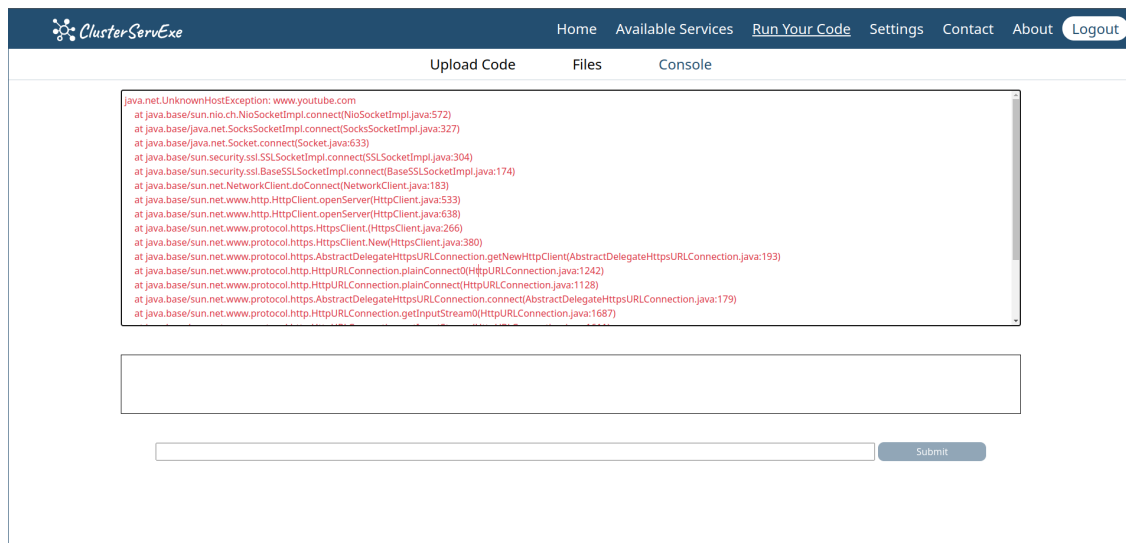


Figura 4.6. Încercarea efectuării de apeluri în rețea în Docker

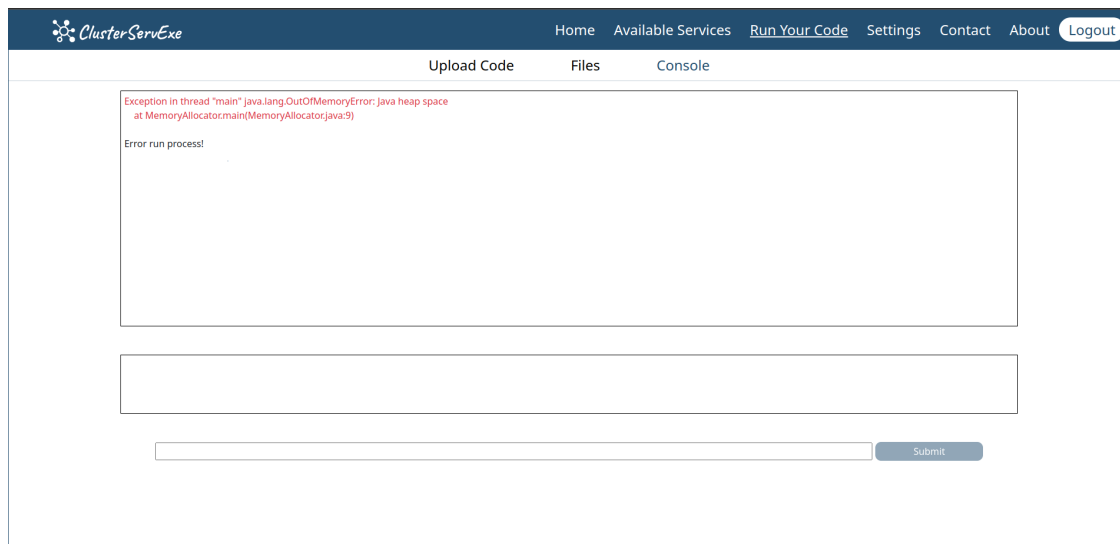


Figura 4.7. Depășirea limitei de memorie în Docker

Un alt aspect demn de menționat este faptul că încercările de a efectua atacuri de securitate prin codul încărcat pe platformă sunt înregistrate la nivel de utilizator. În astfel de cazuri, utilizatorul nu mai are permisiunea de a efectua operații pentru o anumită perioadă după ce a efectuat cinci operații ilegale. Un exemplu în acest sens este prezentat în 4.8. Tot în domeniul securității, există și restricția ca un client să aibă o singură conexiune persistentă deschisă. Prin urmare, la crearea unei noi conexiuni, se verifică utilizatorii deja conectați, iar în cazul în care există deja o conexiune activă, se generează un mesaj de eroare și conexiunea nouă este închisă.

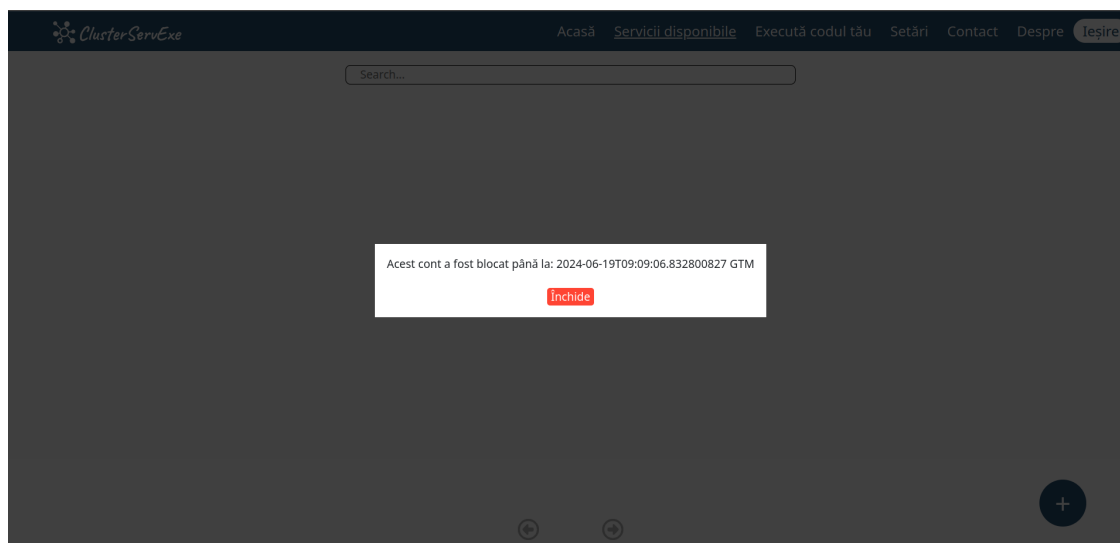


Figura 4.8. Blocarea utilizatorului după efectuarea a cinci operații ilegale

4.3. Aspecte legate de încărcarea procesorului, memoriei și timpului de execuție

Gradul de încărcare al procesorului și consumul de memorie variază odată cu apariția cererilor, motiv pentru care voi prezenta inițial consumul pe care aplicațiile îl au în absența cererilor. De asemenea, prezența containerelor Docker aduce o creștere a resurselor. În continuare, este prezentat un grafic ce ilustrează consumul de memorie al serviciului de *IDM* (*Identity Management*) în Docker și în executarea directă pe sistemul gazdă, în absența cererilor. Toate componentele de *API* sunt dezvoltate în aceeași tehnologie, astfel încât este de așteptat un comportament similar.

Analiza resurselor folosite de componenta *IDM* în urma executării directe pe sistemul gazdă și în Docker a fost realizată folosind două scripturi *bash*, care au creat fișiere CSV cu resursele

folosite de un container și, respectiv, de un proces bazat pe *PID*. Valorile obținute în absența cererilor sunt prezentate în figurile 4.9 și 4.10.

Se observă faptul că nivelul consumului mediu de CPU este inițial mai mare local dar apoi scade și se stabilizează la același nivel, iar consumul de memorie este similar. Astfel, utilizarea Docker nu aduce un consum suplimentar semnificativ de resurse, deoarece diferențele tind să se anuleze la scurt timp după pornirea aplicației.

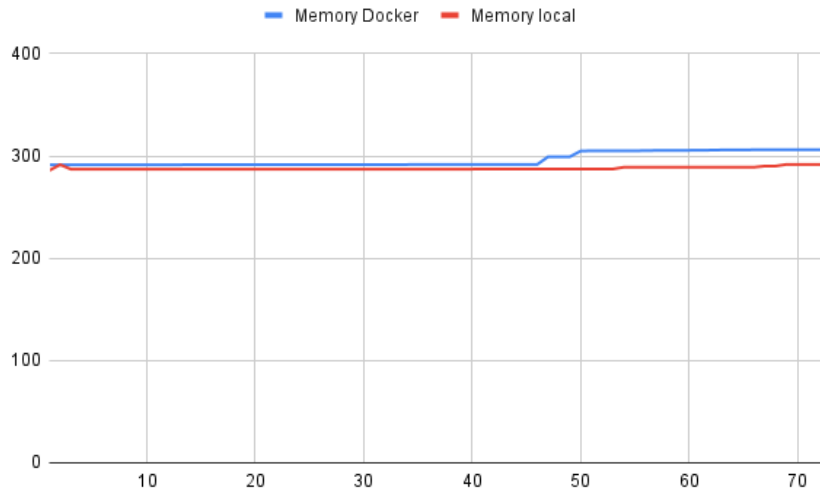


Figura 4.9. Evoluția consumului de memorie pentru componenta IDM

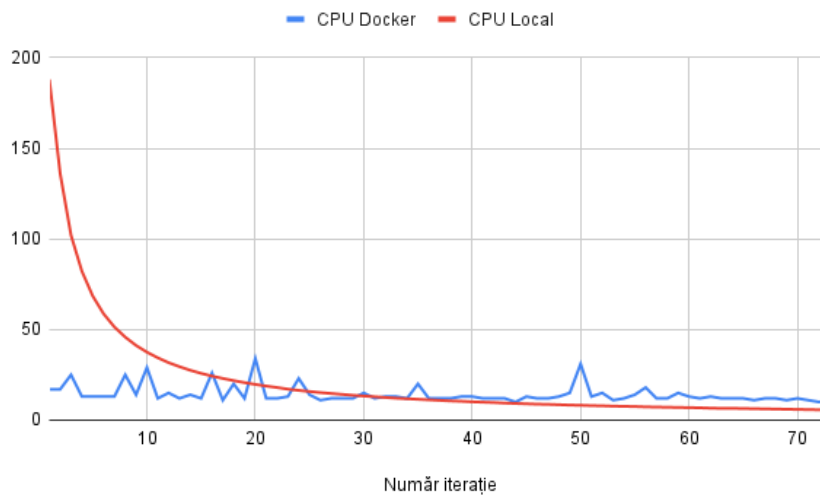


Figura 4.10. Evoluția consumului de procesor pentru componenta IDM

În cadrul graficelor 4.11 și 4.12 s-a analizat procesul de execuție a nodului master. Datele prezentate cuprind și consumul serverului SmartTomcat pe care se face deploy. Analiza a fost efectuată pe o perioadă de aproximativ 75 de minute în care aplicația a rulat independent, fără a interacționa cu alte module. Consumul mediu de CPU a fost de 193%, lucru posibil datorită prezenței unui procesor multi-core în stația pe care s-a efectuat testul. Consumul mediu de memorie a fost de aproximativ 315 MiB.

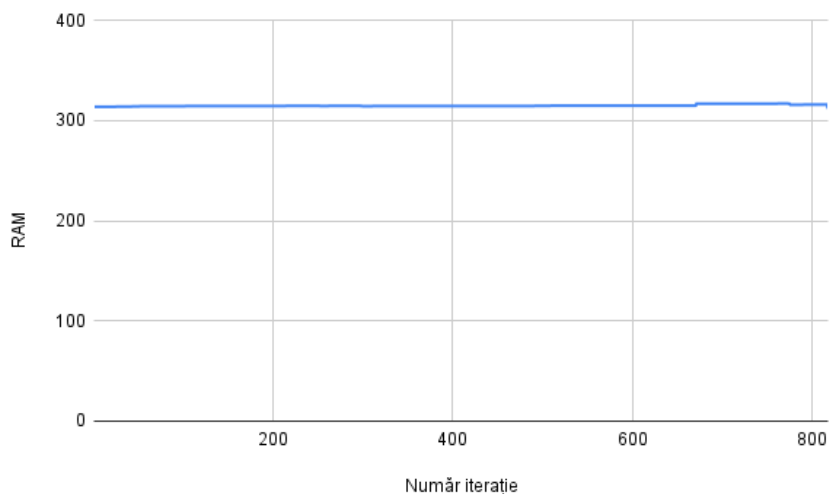


Figura 4.11. Evoluția consumului de memorie pentru nodul master

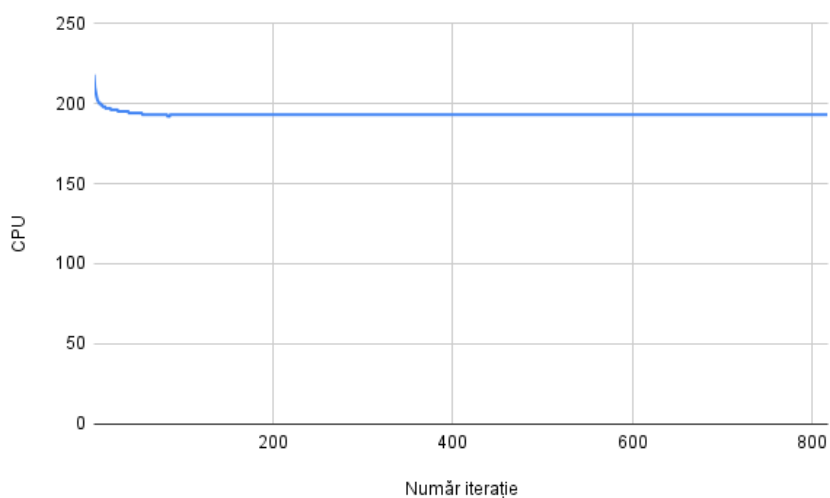


Figura 4.12. Evoluția consumului de procesor pentru nodul master

Și pentru nodul de execuție au fost realizate vizualizări ale consumului de memorie în absența cererilor, iar în figurile 4.14 și 4.13 poate fi consultată evoluția pe aproximativ o oră în absența cererilor. Consumul mediu de memorie este de 59 MiB, iar pentru procesor de 12.11% dintr-un nucleu. Se observă mici variații la memorie, date de anumite operații automate ce se execută la intervale de timp. La nivel de procesor inițial, când aplicația pornește, este înregistrat un consum mai mare care apoi mai scade.

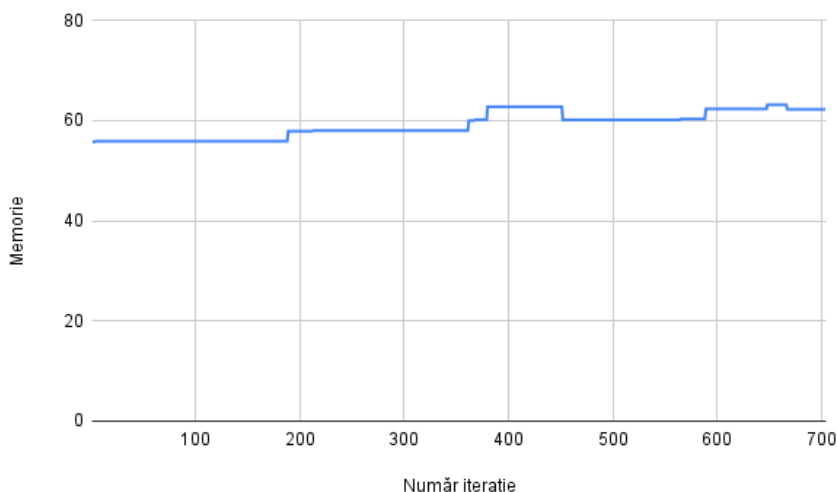


Figura 4.13. Evoluția consumului de memorie pentru nodul de execuție

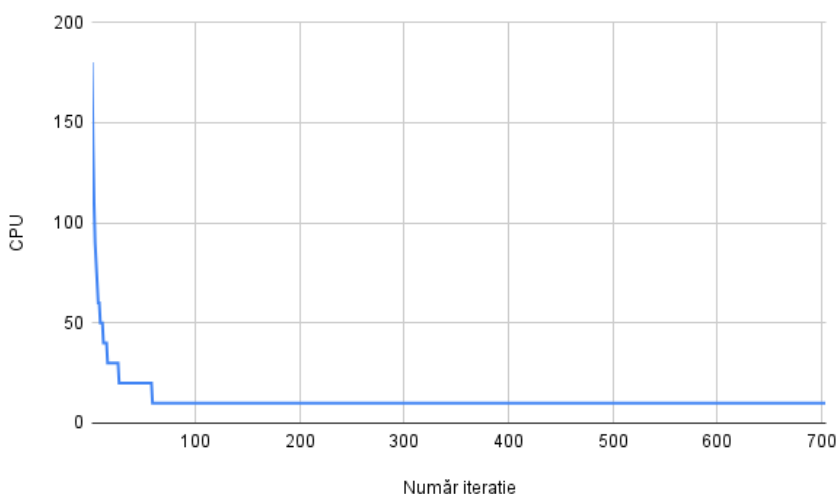


Figura 4.14. Evoluția consumului de procesor pentru nodul de execuție

Din cele prezentate cu privire la nodul de execuție și nodul master se deduc următoarele elemente importante: consumul de procesor este de aproape 16 ori mai mare dacă se folosește nodul ce conține WebSocket-uri, iar cel de memorie de 5,32 ori. Practic, în cadrul pornirii unui număr de noduri care să facă față unui flux mare de cereri, în situația în care codul era executat direct în nodul master, prezența unor noduri neutilizate aducea un consum semnificativ de memorie dacă se foloseau stații diferite și implicit, servere SmartTomcat separate. Chiar dacă toți clienții folosesc un nod master, gradul lui de încărcare scade deoarece el include strict partea de comunicare nu și gestionarea codului remote.

În 4.15 și 4.16 sunt prezentate modificarea consumului de CPU, respectiv de memorie la nivelul nodului master și al celui de execuție. În cadrul acestui test s-au creat două conexiuni element foarte evident din cele două salturi ale consumului de memorie la nivelul nodului master. Programul executat nu a făcut nimic, decât a avut o pauză de zece secunde pentru a permite o valoare de jurnalizare. Au fost zece cereri de execuție cu același cod.

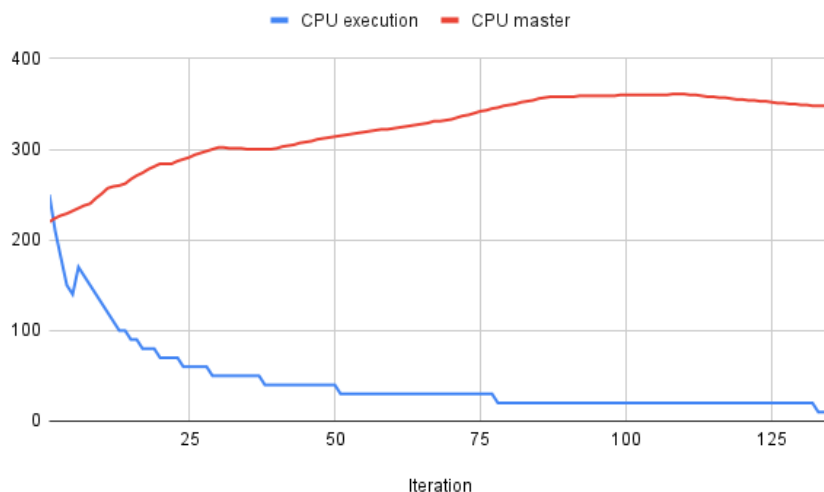


Figura 4.15. Variația consumului de CPU la executarea de cod în JVM

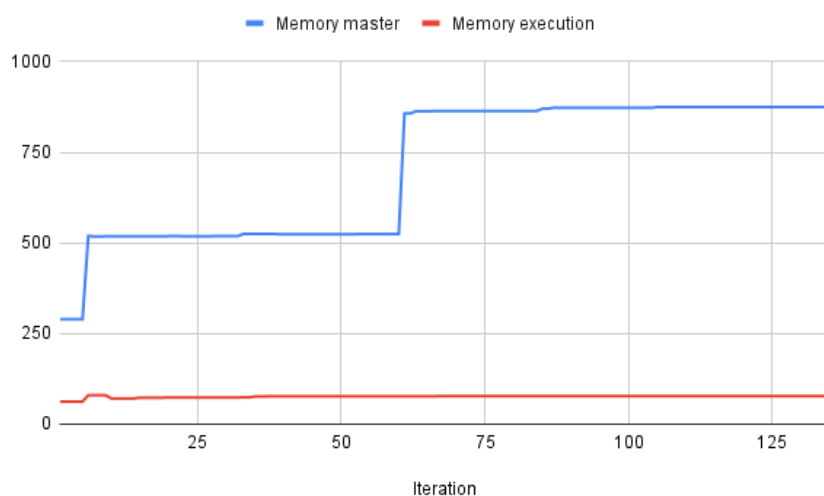


Figura 4.16. Variația consumului de memorie la executarea de cod în JVM

Dacă în cazul evoluției procesorului nu sunt salturi bine definite, la memorie nu se întâmplă același lucru, în cadrul nodului master observându-se în mod evident crearea celor două conexiuni persistente, implicit a firelor de execuție și a resurselor de memorie suplimentare. Cifrele variază semnificativ, respectiv 229 și 332 MiB de memorie. Lipsa modificărilor semnificative în cadrul nodului de execuție arată că, în afară de conexiunile persistente, resursele folosite suplimentar pentru execuție sunt minime, așa cum se poate vedea din 4.17 care este o parte din 4.16.

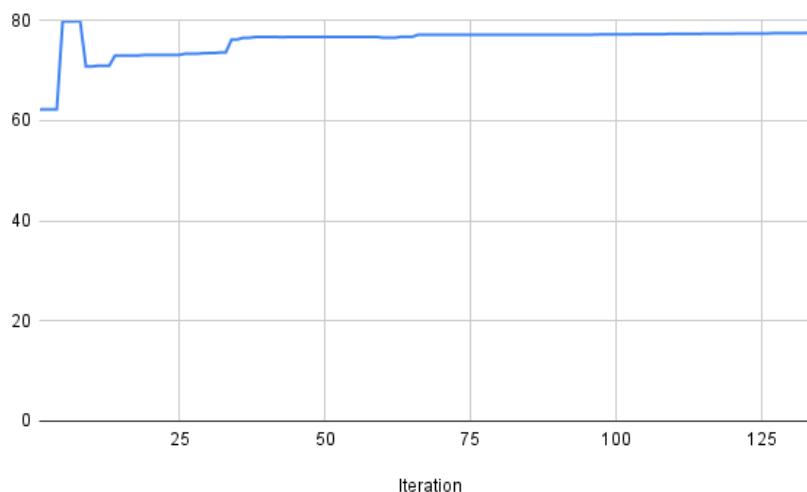


Figura 4.17. Variația consumului de memorie pe nodul de execuție la executarea de cod

Am făcut analize și pe varianta de executare a codului în containere, iar pentru a evidenția strict consumul suplimentar de memorie, codul de test a alocat o memorie constantă de aproximativ 20 MiB. Codul cuprinde o pauză de 10 secunde în execuție astfel încât scriptul ce memorează stările proceselor implicate să poată salva o valoare cu privire la stare. În graficele 4.18 și 4.19 se observă creșterile de memorie corespunzătoare celor șapte execuții de cod. La nivelul nodului de execuție, în anexa A.11 se poate vedea graficul din care se observă că nu sunt modificări semnificative de consum de memorie. În anexa A.12 se observă că nu sunt modificări semnificative nici de consum de procesor. Pentru Docker, media utilizării de CPU este 16.5% (grafic este redat în anexa A.13). Pentru memorie, media utilizării este de 40.54 MiB (graficul A.14 din anexă). Consumul suplimentar dat de container este de aproximativ 20 MiB în plus față de cod, variație aproximativă celei făcute la nodul IDM. Practic, utilizarea Docker nu aduce un consum suplimentar de memorie foarte mare pe lângă codul executat.

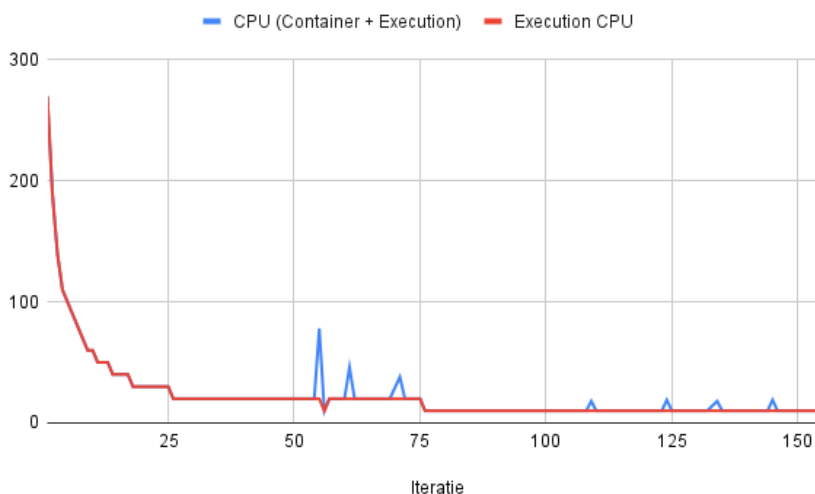


Figura 4.18. Variația consumului de CPU la executarea de cod în Docker

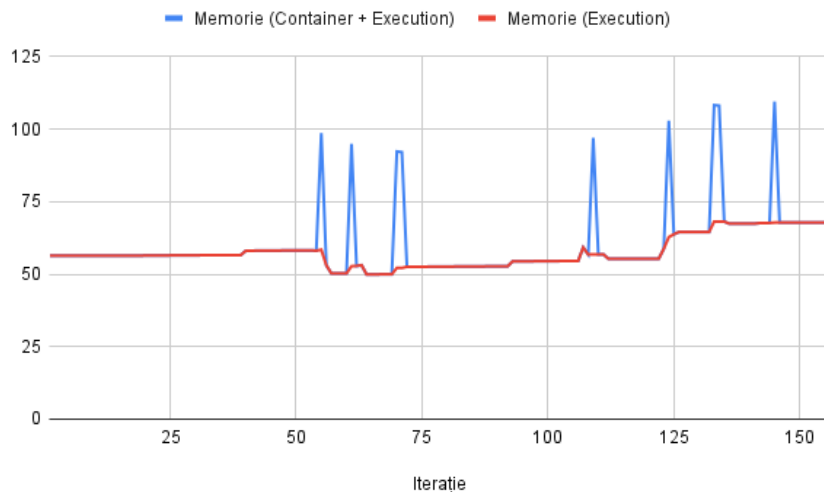


Figura 4.19. Variația consumului de memorie la executarea de cod în Docker

Am făcut și comparații ale timpilor de execuție pe același sistem în care nu s-au efectuat modificări directe ale sistemului, pentru a nu introduce zgomot suplimentar (practic, nu am introdus direct noi procese pe acel sistem). În figura 4.20 sunt prezente 50 de execuții pentru fiecare metodă (timpul este în secunde). Se observă evident faptul că execuția în Docker aduce întârzieri mari chiar dacă la nivelul memoriei și procesorului variațiile sunt minore. O optimizare putea fi refolosirea containerelor, dar acest lucru nu era posibil fără a schimba volumul și implicit a crea din nou containerul. Problema volumului e legată de trimiterea fișierelor către client: pentru accesarea lor, fie volumul trebuie să fie creat explicit, fie copierea să se efectueze în momentul execuției (element greu de implementat). Codul executat în cadrul testelor nu a făcut nimic, deoarece nu am vrut să evaluez decât timpul strict al operațiilor date de pașii pentru execuție. Măsurarea începe la ajungerea codului pe platformă și se termină la finalizare (implică inclusiv operații legate de fluxuri de date, verificări de existență ale fișierelor etc.). În tabelul 4.1 sunt prezentate valorile anumitor parametri pentru timpii rezultați din testare, precum și o coloană în care se exprimă de câte ori este mai mare valoarea de consum la executarea în Docker decât cea din JVM. Se observă faptul că timpii sunt în general mult mai mari și mai variați (dispersia este de 31 de ori mai mare).

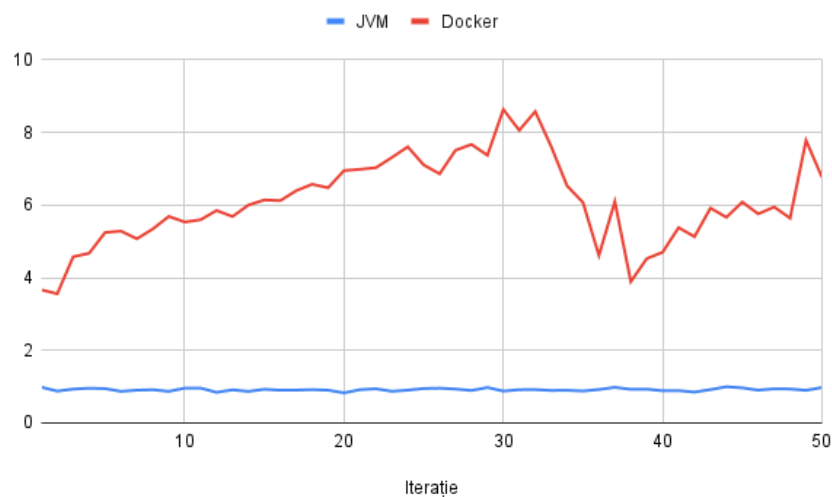


Figura 4.20. Evaluarea timpului de execuție în cele două metode

	Docker	JVM	Raport (Docker/JVM)
Medie	6.105	0.916	6.664
Dispersie	1.201	0.0380	31.544
Minim	3.556	0.728	4.292
Maxim	8.633	0.996	8.666

Tabelul 4.1. Comparatie între JVM și Docker

O concluzie a celor prezentate anterior este faptul că timpii de răspuns la execuția în Docker sunt mult mai mari decât cei de la execuția în JVM. Totuși, așa cum s-a văzut în capitolele anterioare și cum se va evidenția și în 4.4, beneficiile de securitate sunt incomparabile.

4.4. Aspecte legate de securitate și fiabilitate

Securizarea aplicației a fost unul dintre cele mai importante aspecte ale aplicației dezvoltate. Atacurile principale la care aplicația ar putea fi expusă au fost analizate în secțiunea dedicată securității din capitolul 1.2. În capitolul precedent au fost prezentate câteva teste de securitate efectuate asupra aplicației, iar în acesta vor fi prezentate în mod general metodele de securizare luate în considerare.

Prima măsură importantă a fost autorizarea utilizatorilor pe baza JWS-urilor pentru operațiile care nu sunt publice, acestea fiind generate cu ajutorul clasei `Jwts` din `io.jsonwebtoken`. Pentru o creștere a securității, cheia secretă folosită la semnare este regenerată la fiecare pornire a serverului. Orice încercare de utilizare a unui token care are fie semnătură invalidă, fie a expirat sau este deja în blacklist va genera eroarea 401 Unauthorized, în timp ce încercarea de accesare a rutelor specifice adminului va genera eroarea 403 Forbidden conform [55].

La nivel de server, proiecția împotriva SQL injection s-a făcut prin două metode: validarea parametrilor prin regex-uri și folosirea `JpaRepository` din Spring Data JPA, care are suport implicit pentru acest fapt. Așa cum s-a menționat și anterior, în cadrul invalidității parametrilor, este returnat 422 Unprocessable Entity. Pe lângă aceste două metode, la nivelul bazei de date SQL a fost creat un alt utilizator care are drept de acces doar pentru operații CRUD pe tabelele strict utilizate în aplicație.

La nivel de atac *OS Command Execution* în JVM s-au limitat parțial apelurile către sistem permise întrucât există câteva operații pentru care soluțiile din biblioteci nu erau suficiente. În Docker există posibilitatea de a executa orice comandă, dar mediul este oricum izolat. Includ aici și efectuarea apelurilor în rețea, care au fost restricționate, și care în cadrul permisiunii ar putea fi folosite pentru un atac de tipul *Miner de criptomonede*.

Problema încercării de a accesa fișiere nepermise a fost rezolvată în JVM prin utilizarea `SecurityManager`, întrucât varianta de Java utilizată permitea acest fapt. În acest context se verifică începutul căii fișierului de accesat și faptul că el nu conține `"../"`. Varianta de execuție în containere Docker nu a necesitat astfel de verificări întrucât mediul este izolat pentru clientul respectiv.

Pentru *Denial of Service* au fost luate următoarele măsuri: limitarea memoriei consumate de containerul Docker (la nivelul execuției în JVM acest element nu a putut fi implementat), limitarea numărului de elemente cerute la metodele GET pe API-uri prin setarea unui limită pentru dimensiunea pagini, limitarea dimensiunii parolilor pentru că algoritmi de hashing aplicați pot necesita un timp crescut sau multe resurse în unele implementări, limitarea dimensiunii fișierelor încărcate pe server, atât la nivel de frontend cât și la nivel de nod master, limitarea utilizatorilor la o singură conexiune persistentă deschisă. Unul dintre cele mai importante elemente pentru contracararea acestui tip de atac a fost eliminarea unui punct unic de eșec la nodurile ce utilizează conexiuni persistente și sunt vulnerabile, spre deosebire de cele ce acceptă doar cereri HTTP. Prezența mai multor noduri master și a mai multor noduri de execuție aduce și o scalabilitate ridicată a platformei. Din cauza fluxurilor de intrare pentru care în implementarea Java citirea se face la nivel de

octet și nu se poate determina utilizatorul care scrie pe un flux de date în cazul în care acesta agregă mesaje de intrare de la mai mulți utilizatori, s-a luat decizia ca un Socket să fie folosit pentru un singur client. Asta înseamnă că numărul de clienți maxim suportați este egal cu numărul de conexiuni deschise între noduri master și cele de execuție, dar nodurile de execuție nu necesită multă memorie și astfel se pot scala fără mari costuri.

Pentru a nu deschide nici o breșă de securitate chiar dacă există limitări ale operațiilor executate pe server, a fost verificat tipul fișierelor folosind Apache Tika. În capitolul 1.2 am menționat cazul WordPress unde o poză care conținea de fapt un script PHP a fost încărcată pe server. În cadrul aplicației dezvoltate, un atac de tip Arbitrary File Upload este prevenit prin acest test.

De la început am plecat de la problema rețelei în care este executat sistemul, utilizarea unui cluster și cloud-ului aduce după sine astfel de întrebări. În cadrul fiabilității sistemului se vorbește și de rezistența la eșecuri, astfel nodurile master pot fi executate fără probleme în rețele sau pe stații diferite. Nodurile de execuție sunt ascunse în cadrul unei rețele dar pot fi executate pe stații diferite. În acest mod o eroare la nivelul rețelei sau hardware a unei stații nu va lăsa sistemul nefuncțional atâta timp cât sunt prezente suficiente noduri în rețele diferite și pe stații diferite. Un alt aspect este redistribuirea task-urilor spre celelalte noduri de la nivelul nodului master dacă un nod de execuție devine nefuncțional cât acesta așteaptă în coadă, respectiv informarea clientului dacă eroarea apare în timpul execuției pentru a putea reface cererea (reluarea automată a cererii în contextul în care clientul are eventuale rezultate parțiale ar putea determina nedumeriri pentru el). Adiacent cu acest element este și echilibrarea încărcării pe nodurile master, respectiv cele de execuție. În contextul aplicației nu ar trebui ca un nod master, respectiv unul de execuție să aibă un dezechilibru la nivelul numărului de clienți tratați.

Fiabilitatea introduce de asemenea cerințe de audit și monitorizare a stării, motiv pentru care, așa cum am menționat și în capitolul 2, a fost creată o logică separată pentru agregarea jurnalelor în mod optimizat. De asemenea, căderea unui nod master sau a unui nod de execuție duce la notificarea automată prin email a administratorului.

Concluzii

În cadrul lucrării de față și a aplicației dezvoltate am propus o metodă de suport pentru creatorii de programe, un loc unde ei să își poată publica micile creații și unde alți utilizatori să aibă acces la munca lor. Platforma poate deveni astfel un mediu în care să se găsească soluții la probleme diverse și să suplimenteze soluțiile particulare contribuind astfel în mod rapid la dezvoltarea serviciilor disponibile pentru care se oferă suport prin intermediul unui web browser. Prin intermediul ei consider că am reușit să rezolv o parte din deficiențele soluțiilor de executare de cod remote prezentate în capitolul 1 și să creez o soluție la o necesitate, oferirea nu doar a unui mediu în care se publică soluții al programatorilor ci și a infrastructurii de execuție, la care până în momentul finalizării acestui document nimeni nu a oferit o soluție notabilă.

Arhitectura și tehnologiile propuse au fost în mare parte eficiente, prezența unor mecanisme de descoperire automată a nodurilor au dus la posibilități mari de scalare a platformei și au facilitat separări între nodurile mai vulnerabile (cele ce execută cod) și celelalte noduri. De asemenea utilizarea containerelor Docker a adus beneficii de securitate incomparabil superioare dezavantajelor date de consumul de memorie și CPU suplimentar așa cum se poate observa din analizele făcute în capitolul anterior. Problema majoră ce a fost analizată în partea de testare legată de consumul mare de memorie și CPU pentru o conexiune WebSocket este un principal dezavantaj al alegerii acestui protocol, dar aceasta a fost de altfel singura variantă fezabilă. Mutarea platformei în cloud a fost, de asemenea, o decizie bună care a rezolvat problema lipsei unui server fizic precum și a schimbării rețelei în cadrul testării.

Problemele de securizare a aplicație la care mă așteptam de când mi-am propus să execut cod remote au fost în mare parte rezolvabile, totuși suportul slab atât la nivelul JVM cât și la nivelul execuției directe pe sistemul gazdă reprezintă impedimente din cauza cărora dacă această platformă va deveni disponibilă publicului larg metoda de execuție va fi restrânsă doar la execuția în Docker. Configurările suplimentare în cod necesare pentru SecurityManager sunt și ele un motiv pentru care execuția în JVM a fost o alegere în mare parte inoportună din punct de vedere al securității, dar foarte potrivită pentru timpul de răspuns și resursele consumate. Execuția în Docker a adus problema suportului destul de fragil în Java pentru gestionarea complexă a containerelor, volumelor precum și a unor restricții de securitate. API-ul open source Docker Java foarte slab documentat și lipsa unor informații clare despre modul de utilizare, respectiv operațiile implementate în cadrul lui m-a făcut să nu îl utilizez. Docker Engine API prin formarea unor request-uri mari, cu foarte multe configurări a dus la preferarea subproceselor în locul creării unor cereri HTTP.

Întreg contextul oferiri de suport pentru execuția la distanță este o problemă dificilă, care implică, așa cum s-a prezentat pe parcursul prezentei lucrări noțiuni de descoperire automată de noduri (dacă se dorește o scalare puternică și minimalist configurată), mecanisme de echilibrare a încărcării, toleranță la erori, jurnalizare și cel mai important și dificil, securitate. În cadrul creării acestei platforme am oferit soluții la conceptele anterior amintite documentate mai ales din domeniile diferite, întrucât așa cum am menționat și în capitolul 1.2 soluțiile existente nu au oferit prea multe detalii despre implementarea lor, element de înțeles, de altfel, în contextul securității.

Direcții viitoare de dezvoltare

Sunt trei direcții majore de dezvoltare pe care aș dori să le ofer platformei deja implementate, două care ar veni în sprijinul utilizatorului și una în sprijinul dezvoltatorilor și a celor care gestionează platforma.

Prima este crearea unei interacțiuni facile între utilizatori la nivelul comunicării, prin crearea de profiluri, comentarii, o secțiune de chat. Practic aplicația să nu rămână doar la nivelul de a găsi și executa codul care rezolvă o anumită problemă ci să devină un mediu în care oamenii să

comunica și să descopere împreună necesitățile pe care le au alte persoane.

A doua, mai ales în contextul în care, de la varianta de execuție în *JVM*, am ajuns la concluzia că singura variantă cu adevărat sigură este execuția în *Docker*, ar fi suportul multilimbaj extins. În momentul de față, se poate executa cod în limbajele *Java* și *Python*. În ciuda faptului că limbajele *Java* și *Python* sunt încă foarte utilizate, în viitor, odată cu dezvoltarea tehnologiilor, probabil topul celor mai utilizate limbaje se va schimba. Cunoștințele programatorilor, la fel ca și suportul limbajelor, sunt variate, și din acest motiv, o astfel de extindere este necesară atât la nivelul limbajelor utilizate acum, cât și a celor ce vor mai apărea, pentru a putea crea cât mai facil servicii diverse.

Platforma dezvoltată a fost una destul de complexă motiv pentru care o monitorizare a resurselor automată poate constitui un beneficiu major, în general foarte rar realizăm sau mai bine zis acceptăm anumite probleme fără dovezi clare, incontestabile. Monitorizarea consumului poate evidenția anumite probleme din aplicație care nu sunt vizibile cu ochiul liber sau poate fi punctul de plecare al alegerii optime a suportului pe care va fi executat programul.

Lecții învățate pe parcursul dezvoltării lucrării de diplomă

Am învățat în cadrul dezvoltării proiectului de diplomă faptul că în spatele tuturor aplicațiilor de executare online din browser pe care fiecare la un moment dat le-am accesat se execută procese complexe și crearea unor noi trebuie să țină cont de o multitudine de factori. În același context în ciuda faptului că adesea există o multitudine de soluții, informațiile despre ele accesibile publicului sunt limitate, în general există o tendință de secretizare a implementării, justificată totuși. Dezvoltarea suportului cloud a revoluționat modul în care companiile gestionează și accesează datele și aplicațiile lor, oferind flexibilitate și scalabilitate fără precedent, iar oamenii din acest domeniu trebuie să ofere mereu noi servicii care să le facă altora viața mai ușoară.

Bibliografie

- [1] “What is software as a service (saas),” <https://www.ibm.com/topics/saas>.
- [2] “Automatically identify nodes in your cluster,” <https://docs.aws.amazon.com/AmazonElastiCache/latest/memug/AutoDiscovery.HowAutoDiscoveryWorks.html>.
- [3] “Kubernetes auto discovery,” <https://docs.hazelcast.com/hazelcast/5.3/kubernetes/kubernetes-auto-discovery>.
- [4] “Tcp and udp discovery methods,” https://support.biamp.com/Audia-Nexia/Control/TCP_and_UDP_discovery_methods, 2020.
- [5] “Types of load balancing algorithms,” <https://www.cloudflare.com/learning/performance/types-of-load-balancing-algorithms/>.
- [6] “Fault tolerance,” <https://www.imperva.com/learn/availability/fault-tolerance/>.
- [7] “Recovering data in case of errors,” <https://www.ibm.com/docs/en/cloud-paks/cp-biz-automation/19.0.x?topic=troubleshooting-recovering-data-in-case-errors>, 2021.
- [8] “High availability versus fault tolerance,” <https://www.ibm.com/docs/en/powerha-aix/7.2?topic=aix-high-availability-versus-fault-tolerance>, 2024.
- [9] “Information security office,” <https://security.berkeley.edu/education-awareness/how-protect-against-sql-injection-attacks>.
- [10] “How to prevent sql injection: 5 key prevention methods,” <https://www.esecurityplanet.com/threats/how-to-prevent-sql-injection-attacks/>.
- [11] “xp_cmdshell (transact-sql),” <https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/xp-cmdshell-transact-sql?view=sql-server-ver16>.
- [12] “Web application attack trends,” <https://www.ptsecurity.com/ww-en/analytics/web-application-attack-trends-2017/>, 2017.
- [13] W. Zhong, “Command injection,” https://owasp.org/www-community/attacks/Command_Injection.
- [14] “Os command injection defense cheat sheet,” https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html.
- [15] “Path traversal,” https://owasp.org/www-community/attacks/Path_Traversal.
- [16] “Path traversal,” <https://portswigger.net/web-security/file-path-traversal>.
- [17] Nsrav, “Denial of service,” https://owasp.org/www-community/attacks/Denial_of_Service.
- [18] “Denial of service cheat sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Denial_of_Service_Cheat_Sheet.html.
- [19] “Distributed denial of service (ddos),” <https://www.imperva.com/learn/ddos/denial-of-service/>.

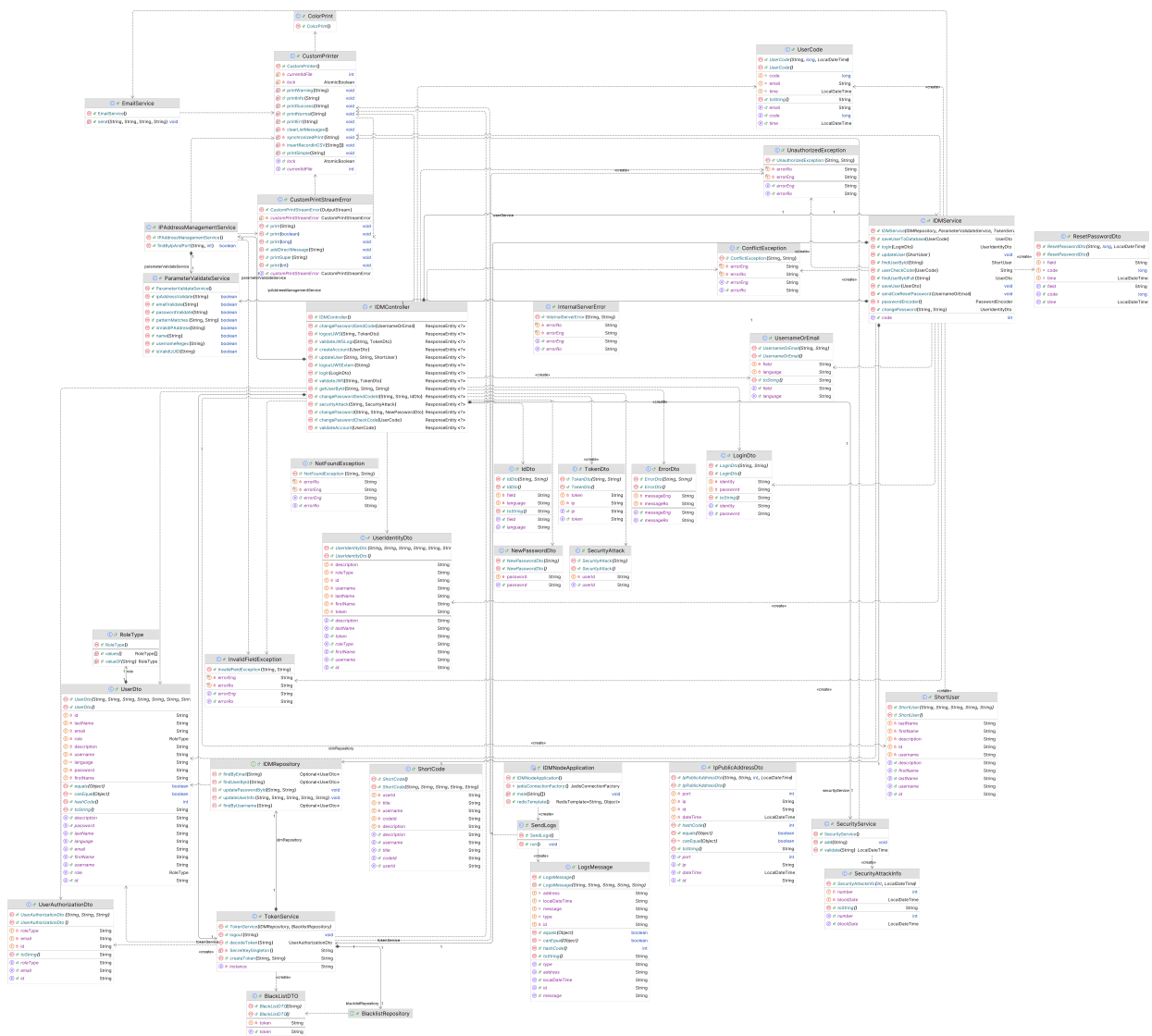
- [20] A. KL, “What is arbitrary file upload vulnerability? how to protect form it?” <https://theseckmaster.com/what-is-arbitrary-file-upload-vulnerability-how-to-protect-form-it>.
- [21] “List of file signatures,” https://en.wikipedia.org/wiki/List_of_file_signatures.
- [22] “Angular security,” <https://angular.io/guide/security>.
- [23] R. Accorsi, “Log data as digital evidence: What secure logging protocols have to offer?” Annual IEEE International Computer Software and Applications Conference, Tech. Rep., 2009.
- [24] D. M. J. Perhač and L. Rel’osky, “Distribution of linux log files described by predicate linear logic formula,” IEEE 12th International Symposium on Applied Computational Intelligence and Informatics, Tech. Rep., 2018.
- [25] C. B. Jiang, “Distributed log system in cloud digital forensics,” 2016 International Computer Symposium (ICS), Tech. Rep., 2016.
- [26] “Benefits of clustering,” https://www2.microstrategy.com/producthelp/Current/SystemAdmin/WebHelp/Lang_1033/Content/Benefits_of_clustering.htm#:~:text=Increased%20performance%3A%20Multiple%20machines%20provide%20large%20or%20rapidly%20growing%20systems.
- [27] “What are the benefits of server clustering?” <https://www.racksolutions.eu/news/blog/benefits-of-server-clustering/>.
- [28] “Cluster advantages and disadvantages,” https://docs.oracle.com/cd/E78901_01/html/E78912/goori.html.
- [29] “Secure shell (ssh) protocol,” <https://www.cloudflare.com/learning/access-management/what-is-ssh/>.
- [30] M. Horan, “Sftp vs. ftp: Understanding the difference,” <https://www.sharetru.com/blog/sftp-vs-ftp-understanding-the-difference>, 2023.
- [31] “Remote desktop protocol,” <https://learn.microsoft.com/en-us/windows/win32/termserv/remote-desktop-protocol>, 2020.
- [32] “Containerization,” <https://6sense.com/tech/containerization>, 2024.
- [33] C. Bolt, “How to run a docker container on the cloud: Top 5 caas solutions,” <https://bluelight.co/blog/how-to-run-a-docker-container-on-the-cloud>, 2024.
- [34] I. Pantsyr, “Classloaders in jvm: An overview,” <https://dzone.com/articles/classloaders-in-jvm-an-overview>, 2023.
- [35] “Apache tomcat 9 class loader how-to,” <https://tomcat.apache.org/tomcat-9.0-doc/class-loader-howto.html>, 2024.
- [36] “Create a container,” <https://docs.docker.com/engine/api/v1.45/#tag/Container/operation/ContainerCreate>.
- [37] “Java runtime.exec(),” <https://www.javaguides.net/2023/09/java-runtimeexec.html>.
- [38] “Class multicastsocket,” <https://docs.oracle.com/javase/8/docs/api/java/net/MulticastSocket.html>.

- [39] “Broadcasting and multicasting in,” <https://www.baeldung.com/java-broadcast-multicast>.
- [40] “Class serversocket,” <https://docs.oracle.com/javase/8/docs/api/java/net/ServerSocket.html>.
- [41] “Serversocket class in java,” <https://www.geeksforgeeks.org/java-net-serversocket-class-in-java/>.
- [42] “Threadpoolexecutor - java thread pool example,” <https://www.digitalocean.com/community/tutorials/threadpoolexecutor-java-thread-pool-example-executorservice>.
- [43] “Class loaders in java,” <https://www.baeldung.com/java-classloaders>.
- [44] “Send email using java program,” <https://www.geeksforgeeks.org/send-email-using-java-program/>.
- [45] “Javamail api – sending email via gmail smtp example,” <https://mkyong.com/java/javamail-api-sending-email-via-gmail-smtp-example/>.
- [46] “How to use cookies in angular,” <https://www.tutorialsworld.com/how-to-use-cookies-in-angular/>.
- [47] “Angular websockets tutorial,” <https://tutorialedge.net/typescript/angular/angular-websockets-tutorial/>.
- [48] “Real-time applications with angular and websocket,” <https://borstch.com/blog/development/real-time-applications-with-angular-and-websocket>.
- [49] “chat-websocket-application,” <https://github.com/ariphidayat/chat-websocket-application>.
- [50] “Router guards,” <https://codecraft.tv/courses/angular/routing/router-guards/>.
- [51] “A guide to the java api for websocket,” <https://www.baeldung.com/java-websockets>.
- [52] “Class inputstream,” <https://docs.oracle.com/javase/8/docs/api/java/io/InputStream.html>.
- [53] “Can a java application use more memory than the heap size,” <https://www.baeldung.com/java-memory-beyond-heap>.
- [54] “Http extensions for web distributed authoring and versioning (webdav),” <https://www.rfc-editor.org/rfc/rfc4918#page-78>, 2007.
- [55] “Hypertext transfer protocol (http/1.1): Semantics and content,” <https://datatracker.ietf.org/doc/html/rfc7231#page-59>, 2014.





57



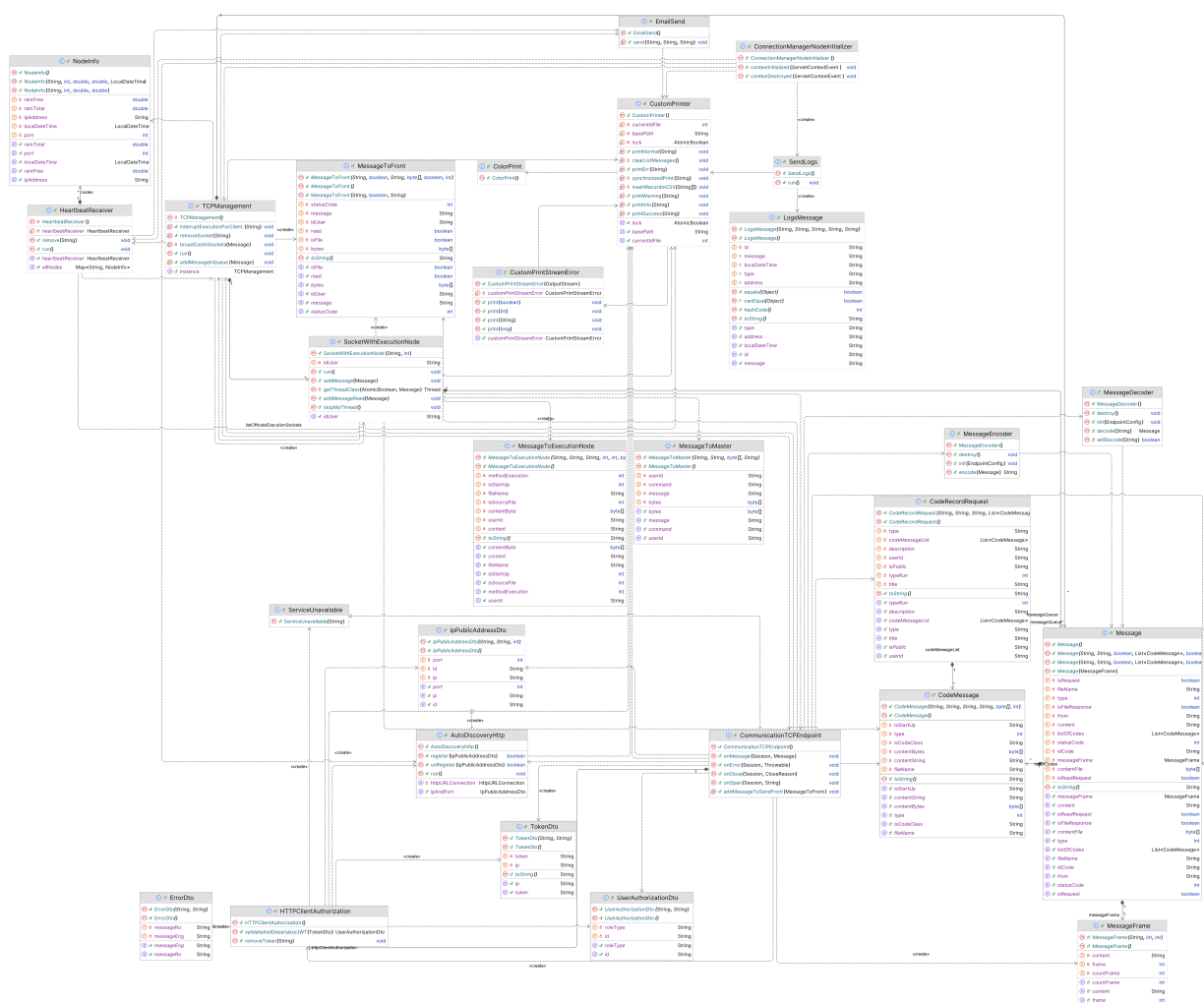


Figura A.6. Diagrama de clase pentru nodul master



Figura A.8. Diagrama de clase pentru nodul de jurnalizare

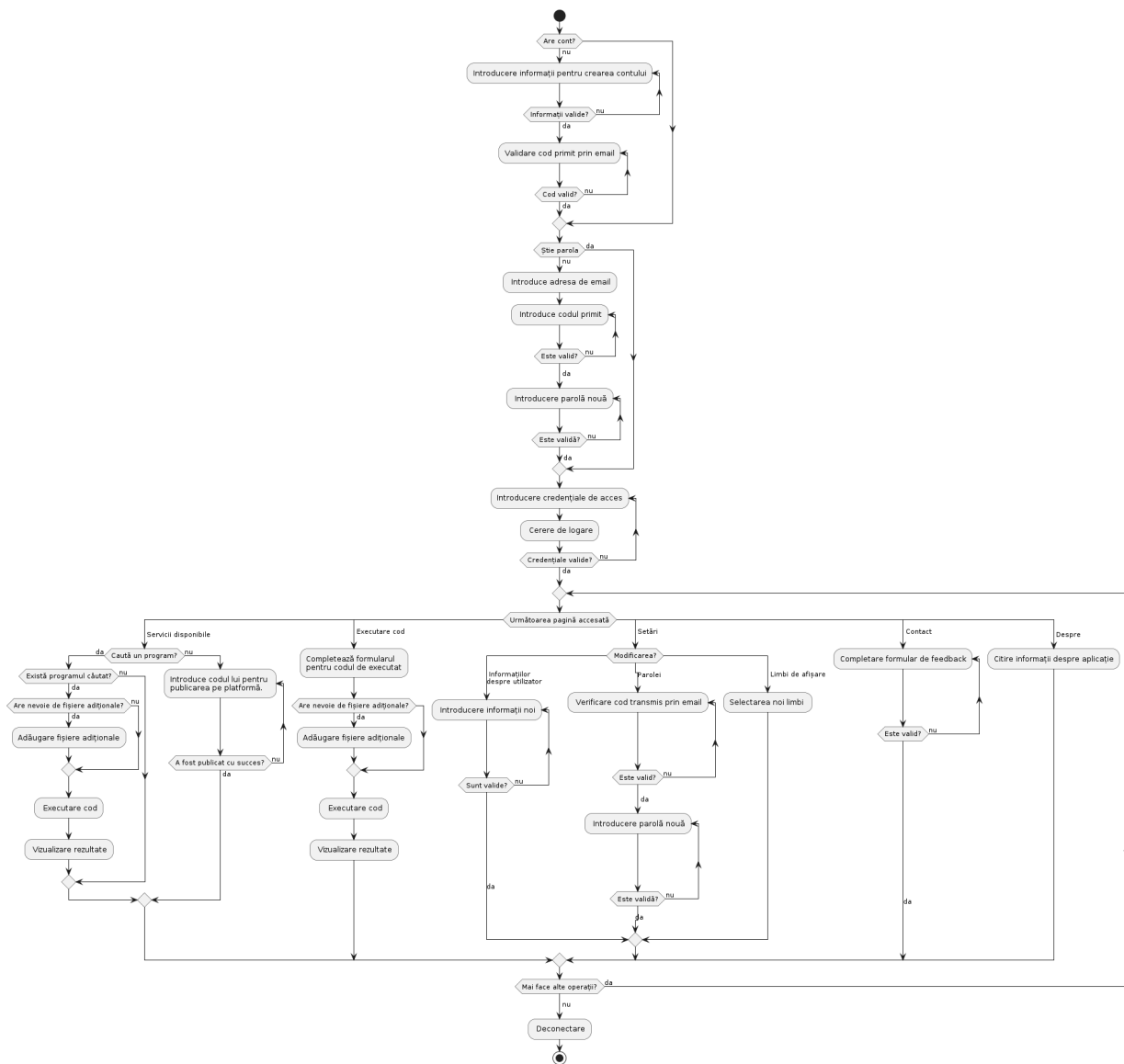


Figura A.9. Diagrama de activități

Anexa 2. Secvențe de cod

```

1  private static HttpURLConnection getHttpURLConnection() throws
    ↳ IOException {
2  String urlAddressAndPort = "44.219.73.72:8100";
3  URL url = new URL("http://" + urlAddressAndPort +
    ↳ "/api/cluster/ipAddress");
4
5  HttpURLConnection conn = (HttpURLConnection)
    ↳ url.openConnection();
6  conn.setDoOutput(true);
7  conn.setRequestMethod("DELETE");
8  conn.setRequestProperty("Content-Type", "application/json");
9
10 String token = "...";
11 conn.setRequestProperty("Authorization", token);
12 return conn;
13 }

```

Listing 1. Crearea obiectului *HttpURLConnection* pentru o cerere de delete

```

1  <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2      version="2.4" xmlns="http://java.sun.com/xml/ns/j2ee"
3      xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
    ↳ http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
4      <context-param>
5          <param-name>org.apache.tomcat.websocket.textBufferSize
    ↳ </param-name>
6          <param-value>67108864</param-value>
7      </context-param>
8      <context-param>
9          <param-name>org.apache.tomcat.websocket.binaryBufferSize
    ↳ </param-name>
10         <param-value>67108864</param-value>
11     </context-param>
12 </web-app>

```

Figura A.10. Configurare buffere pentru WebSocket-uri

```

1  public void executeCode(String filePath, String fileName) throws
    ↳ StopWithException {
2      try{
3          URLClassLoader classLoader = new URLClassLoader(new
    ↳ URL[]{new File(filePath).toURI().toURL()});
4          Class<?> loadedClass = classLoader.loadClass(fileName);
5
6          Constructor<?> constructor =
    ↳ loadedClass.getDeclaredConstructor();
7          constructor.setAccessible(true);
8          Object instance = constructor.newInstance();
9
10         loadedClass.getMethod("run").invoke(instance);

```

```
11
12         classLoader.close();
13     } catch (Throwable exception){
14         ...
15     }
16 }
```

Listing 2. Executarea codului din fișiere .class locale

```
1  @Override
2  public Class<?> findClass(String name) {
3      byte[] classBytes = code.getContent();
4      try {
5          Class<?> loadedClass = this.listOfClass.get(name);
6          if (loadedClass != null) {
7              return loadedClass;
8          } else {
9              CustomPrinter.println("Defined new class: " + name);
10             return defineClass(name, classBytes, 0,
11                               ↪ classBytes.length);
12         }
13     } catch (Throwable exception) {
14         ...
15     }
16     return null;
17 }
```

Listing 3. Metoda de creare a unei clase pe baza conținutului

```
1  public void execute(ByteCode code) throws StopWithException {
2      try {
3          CustomPrinter.println("Run class : " +
4                               ↪ code.getFileName());
5          this.code = code;
6          Class<?> myClass;
7          if (!listOfClass.containsKey(code.getFileName())) {
8              myClass = this.findClass(code.getFileName());
9              this.listOfClass.put(code.getFileName(), myClass);
10         } else {
11             myClass = this.listOfClass.get(code.getFileName());
12         }
13         Constructor<?> constructor =
14             ↪ myClass.getDeclaredConstructor();
15         constructor.setAccessible(true);
16         Object instance = constructor.newInstance();
17         myClass.getMethod("run").invoke(instance);
18     } catch (Throwable exception) {
19         ...
20     }
21 }
```

Listing 4. Executarea codului din clasa CodeExecutorByteCode

```

1  public void executeCode(String filePath,String runClassName) throws
    ↳ StopWithException {
2      try{
3          URLClassLoader classLoader = new URLClassLoader (new
    ↳ URL[]{new File(filePath).toURI().toURL()},
    ↳ this.getClass().getClassLoader());
4          Class<?> loadedClass = Class.forName (runClassName, true,
    ↳ classLoader);
5          Constructor<?> constructor =
    ↳ loadedClass.getDeclaredConstructor();
6          constructor.setAccessible(true);
7          Object instance = constructor.newInstance();
8          loadedClass.getMethod("run").invoke(instance);
9          classLoader.close();
10     } catch (Throwable exception){
11         ...
12     }
13 }

```

Listing 5. Executarea codului din fişiere JAR

```

1  CREATE USER 'userCRUD'@'%' IDENTIFIED BY 'password';
2  GRANT SELECT, INSERT, UPDATE, DELETE ON cluster.user TO
    ↳ 'userCRUD'@'%';
3  GRANT SELECT, INSERT, UPDATE, DELETE ON cluster.code_message TO
    ↳ 'userCRUD'@'%';
4  GRANT SELECT, INSERT, UPDATE, DELETE ON cluster.feedback_dto TO
    ↳ 'userCRUD'@'%';
5  GRANT SELECT, INSERT, UPDATE, DELETE ON cluster.code_record TO
    ↳ 'userCRUD'@'%';
6  FLUSH PRIVILEGES;

```

Listing 6. Creare utilizatorului pentru baza de date MariaDB

```

1  "post": {
2      "tags": [
3          "idm-controller"
4      ],
5      "operationId": "createAccount",
6      "requestBody": {
7          "content": {
8              "application/json": {
9                  "schema": {
10                     "$ref": "#/components/schemas/UserDto"
11                 }
12             }
13         },
14         "required": true
15     },
16     "responses": {
17         "201": {
18             "description": "User created!",

```

```

19     "content": {
20         "application/json": {
21             "schema": {
22                 "$ref": "#/components/schemas/ShortUser"
23             }
24         }
25     },
26 },
27 "422": {
28     "description": "Invalid field!",
29     "content": {
30         "application/json": {
31             "schema": {
32                 "$ref": "#/components/schemas/ErrorDto"
33             }
34         }
35     },
36 },
37 "409": {
38     "description": "Conflict!",
39     "content": {
40         "application/json": {
41             "schema": {
42                 "$ref": "#/components/schemas/ErrorDto"
43             }
44         }
45     },
46 },
47 }
48 }

```

Listing 7. Fragment din documentul OpenAPI

```

1  if (this.type) {
2      // static
3      if (currentNode >= listOfNodeExecutionSockets.size() || currentNode
4          < 0) {
5          currentNode = 0;
6      }
7      String keyNode = keys.get(currentNode);
8      currentNode++;
9      currentNode %= listOfNodeExecutionSockets.size();
10     socket = listOfNodeExecutionSockets.get(keyNode);
11 } else {
12     // dynamic
13     Map < String, NodeInfo > map = HeartbeatReceiver.getAllNodes();
14
15     List < Map.Entry < String, NodeInfo > > nodesByRam = new ArrayList
16         < > (map.entrySet());
17     nodesByRam.sort((a, b) ->
18         Double.compare(b.getValue().getRamFree(),
19             a.getValue().getRamFree()));

```

```

17
18 List < Map.Entry < String, NodeInfo >> nodesByCpu = new ArrayList
    ↪ < > (map.entrySet());
19 nodesByCpu.sort((a, b) ->
    ↪ Double.compare(b.getValue().getProcessors() -
    ↪ b.getValue().getLoadAverage(), a.getValue().getProcessors() -
    ↪ a.getValue().getLoadAverage()));
20
21 Map < String, Integer > scores = new HashMap < > ();
22
23 for (int i = 0; i < nodesByRam.size(); i++) {
24     String nodeName = nodesByRam.get(i).getKey();
25     scores.put(nodeName, i + 1);
26     CustomPrinter.printNormal("RAM: " + nodeName + " : " + (i +
    ↪ 1));
27 }
28
29 for (int i = 0; i < nodesByCpu.size(); i++) {
30     String nodeName = nodesByCpu.get(i).getKey();
31     scores.put(nodeName, scores.getOrDefault(nodeName, 0) + (i +
    ↪ 1));
32     CustomPrinter.printNormal("CPU: " + nodeName + " : " + (i +
    ↪ 1));
33 }
34
35 String bestNode = null;
36 int bestScore = Integer.MAX_VALUE;
37
38 for (Map.Entry < String, Integer > entry: scores.entrySet()) {
39     CustomPrinter.printSuccess("Score for: " + entry.getKey() + "
    ↪ : " + entry.getValue());
40     if (entry.getValue() < bestScore) {
41         bestScore = entry.getValue();
42         bestNode = entry.getKey();
43     }
44 }
45
46 if (bestNode == null) {
47     // ..
48     socket = listOfNodeExecutionSockets.get(bestNode);
49 }
50 }

```

Listing 8. Echilibrarea încărcări pe nodurile de execuție

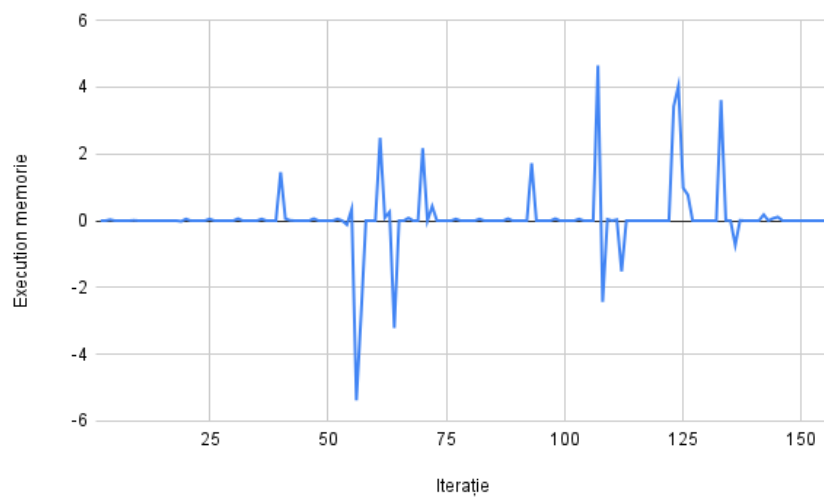
Anexa 3. Grafice suplimentare

Figura A.11. Modificările consumului de memorie la nivelul nodului de execuție la rularea de cod în Docker

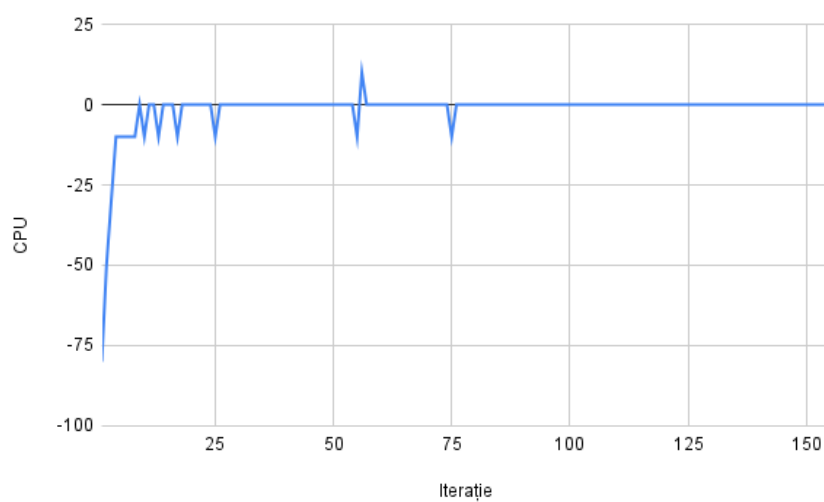


Figura A.12. Modificările consumului de CPU la nivelul nodului de execuție la rularea de cod în Docker

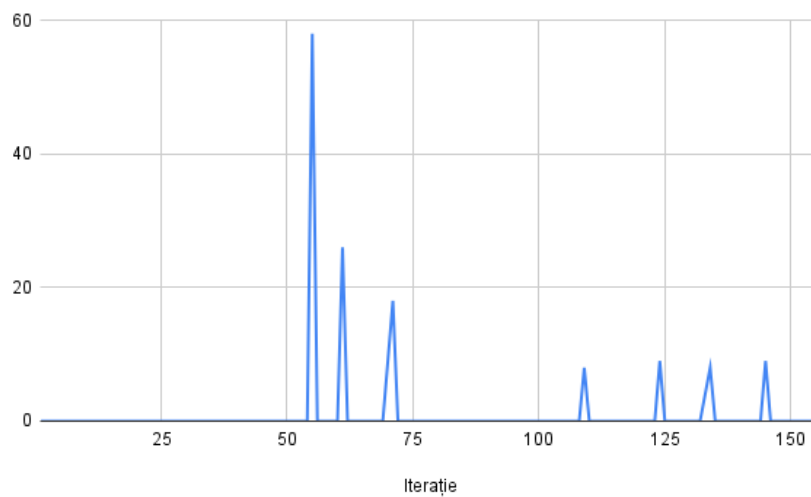


Figura A.13. Modificările consumului de CPU la nivelul containerului la rularea de cod în Docker

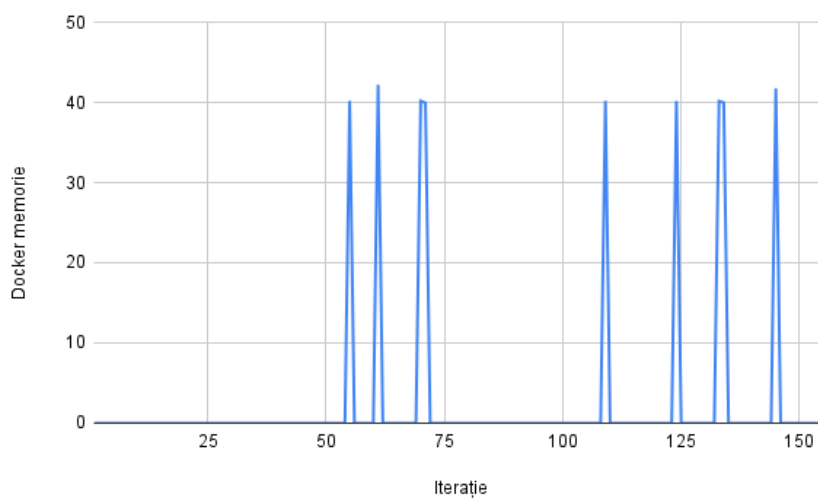


Figura A.14. Modificările consumului de memorie la nivelul containerului la rularea de cod în Docker

